Expert Systems With Applications 280 (2025) 127606

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

A new method for load balancing in DSDN-Based data centers using adaptive clustering and normal Cone-Based estimation approaches

Marjan Mahmoudi [a], Behrang Barekatain [a,b,*], Zahra Beheshti [a,b], Alfonso Ariza Quintana [c,a], Mohammad Rasoul Velayati [d,b]

[a] Faculty of Computer Engineering, Najafabad Branch, Islamic Azad University, Najafabad, Iran
[b] Big Data Research Center, Najafabad Branch, Islamic Azad University, Najafabad, Iran
[c] E.T.S.I. Telecomunicación, Universidad de Málaga, Spain
[d] Department of Mathematical, Faculty of Computer Engineering, Najafabad Branch, Islamic Azad University, Najafabad, Iran

ARTICLE INFO

Keywords:
Distributed Software Defined Networking
Clustreing
Load balancing
Fog Computing
Virtual Machine
Migration
Comparative Algebra
Cone Estimation Function

ABSTRACT

According to recent literature, appropriate load balancing in fog Distributed Software-Defined Network (Fog-DSDN) not only leads to optimal resource utilization, increased network efficiency, and reduced response latency but also realizes scalability and reduced traffic bottleneck. Load balancing provides the conditions for affording real-time responses to user requests effectively and dynamically. To address this challenge, this study proposes an interactive method based on adaptive algebra in density clustering and data rules. The proposed method called CAAF-DSDN, applies a factorization-based neighborhood distribution estimation method to cluster data and determine the similarity threshold. Using the normal cone approximation function, this method introduces a measure for ranking servers regarding processing capacity matching. In the migration process, the selected server should be moved to a low-traffic network region. The CAAF-DSDN method considers the interactions between various factors including load balancing (LB), network efficiency, response time, migration cost, and network latency. This is realized using control layer data and updating virtual machines (VMs) to extract optimal solutions. Simulations in OMNET++ showed that the proposed method leads to an average of 15.13 %, 18.73 %, and 14.50 % improvement in network latency, load balancing, and response time, respectively, compared to BMU-COA, FPTA and RL-Driven Decision methods.

1. Introduction

Due to the continuous increase in user demand, the number of smart devices, and the diversity of applications, energy efficiency management has become a critical priority in the development of energy efficiency solutions. The rapid development of computer networks and information processing has strengthened the need for efficient energy management. One of the methods for optimizing energy consumption in networks is to realize intelligent dynamic load balancing in the network, defined as the optimal allocation of network resources to respond to workload changes. This method can reduce energy consumption and improve network efficiency. Unlike software-defined networks (SDNs), the capability of distributed SDN (DSDN)-based networks provides conditions for more optimal implementation of load balancing (Alhilali & Montazerolghaem, 2023; Malakar, Mahapatro, & Ghosh, 2024). Optimizing energy efficiency in DSDN requires careful analysis of workload distribution models. This can be realized by using intelligent mathematical algorithms and network management tools.

Repetitive activities in the FOG DSDN architecture lead to increased processing time, contrary to the objective of improving efficiency. To address this challenge, dynamic load balancing and virtualization in data centers are implemented within the FOG DSDN framework. The use of intelligent and adaptive algorithms ensures dynamic load distribution among active devices. The key objective is to allocate the most optimal number of active devices to process network traffic and deactivate semi-active devices to increase energy efficiency. Another effective solution to improve energy efficiency and network resources is to modify the network infrastructure (Arefian, Khayyambashi, & Movahhedinia, 2023; Javanmardi, Shojafar, Mohammadi, Persico, & Pescapè, 2023). The virtual machine (VM) migration has also been proposed as a suitable method for managing, reducing response time, increasing performance, and load balancing in data centers. According to recent research,

* Corresponding author.
E-mail address: Behrang_Barekatain@iau.ac.ir (B. Barekatain).

https://doi.org/10.1016/j.eswa.2025.127606
Received 17 December 2024; Received in revised form 22 March 2025; Accepted 4 April 2025
Available online 5 April 2025
0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

dynamic load balancing can be used as an effective solution to increase network efficiency. Resource distribution in the network should be done in such a way that requests are distributed appropriately among devices. Implementing dynamic load balancing in networks requires intelligent algorithms to optimize resource and load distribution. Such algorithms can be selected based on various criteria, including appropriate clustering of VMs, detection of when to migrate tasks or VMs before overload, device capacity, latency, and traffic patterns. To achieve load balancing and energy efficiency in FOG DSDN, certain requirements must be addressed (Jasim & Al-Raweshidy, 2024; Salehnia, Montazerolghaem, Mirjalili, Khayyambashi, & Abualigah, 2024), including load balancing algorithms, traffic scheduling(Farhan, Shah, Wang, Muntean, & Song, 2024; Gunavathie & Umamaheswari, 2024), direct connection to application servers, support for network equipment, continuous monitoring and management, etc.

In particular, in Fog DSDN architecture, the imbalance in the distribution of incoming traffic between VMs leads to congestion and overload (Ebneyousef & Shirmarz, 2023; Ebrahim & Hafid, 2023; Rezaee, Hamid, Hussin, & Zukarnain, 2024). In addition, implementing density-based clustering methods under the supervision of a controller can be effective in this context. However, these methods face certain limitations such as the appropriate:

- selecting key parameters such as Epsilon for similarity,
- determining the minimum points in a cluster (MinPts),
- detecting outliers,
- dealing with low-density regions,
- identifying data far from clusters as noise,
- identifying data that does not fit into clusters as noise,
- aggregating data as points in different regions, and forming ambiguous clusters.

Such matters can lead to significant difficulties in the effective implementation of density-based clustering algorithms. In fact, the incorrect selection of relevant parameters can lead to inaccurate results. To address these challenges and solve one of the biggest network problems, namely load balancing, a method called cluster-aware adaptive method for distributed software-defined networking (CAAF-DSDN) is introduced in this paper. To reduce response time, increase network efficiency, optimize productivity management, reduce network complexity, and balance network-wide load, a new method based on mathematical algorithms is introduced to solve the NP-Hard problem.

The migration of Virtual Machines (VMs) and load balancing in Software-Defined Networking (SDN) and Fog Computing environments are crucial challenges in the optimal management of resources, minimizing network latency, and enhancing the performance of time-sensitive services. In this context, various strategies have been proposed to optimize resource allocation and VM migration, each offering distinct advantages and limitations.

A significant approach in this field is the Updated Blue Monkey Hybrid Optimized Chimpanzee Algorithm (BMUCOA), which integrates two heuristic optimization algorithms to improve the efficiency of VM migration in SDN environments (Sharma & Seth, 2024). This method enhances resource utilization and minimizes migration costs by analyzing network traffic loads and redistributing traffic from high-load nodes to lower-load ones. However, the combination of two optimization algorithms introduces higher computational complexity, which may adversely affect performance in large-scale environments. Another notable approach involves a Mixed Integer Programming (MIP) model proposed to determine the optimal migration sequence and bandwidth allocation (Huandong Wang, Li, Zhang, & Jin, 2017). This model utilizes a Fully Polynomial-Time Approximation Algorithm (FPTA), which significantly reduces computation time and optimizes VM migration by up to 40 % relative to previous techniques. Nonetheless, the NP-Hard complexity of the problem causes implementation challenges in large-scale environments. Moreover, the impact of network fluctuations on its performance has not been comprehensively assessed. A reinforcement learning-based model for live VM migration in Fog Computing environments has been also proposed (Alqam, Alzeidi, Touzene, & Day, 2023). This approach optimizes decision-making by considering multiple influencing factors simultaneously, thereby reducing latency for time-sensitive services. However, this method requires meticulous fine-tuning of reinforcement learning parameters and suffers from high computational complexity. Moreover, a comprehensive comparative analysis with other AI-driven algorithms, such as deep learning and heuristic optimization, remains absent.

In order to addresss rhese issues, this paper introduces a novel method titled Cluster-Aware Adaptive Method for Distributed Software-Defined Networks (CAAF-DSDN), which overcomes the limitations of existing approaches while providing more efficient capabilities for VM migration and resource management in distributed SDN and Fog Computing environments. The main innovations of CAAF-DSDN (our proposed method), as compared to prior research, can be categorized as follows:

1. Adaptive Algebraic Clustering: In contrast to the BMUCOA(Sharma & Seth, 2024) method, which combines multiple optimization algorithms resulting in increased computational complexity, the proposed method employs adaptive algebraic modeling for data clustering, along with defining global sets and neighborhood rules. This approach reduces computational overhead while enhancing the accuracy of network traffic identification and differentiation.
2. Optimization of Data Similarity Detection: While the FPTA (Huandong Wang et al., 2017) method, focuses primarily on bandwidth allocation, the proposed method uses a set of algebraic rules and neighborhood distribution estimation to determine data similarity thresholds. This technique improves the accuracy of identifying similar data and reduces both processing time and decision-making latency in resource allocation.
3. Server Ranking Algorithm Based on Normal Cone Function: Unlike the reinforcement learning model (Alqam et al., 2023) which relies on complex parameter tuning, the proposed method leverages an optimization algorithm to rank servers based on fitness values. This method optimizes load distribution and reduces both computational and migration costs by selecting low-traffic servers for VM migration.
4. Comprehensive Analysis of the VM Migration Process: Previous methods typically address one of cold or warm migration types. However, CAAF-DSDN offers an integrated analytical framework that evaluates both migration types, using multidimensional criteria to select the most optimal destination for migration.
5. Reduction of NP-Hard Complexity in Network Resource Allocation: In contrast to the FPTA model (Huandong Wang et al., 2017) which faces computational challenges at large scales, the proposed method reduces processing time and improves decision-making accuracy through advanced mathematical modeling and hybrid optimization algorithms.
6. Integration with SDN Controllers: Unlike previous studies, this method optimizes the server listing and selection process within distributed SDN controllers, leading to reduced traffic, enhanced server communication efficiency, and increased network link capacity.

As a result, this paper presents a novel approach for adaptive VM migration and resource management in SDN and Fog Computing environments, addressing the limitations of previous methods. The proposed method, which incorporates algebraic models, adaptive clustering, neighborhood distribution estimation, and intelligent server ranking, results in reduced response times, improved load balancing, lower computational costs, and enhanced system efficiency. A comparison of the proposed method with previous models demonstrates significant improvements in key performance indicators (KPIs) such as migration time, processing delay, resource utilization, and migration costs.

Table 1
Studies based on ant colony algorithm for load balancing in SDN and their attributes.

Reference	Main Objective	Advantages	Disadvantages
2016 (Hu et al., 2016)	Maintaining link load balance and reducing end-to-end transmission delay	• Network link load balancing • Improved Quality of Service (QoS) • Reduced network overhead	• Susceptible to local optima • Does not consider delay • Does not account for throughput
2016 (W. Lin & Zhang, 2016)	Developing a dynamic load-balancing algorithm with task classification	• Enhanced load balancing	• Susceptible to local optima • High overhead
2016 (Sathyanarayana & Moh, 2016)	Load balancing of servers and paths leading to servers	• High network throughput • Low latency	• Single-controller bottleneck • Lack of scalability • Lack of availability
2019 (Xue et al., 2019)	Dynamic load balancing to further enhance SDN performance	• Higher load balancing • Reduced response time	• Lacks real-world evaluation • Does not assess packet loss rate • Does not evaluate throughput

Table 2
Genetic algorithm-based studies for load balancing in SDN and their attributes.

Reference	Main Objective	Advantages	Disadvantages
2014(Chou et al., 2014)	Algorithm for balancing workload among multiple controllers	Improved load balancing	• Lack of delay consideration • Lack of overhead consideration • High complexity
2016 (Kang & Kwon, 2016)	Addressing the single point of failure in SDN	Improved failure recovery time	• Low load balancing degree • High controller-switch delay • Lack of runtime analysis • Poor local search • No comparison with other methods • High network complexity due to multiple controllers
2016 (Fang, Wang, & Wang, 2016)	Load balancing in elastic optical networks	Improved load balancing	• Slow speed, lack of scalability • Unavailability • No consideration of packet loss rate
2019 (Lambora, Gupta, & Chopra, 2019)	OpenFlow-based load balancing system	High performance	• Lack of scalability • Unavailability • No overhead consideration • High computation

Implementing this method in distributed SDN and Fog Computing environments can notably enhance network performance and reduce latency in time-sensitive services. In light of the above, the contributions in the fields of data clustering, network resource optimization, and improving migration processes in complex networks can be generalized to other research areas. These advancements provide valuable insights that can be applied to various research areas facing similar challenges and systems.

The rest of the paper is organized based on a systematic method as follows. Section 2 is a review of the current state-of-the-art of energy efficiency and load balancing in Fog DSDN architecture. Section 3 describes the proposed method, its principles, and its specific algorithms, focusing on load balancing and cost optimization. Section 4 analyzes the performance through simulation and testing and provides specific practical insights. Section 5 highlights the study's conclusion and future work.

2. Related work

This section reviews the research related to load balancing in SDN networks. Wang et al. (C. Wang, Zhang, Xu, & Chen, 2016) presented an algorithm for link load balancing based on ant colony optimization (ACO). Based on the ACO search principles, this algorithm considers link load, delay, and missing packets as influencing factors in selecting the next node. The objective of this algorithm is to maintain link load balance and reduce end-to-end transmission delay. Basically, ants search for the widest and shortest path among all possible paths. Lin et al. (W. Lin & Zhang, 2016) proposed a dynamic algorithm for load balancing and task classification based on an ant colony algorithm in SDN cloud computing platforms. This algorithm can classify server nodes with similar processing capabilities into small cluster subnets. Sathyanarayana et al. (Sathyanarayana & Moh, 2016) presented a dynamic load-balancing method for SDN focusing on balancing the load of servers and the paths leading to the servers. To select the best server, the load balancing policy of the server with the least load is used. The selection of the best path to reach the selected server is realized by ACO. Xue et al. (Xue, Kim, & Youn, 2019) introduced a new scheme for dynamic load balancing using ACO with significant improvement in SDN performance and reduced response time. However, the testbed and the packet loss rate and throughput have not been evaluated realistically. The advantages and disadvantages of studies based on ant colony algorithms are summarized in Table 1.

Hu et al. (Hu et al., 2016) have investigated the challenges and solutions associated with maintaining consistency during the virtual machine (VM) migration process in Software-Defined Networks (SDNs). Their work primarily focuses on proposing methods to ensure the consistency of network policies and effective resource management during VM migrations, with the goal of optimizing performance and minimizing disruptions in the network. Mahlab et al. (Mahlab et al., 2017) introduced an OpenFlow-based load-balancing system using a genetic algorithm (GA). The evaluation of the study results shows the superior performance of the proposed method. However, constraints such as the use of arithmetic mean for the coefficient of variation metric and scalability and availability matters remain unresolved. The overload and efficiency of the algorithm have not been properly investigated and have imposed high computational time. Chou et al. (Chou, Yang, Hong, Hu, & Jean, 2014) proposed a GA-based load-balancing algorithm for SDN focused on balancing the load between multiple controllers. However, this algorithm was only evaluated regarding the load.

balancing factor. Critical metrics such as latency and overload were not considered. Kang et al. (Kang & Kwon, 2016) addressed the single point of failure in SDN and proposed a new method for fast and load-aware control. The researchers used GA to develop a near-optimal recovery scheme and achieved improvements in reducing the failure time and load balancing of the controller. However, the load balancing realized is insignificant compared to other methods and imposes a higher controller switching delay. Also, other metrics such as running time have not been evaluated and the weaknesses of the proposed method in local search are evident. The following Table 2 presents the advantages and disadvantages of genetic algorithms.

Table 3
Studies based on particle swarm algorithm for load balancing in SDN and their attributes.

Reference	Main Goal	Advantages	Disadvantages
2016 (He et al., 2016)	Focus on intelligent transportation system and Internet of Vehicles	• Improved delay	• Lack of QoS analysis
2021 (Albowarab et al., 2021)	Addressing load balancing challenges in SDN	• Availability • Improved load balancing	• Lack of overhead analysis • Lack of delay analysis • Lack of throughput analysis
2022 (Nagasri & Anbarasu, 2022)	Solving the problem of unbalanced network infrastructure communication in mobile networks	• Improved response time • Improved throughput • Enhanced resource utilization	• Scalability • Availability • Lack of runtime analysis

Albowarab et al. (Albowarab, Zakaria, & Zainal Abidin, 2021) introduced a mathematical formula to solve load balancing challenges in SDN and subsequently proposed two models of particle swarm optimization (PSO) algorithm. Unlike models based on the congestion distance for selecting solutions, the models proposed in this study combine both the congestion distance and congestion angle. Specifically, only solutions with a congestion distance probability higher than 0.5 and a higher angular distribution are selected. These models use a centralized controller, which leads to reduced availability and scalability. In addition, the overload, latency, and capacity of the algorithm are not considered. Ren et al. (He, Ren, Shi, & Fang, 2016) focused on intelligent transportation systems and the Internet of Things (IoT). Although the proposed solution combines fog computing, cloud computing, and SDN to improve latency and location-aware activation, critical criteria such as optimal utilization and load balancing are not considered. Quality of service parameters such as capacity and security are also not considered. Nagasri et al. (Nagasri & Anbarasu, 2022) developed a shortest path algorithm to regularly calculate topological information in SDN nodes. A congestion control strategy based on the OpenFlow protocol is proposed to realize load balancing in software-defined mobile networks. The advantages and disadvantages of the related papers on the ant colony algorithm are summarized in Table 3.

Neghabi et al. (Neghabi, Navimipour, Hosseinzadeh, & Rezaee, 2020), proposed a new dynamic routing method to solve link load balancing problems using multi-objective artificial bee colony algorithms and genetic operators. Nagasri et al. (Nagasri & Anbarasu, 2022), the same researchers analyzed the multi-path routing and load-balancing processes in SDN using the bee colony algorithm. Sridevi et al. (Sridevi & Saifulla, 2023), introduced a distributed load balancing framework for SDN controllers, leveraging Artificial Bee Colony Optimization (ABC) to enhance the network's efficiency. Their approach optimizes control load distribution, reduces latency, and mitigates congestion, contributing to better scalability and reliability. However, the proposed method has limitations, particularly in its computational complexity, which poses challenges for deployment in large-scale SDN scenarios. These trade-offs highlight the need for further refinement to ensure broader applicability while maintaining performance improvements. Kurroliya et al. (Kurroliya, Mohanty, Kanodia, & Sahoo, 2020) a new mechanism for controlling plane load balancing in SDN is presented. A new algorithm for edge weight optimization based on the gray wolf algorithm is introduced, focusing on load balancing optimization in SDN networks.

Min et al. (Min, Sun, Bao, Gokhale, & Gokhale, 2021), modeled and optimized service-level and system-level objectives in dynamic Internet of Things (IoT) scenarios using M/M/1 queues. This study includes several heuristic solutions, including a genetic algorithm, simulated annealing algorithm, and modified greedy algorithm, to reduce the queue length and processing time of requests, as well as to balance the load of controllers and consider the migration of switches.

Hsieh et al. (Hsieh & Wang, 2019) proposed a simulated annealing mechanism implemented on global controllers to reduce the load of local controllers and thus enhance the availability of hierarchical SDN controllers. Obaida et al. (Obaida & Salman, 2022) used the SDN platform to implement the firefly optimization algorithm to determine the shortest path between two nodes in the network. The results of this algorithm provide shorter paths and improved load balancing. Guan et al. (Y. Li, Sun, & Guan, 2019) proposed the deployment of multiple firefly-based controllers focusing on optimizing the average delay and controller load difference. Li et al. (G. Li, Li, Liu, & Pan, 2019) proposed an efficient dynamic load balancing scheme using an improved firefly algorithm for SDN networks. In the first step, switch migration is transformed into a network mapping relation reconstruction problem considering the connectivity of network nodes. Then, the proposed model is developed to optimize two conflicting objectives, namely migration cost and load balancing.

Kim et al. (Sejun Kim, Ebay, Lee, Kim, & Youn, 2019) proposed a new load balancing scheme with switch migration to effectively solve the dynamic controller provisioning problem in SDN. This scheme uses a *meta*-heuristic approach based on harmony search and K-means clustering to manage switches. Birzhandi et al. (SangYoun Kim, Kim, Birzhandi, Li, & Youn, 2019) proposed another load-balancing scheme for solving controller provisioning in SDN using harmony search to maximize the accuracy of finding the optimal solution and reduce the search time. Wang et al. (Haibo Wang et al., 2020) focused on data-level load balancing using SDN switches and introduced a system called PrePass to address load-balancing challenges without requiring additional hardware and software on the switches. Montazer Al-Qaem et al. (Montazerolghaem & Yaghmaee, 2020) studied the QoS and load balancing challenges in the IoT and proposed an SDN-based framework to satisfy the QoS requirements. They also used a predictive discovery mechanism based on time series analysis and fuzzy logic to realize traffic balancing between IoT service providers.

Ateya et al. (Ateya et al., 2019) proposed a similar chain jamming algorithm (SSOA) to incorporate multiple controllers in SDN networks, focusing on introducing a mapping process to optimize the performance of SDN networks. Polarakis et al. (Poularakis, Qin, Nahum, Rio, & Tassiulas, 2019) presented a flexible controller for SDN networks based on Ad Hoc networks. A highly flexible protocol was developed to divide the control of Ad Hoc networks between SDN central controllers and data-level nodes. Lin et al. (F. P.-C. Lin & Tsai, 2019) proposed a new method based on hierarchical load sharing using edge pruning in SDN cloud networks. Prabhakaran et al. (Prabakaran & Ramar, 2021) compared two load-balancing algorithms with distributed controller architectures. The results showed that the flow-based load balancing algorithm can reduce the response time by 94 % and increase the data exchange rate by 14 %. Meanwhile, the traffic pattern-based load balancing algorithm improves the availability by 2.69 %. Chen et al. (Chen et al., 2022) introduced an automatic load-balancing architecture based on reinforcement learning in SDN and used an optimized model for heavy traffic scenarios. Also, an improved deep deterministic policy gradient (DDPG) was used to find the optimal path between network hosts.

Sun et al. (Sun, Guo, Wang, Lan, & Hu, 2020) proposed a scheme called enabling controller load balancing in software-defined networks with multi-agent reinforcement learning (MARVEL) for load balancing of controllers in SDN using multi-agent reinforcement learning. Chakraverti et al. (Chakravarthy & Amutha, 2022) evaluated the impact of using SDN services on the performance of various multimedia applications and implemented and validated a genetic load balancing algorithm

Table 4
Studies on load balancing in SDN and related attributes.

Reference	Main Objective	Disadvantages	Advantages
2019 (G. Li et al., 2019)	Efficient dynamic load balancing scheme	• Challenges with real-time performance • Overhead from negotiation • Convergence time • Computational complexity • Limited real-world testing	• Improved load balancing efficiency • Dynamic adaptability • Fairness • Reduced latency • Energy efficiency
2019 (Y. Li et al., 2019)	Optimizing average control delay and load imbalance	• Scalability concerns with very large networks • Overhead in dynamic environments • Dependency on initialization	• Nature-inspired optimization • Improved controller placement • Global and local search balance
2019 (SangYoun Kim et al., 2019)	Load balancing scheme for effective controller provisioning	• Local optima • Adaptation in highly dynamic environments • Parameter sensitivity • Computational complexity	• Optimized controller provisioning • Improved network performance • Fault tolerance
2019 (Sejun Kim et al., 2019)	Novel load balancing scheme with switch migration for dynamic controller provisioning	• Risk of local optima • Dependency on accurate network data • Overhead from adaptation • Convergence time	• Efficient load distribution • Dynamic adaptation • Simplicity in implementation
2019 (Hsieh & Wang, 2019)	Reducing local controller load in hierarchical SDN controllers	• High computational cost • Slow convergence • Sensitivity to initial conditions • Implementation complexity • Limited real-world validation	• Improved fault tolerance • Proven optimization technique • Optimal failover placement • Reduced downtime • Scalability
2020 (Neghabi et al., 2020)	Solving the link load balancing problem	• Computational complexity • Sensitivity to parameter tuning • Overhead from optimization	• Multi-objective optimization • Use of genetic operators • Robust optimization • Improved network performance
2020 (Kurroliya et al., 2020)	Load balancing in SDN networks	• Computational complexity • Convergence time • Parameter sensitivity • Dependency on real-time data	• Energy-saving • Real-time traffic adaptation • Load balancing • Improved network resilience
2021 (Min et al., 2021)	Modeling and optimizing system and service-level objectives in DSDN	• Computational overhead • Dependency on real-time data • Potential instability	• Self-adaptability • Minimal manual intervention • Enhanced Quality of Service (QoS)
2022 (Perepelkin & Nguyen, 2022)	Analysis of multipath routing and load balancing in SDN	• Computational complexity • Risk of local optima • Limited real-world validation • Implementation complexity	• Optimized multipath routing • Improved load balancing • Adaptability to network dynamics
2022 (Obaida & Salman, 2022)	Shortest path determination between two nodes in a network	• Computational overhead • Dependency on initialization • Real-time adaptation challenges	• Optimal path selection • Swarm intelligence • Dynamic adaptability • Flexibility in objectives
2022 (Guan, Li, Li, & Wang, 2022)	Optimized Firefly Algorithm for controller placement problem	• Real-time adjustments may add overhead • Dependence on accurate network data	• Robust optimization • Reduced latency and improved QoS • Adaptability to dynamic environments
2022 (Bakhtiari & Guran Ourimi, 2022)	Dynamic SDN switch migration scheme for load balancing with low migration cost	• Computational complexity • Parameter sensitivity • Overhead from switch migration • Dependency on accurate network data	• Improved load balancing • Dynamic switch migration • Adaptability to network changes
2023 (Sridevi & Saifulla, 2023)	Focused on reducing the load on overloaded controllers	• Computational overhead • Convergence time • Risk of local optima • Switch migration overhead • Integration complexity	• Effective load balancing • Dynamic adaptability • Energy efficiency
2023 (Ebneyousef & Shirmarz, 2023)	Heuristic approach for load balancing using Artificial Bee Colony Optimization	• Lack of detailed performance analysis • Limited real-world validation • Potential for outdated information • Broad generalizations • Overload of information	• Comprehensive overview • Clear categorization • Guidance for researchers and practitioners • Applicability to diverse fog environments
2023 (Yuwen Zhou et al., 2023)	Reinforcement Learning Agents on Switches (RLoS)	• Learning time • High computational demand • Complex implementation • Potential instability • Increased overhead	• Dynamic decision-making • Decentralized control • Traffic optimization • Improved fault tolerance • Energy efficiency
2024 (Yue Zhou & Wu, 2024)	Load-balancing system for SDN data center based on improved ant colony optimization	• Computational complexity • Convergence time • Parameter sensitivity • Model accuracy dependency • Implementation complexity	• Improved network performance • Reduced latency • Resource efficiency • Scalability
2024 (Shahrbabaki, Coutinho, &	SDN-LB: A novel server workload balancing	• High complexity	• Efficient load balancing

(continued on next page)

Table 4 (*continued*)

Reference	Main Objective	Disadvantages	Advantages
Shayan, 2024)	algorithm for IoT video analytics	• Dependency on SDN infrastructure • Challenges in real-time adaptation • Implementation cost • Parameter tuning	• Real-time processing • Energy efficiency • Integration with SDN • Scalability
2024 (Mahmoudi, Barekatain, Beheshti, & Quintana, 2024)	MBL-DSDN: a novel load balancing algorithm in distributed software-defined networks based on micro-clustering and B-LSTM methods	• High computational overhead • Dependency on data quality • Potential latency in training	• Enhanced load balancing • Predictive capabilities • Distributed architecture • Dynamic adaptability • Improved QoS • Energy efficiency
2024 (Tache, Păscuțoiu, & Borcoci, 2024)	Optimization Algorithms in SDN: Routing, Load Balancing, and Delay Optimization	• Potential algorithmic trade-offs • Real-time constraints • Scalability issues in some methods • Vulnerability to attacks	• Improved network performance • Dynamic adaptability • Support for diverse applications • Holistic solutions
2024 (El Kamel, 2024)	Using FlowVisor and evolutionary algorithms to improve the switch migration in SDN	• Limited real-world validation • Potential performance overhead • Dependency on parameter settings • Complexity of integration	• Optimized switch migration • Enhanced controller load balancing • Flexibility • Improved fault tolerance
2024 (Singh et al., 2024)	A novel framework for capacitated SDN controller placement: Balancing latency and reliability with PSO algorithm	• Scalability bottlenecks • Data dependency • Limited real-time responsiveness • Parameter sensitivity	• Improved latency-reliability trade-off • Optimized controller placement • Dynamic adaptability • Enhanced fault tolerance

(GLBA) using a POX controller and Mininet simulator. Babar et al. (Babbar, Parthiban, Radhakrishnan, & Rani, 2022) investigated the impact of using SDN services on the performance of various multimedia applications and implemented and validated the Genetic Load Balancing Algorithm (GLBA) using POX controller and Mininet simulator. Hamdan et al. (Hamdan et al., 2021) first identified the controller and switches and expedited forwarding (EF) through a pair of classifiers in SDN and then extracted more optimal paths for network links to direct EF adaptively. Kolakov et al. (Kulakov, Kohan, Kopychko, & Cherevatenko, 2021) used a multipath routing approach and centralized information gathering in the SDN controller to increase the efficiency of dynamic traffic redirecting. Mahmoudi et al. (Mahmoudi, Avokh, & Barekatain, 2022) addressed the load balancing problem in SDN using dynamic voltage and frequency scaling (DVFS). The proposed method can balance the traffic load and reduce the synchronization cost and response time. Batista et al. (Batista, Figueiredo, & Prazeres, 2022) proposed a novel scheme for load balancing between fogs and the cloud through scheduling capabilities using SDN networks, which prevents load generation in IoT gateways and network links. Zhou et al. (Yuwen Zhou et al., 2023) presented a transaction-based solution using reinforcement learning of system (RLoS) to actively select the best controller for each transaction.

Ider et al. (Ider & Barekatain, 2021) presented an approach to solving load imbalance in software-based distributed networks between controllers. The approach uses the analytic hierarchy process (AHP) and technique for order of preference by similarity to ideal solution (TOPSIS) methods. Lai et al. (Lai, Wang, Chen, & Tsai, 2022) introduced the time-sliced switch migration (TSSM) technique for more accurate load balancing between controllers and provided the conditions for load sharing between two controllers in alternating periods. The TSSM method was proposed to improve load balancing between controllers. This method allows switches to distribute the load between two controllers in alternating time periods. When a controller reaches a load threshold, TSSM selects auxiliary controllers for load distribution and determines the appropriate switches and times for load migration. In this way, the load of each controller is maintained within a defined bound. The implementation of TSSM in an open network operating system (ONOS) has proven its performance capabilities. According to experimental results, TSSM can reduce the excess load of controllers by 98 % compared to the original OpenFlow method. In addition, TSSM can reduce the migration costs caused by relocation by about 78 % compared to conventional migration methods.

Zhong et al. (Zhong, Fang, & Cui, 2018) introduced a load balancer called load balancer based on server response time (LBBSRT). This method uses DN resilience and server response time to measure the actual response time of each server under a controller. The objective of this method is to balance the load and process user requests in a way that the load is evenly distributed among the servers.

Wang et al. (Huandong Wang et al., 2017) introduced live migration as a key technique for managing VMs in data center networks. Live migration provides the conditions for resource optimization, increased fault tolerance, and load distribution. However, the performance degradation during the migration process caused by the continuous efforts to reduce migration time and achieve appropriate bandwidth to minimize the negative impacts has become one of the dominant challenges. To solve this challenge, Wang et al. formulated a mixed integer model for each migration process. Using FPTA, the proposed method controls the migration schedule. The disadvantages of this method include the static nature of the proposed approach, the lack of specific thresholds, and the lack of calculation of the processing load of the destination VM. Table 4 summarizes various studies analyzing load-balancing methods and techniques in SDN along with their key features. These features cover various aspects such as load balancing strategies, efficiency, scalability, and implementation challenges. The key objective of Table 4 is to review the advantages and disadvantages of various studies (excluding the papers in Tables 1, 2, and 3) to gain a better understanding of load balancing in SDN networks.

3. Problem statement

Due to recent advances in network technology and the emergence of novel models such as fog and DSDN networks, effective traffic management and load balancing in such complex platforms has become a prominent challenge. In Fog DSDN architectures, the ability to evenly distribute the load between VMs and physical hosts (PHs) is crucial to maintaining network efficiency and responsiveness. The imbalance of distributed traffic can lead to serious problems such as congestion, overload, increased response time, and high energy consumption, which in turn significantly degrade network performance (Abderrahmane, Drid, & Behaz, 2024; Isyaku et al., 2024; Mahmoudi et al., 2024; Rostami & Goli-Bidgoli, 2024). In addition, certain problems such as the inability to recognize nonlinear and complex patterns, difficulty in determining the number of clusters (Chentouf, Miyara, & Chiba, 2024; Xiao et al., 2024), lack of adaptability to different conditions (Tamri, Antari, & Iqdour, 2024), failure to recognize clusters with different

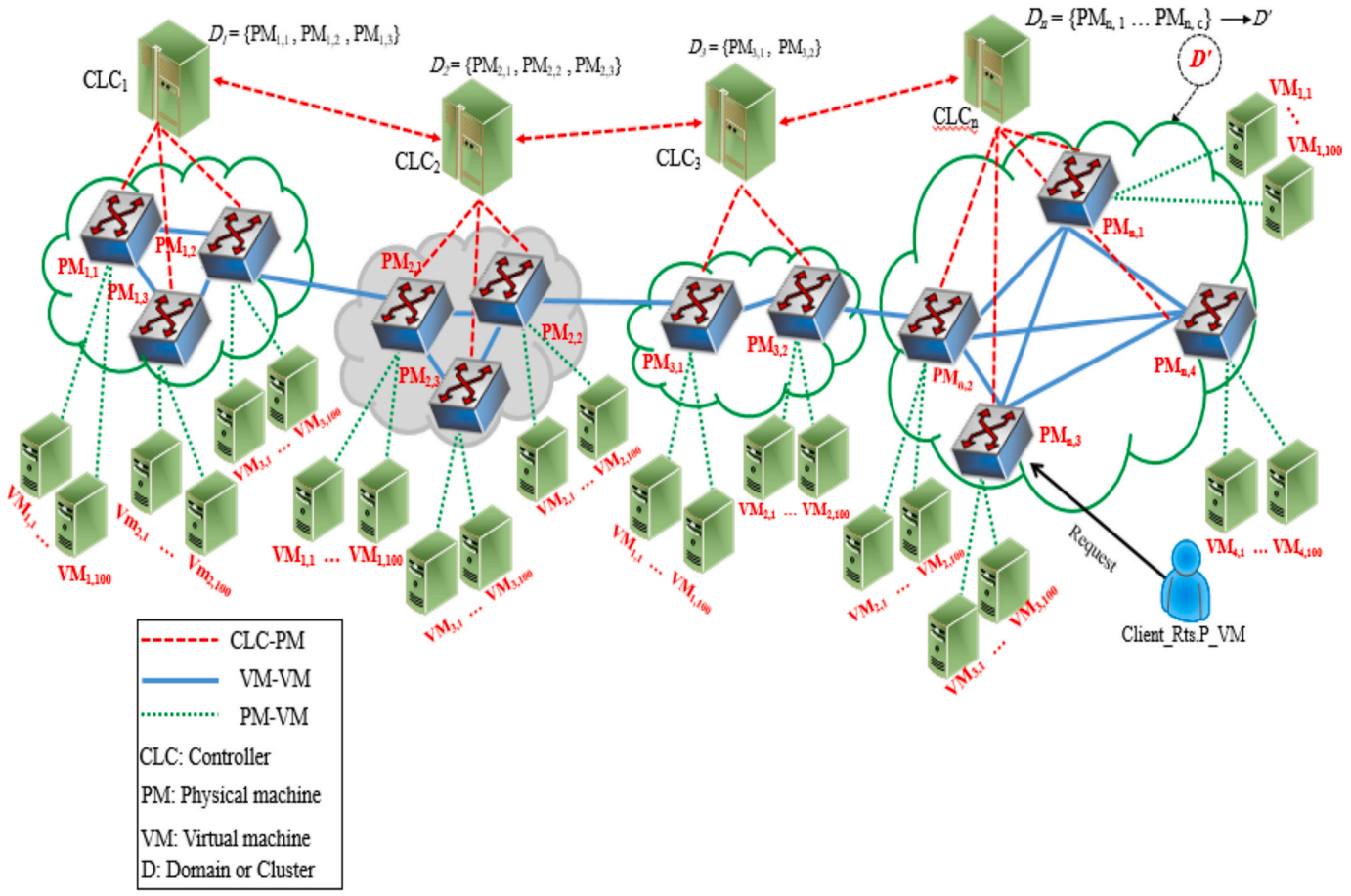

Fig. 1. The network topology utilized in the proposed CAAF-DSDN method.

shapes (Sikder, Hossain, Alenazi, & Atiquzzaman, 2024), and lack of optimal resource allocation (Minardi, Vu, Maity, Politis, & Chatzinotas, 2024), lead to inaccurate data analysis and inefficient traffic management. Solving such challenges requires further investigation and development of solutions to improve the performance of fog and DSDN networks. Incorrect data analysis can lead to major problems in traffic management. Although density-based clustering may be considered an effective solution, it must effectively consider the following specific issues. Unwise selection of parameters can lead to incorrect consequences and reduce the accuracy of the analysis:

- Selecting the appropriate parameters: This includes the accurate determination of the similarity radius and the minimum number of points required to form a cluster
- Outlier detection: Identifying and managing points that are distinct from the population of points
- Dealing with low data density: Identifying low-density regions that lead to the formation of unreasonable or small clusters
- Noise and unusual data detection: Considering data that does not belong to any of the clusters as noise
- Ambiguous point detection: Considering data that is far from all clusters as noise
- Another serious challenge in density-based clustering methods is ambiguous data due to various factors:
- Lack of information transparency: The data may not be transparent enough, limiting the possibility of effective use.
- Presence of noise: Noisy data affects the quality of the analysis.
- Information ambiguity: The lack of transparency of some information leads to serious problems in the analysis.
- Defects in data collection or recording: Deficiencies in data collection process lead to analysis problems.

Ambiguous points in data can complicate statistical processes, modeling, and the implementation of machine learning techniques due to the lack of access to accurate and complete information. Ambiguous data may lead to the formation of unreasonable and unstable clusters in sparse regions. Such regions usually contain noisy or outlier data. It is difficult to distinguish noisy data from valid data in a cluster, leading to a decrease in clustering accuracy. Ambiguous data can also lead to the formation of logically disjoint clusters, making the analysis and interpretation of clusters more complex. In addition, density-based clustering algorithms require fine-tuning of internal parameters. Ambiguous data can make it more difficult to find optimal parameter values.

Therefore, this study aims to address the above problems in load balancing and traffic management in fog and DSDN networks, focusing on providing optimal and efficient solutions to improve the load distribution and overall performance of the networks. To address these challenges, the proposed method includes the use of an adaptive structure and the fundamental concepts of adaptive algebra. The present paper aims to provide a more suitable method to resolve the aforementioned problems and achieve load balancing, improve network efficiency and quality of service parameters, and reduce network processing cost. In other words, once the optimal switch and controller are selected, the migration process is performed in such a way that the switch with the lowest traffic load is selected and the source controller returns to a stable state. In the next section, the features of the proposed method and the solutions provided are examined in detail.

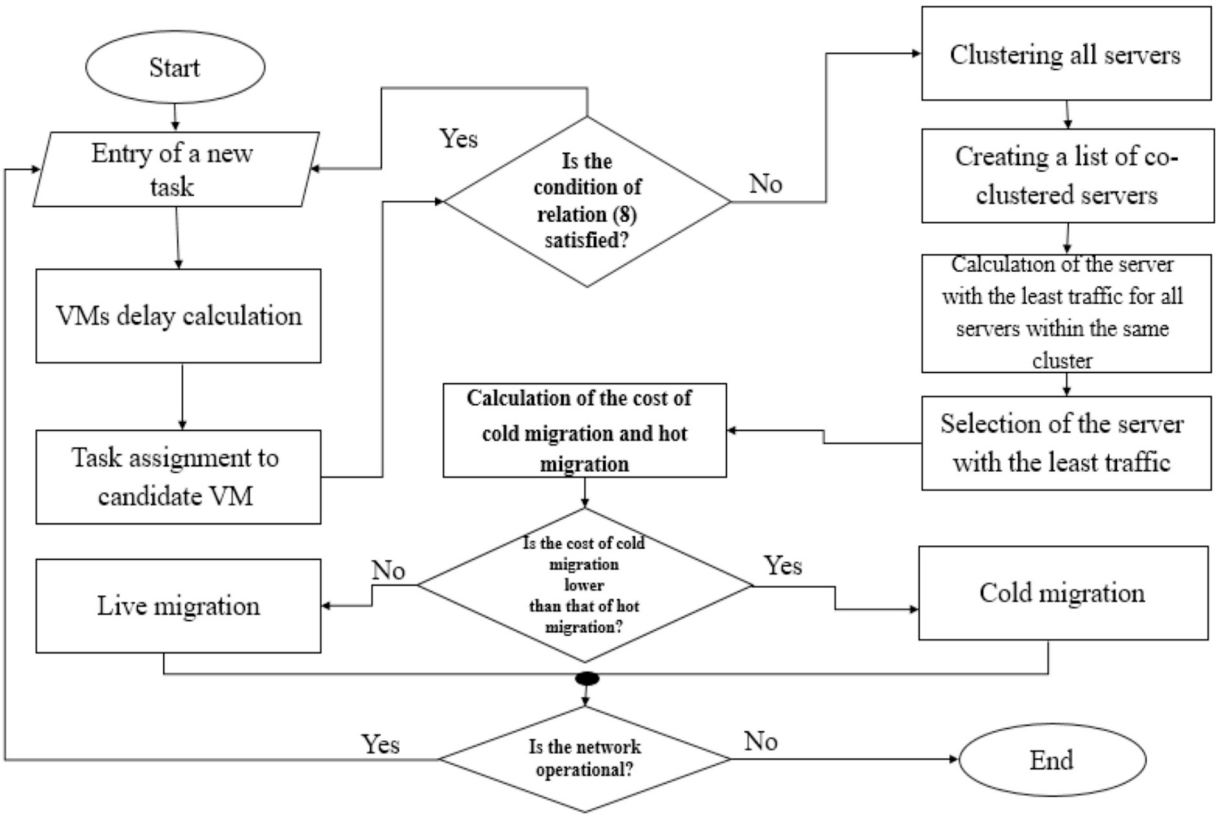

Fig. 2. Flowchart of the Proposed CAAF-DSDN Method.

4. CAAF-DSDN: The proposed method

By using density-based clustering and assigning adaptive weights to servers, the proposed method improves the VM migration process. This method selects a destination server with the highest similarity to the current server and ensures migration without disrupting the network and traffic. In light of the above, using the proposed density-based clustering algorithm leads to the realization of the following objectives.

- Reduce the materialization of unreasonable clusters: By more accurately estimating the data distribution, the model can identify regions with no data and sparseness and prevent the materialization of unreasonable clusters.
- Effectively deal with noisy data: More accurate distribution helps the model to separate points belonging to meaningful clusters from noisy points.
- Improved interpretation of data density changes: With flexibility in interpreting data density changes, the model can provide better clustering and accurately identify changes in data density.

Based on the optimal decision made in clustering VMs, the load distribution is optimized and the load balancing challenge in the network is resolved, which in turn improves QoS. The proposed method, CAAF-DSDN, is proposed as an innovative and effective method to optimize resource allocation and traffic management in distributed networks. This method focuses on complex networks with a large number of nodes and multiple processing tasks. Also, the topological complexity of these networks is significantly high due to the wide range and variety of processing tasks.

Distributed network control in this method is effectively achieved through network controllers. These controllers collect and analyze information related to resource status, network traffic, and service requirements. Based on the resulting data, more intelligent decisions are made for load balancing at the network level. Such decisions include redirecting requests based on optimal resource allocation for services and establishing dynamic policies for load management. This method aims to improve network QoS, optimize resource utilization, and increase network flexibility and efficiency in the face of changing user request trends and changing requirements. This procedure generally leads to improved load balancing. Fig. 1 represents the proposed CAAF-DSDN topology and its structure and performance.

According to the requirements of the network under evaluation, the proposed CAAF-DSDN method is implemented in two different operating modes, including normal load distribution and network overload.

- The normal load distribution mode does not require migration of the processing load, and the system optimally manages and distributes resources, and naturally achieves optimal performance. In this mode, servers or nodes distribute the processing load among them in a balanced manner and use the available resources in the best way. As a result, benefits such as reduced latency, reduced operating cost, and greater network stability are realized. Also, optimizing resource usage reduces energy and maintenance costs. In general, the system automatically and optimally distributes the processing load among the nodes and does not require human intervention or structural changes.
- According to Eq. (6), in the overload mode, when the processing load exceeds the threshold, the system needs to transfer the processing load to another VM with greater processing capacity. It should be said that network servers (hardware machines) have different characteristics, including diverse hardware, processing power, and responsiveness, which should be considered in evaluating and implementing the proposed method to make the conditions more realistic. Fig. 2 and

Table 5
Definitions of Variables and Functions Used in Algorithm 1 (Fig. 3).

Variables	Descriptions
VM	A set of VMs that execute task processing.
Task	Processing tasks that need to be assigned to VMs.
Sr	The physical server hosting the VMs.
D_{VM}	A function that calculates the delay of each VM.
Match_clustering (Sr, α)	A function that clusters the servers based on the parameter α.
Calculate F(Sr_i)	A function that calculates the cost function value for the server Sr_i.
$C^{C}_{Sr\ sr\ to\ immigration}$	A function that calculates the cost of Cold Migration.
Calculate $C^{H}_{Sr\ sr\ to\ immigration}$	A function that calculates the cost of Hot Migration.

Algorithm 1 show the flowchart and pseudocode of the proposed method, respectively.

Fig. 2 illustrates a detailed, step-by-step process for managing load distribution and task migration between VMs and servers. The process begins with the introduction of a new task into the system, followed by the calculation of delays for all VMs. The task is then assigned to the most suitable candidate VM. Next, the condition specified in Eq. (8) is evaluated to determine whether the workload falls within the acceptable range. If the condition is not satisfied, the server clustering process is initiated, generating a list of co-located servers. From this list, the server with the least traffic is selected. Subsequently, the cold and hot migration costs for the destination server are computed and compared. If the cold migration cost is lower than the hot migration cost, cold migration is performed; otherwise, live migration is initiated. Finally, the operational status of the network is assessed. If the network is stable, the process continues; otherwise, the operation is halted. This mechanism is primarily designed to optimize the selection of the destination server, minimize network delays, and reduce migration costs. For better clarity and understanding of the algorithm's logic, all variables and functions used in Algorithm 1 are comprehensively explained in Table 5, as shown in Fig. 3.

Fig. 3 illustrates a system designed to manage the distribution of processing load and make decisions regarding task migration between VMs and servers. The objective of this algorithm is to improve the efficiency and latency of the network by selecting the best server for task migration and determining the type of migration (cold or warm), based on the load and delays. Fig. 3 illustrates the optimization process for workload distribution and migration between VMs and servers. The process begins with the arrival of a new task, followed by the calculation of delays for all VMs. The VM with the minimum delay is then selected. If the workload remains within the acceptable threshold, load distribution proceeds normally. However, if the workload exceeds this threshold, an overload condition is detected, triggering the migration process. Servers are clustered based on a specific parameter, and migration costs are calculated for each server. The server with the lowest migration cost is selected, and the migration type (cold or hot) is determined based on a cost comparison. This process continues as long as new tasks are introduced into the system.

Algorithm 1. Load Balancing in the Proposed CAAF-DSDN Method

1: Function Propagation_load (VM, task, controller) *//Function introduction*
2: Input: VM: Virtual machin, Task: task, Sr: server *// Introducing Inputs: VM, task, server*
3: Output: selected server to migration, type of migration *// Introducing Outputs*

4: Do *//start*
5: $Task_n$= Get new task *// Assign new task: It is the function that receives the new task.*
6: D_{VM} = Calculate delay of all VM's; *// It is a function that calculates the delay of all virtual machines .*
7: $Selecte_{VM}$= Min(D_{VM}); *// Choosing the virtual machine with the least delay*
8: ***IF (If*** $(\sum_{VM=1}^{n} \overline{D_{vm}} < \overline{T}_D$ ***and*** $Best_{LB_t} < \overline{T}_{LB})$ *then* ***Normal load distribution occurs*** *else* ***Overload occurs)***.
9: Go to line 5
10: ***Else***
11: Match_clustering (Sr, α); *// It is a function that performs clustering of servers based on the parameter α.*
12: $List_{current}$ = Sr in The cluster that contains the current server *// It is a function that returns the servers in the cluster where the server Sr is located.*
13: ***For i=1 to*** $|List_{current}|$ *// Calculate the cost function for each server in the list*
14: Calculate F(Sr_i) from eq (41); *// Calculate_F(Sri): is a function that calculates the cost function for server Sri.*
15: ***End for***
16: $Sr_{sr\ ro\ immigration}$= Min (F(Sr)); *//Choosing a server for migration*
17: Calculate $C^{C}_{Sr\ sr\ to\ immigration}$ using eq. (43); *// Calculation of cold migration cost for migration server*
18: Calculate $C^{H}_{Sr\ sr\ to\ immigration}$ using eq. (44); *// Calculation of hot migration cost for migration server*
19: ***IF(*** $C^{H}_{Sr\ sr\ to\ immigration} > C^{C}_{Sr\ sr\ to\ immigration}$***)*** *// Selection of migration type: if hot migration cost > cold migration cost, then (Cold Migration) otherwise (Hot Migration)*
20: Do a cold migration
21: ***Else***
22: Do a hot migration
23: ***End IF***
24: ***End IF***
25: While (new tasks exist) *// Repeat until there are new tasks*
26: End functuon

Fig. 3. Load balancing Pseudocode in CAAF-DSDN method.

4.1. Load allocation in normal mode

Under normal conditions, the load allocation to each VM is realized based on the impact of the allocation on the load balance in the entire network. In general, the network policy seeks the lowest possible delay and the highest possible uniformity in load distribution. Little's Law has been used to calculate the relationship between the delay and processing load. In queuing systems, two prominent metrics are the average waiting time (W) and the average number of items waiting to be served (L). Little's Law relates these two metrics through the average rate of entry of items into the queuing system. According to Little's Law, this relationship under steady-state conditions is expressed as follows:

$$L = \lambda \bullet W \tag{1}$$

- where L is the average number of packets in the network, whether waiting or being served,
- λ is the average rate of packet arrivals to the network,
- and W is the average time spent by each client in the network (the sum of waiting time and service time).

The innovation of this study is based on Little's Law. If Little's Law is taken as the basis for calculating the relationship between latency and processing load in a VM, then the latency in a VM is a function of the rate of requests arriving and the time required to process them. According to this law, the relationship between latency and processing load is determined.

$$\overline{D_{vm}} = \frac{\overline{C_{vm}}}{\overline{F_{vm}}} + D_{vm}^{tr} \tag{2}$$

- where D_{vm} is the average waiting latency for a VM,
- $\overline{C_{vm}}$ represents the average occupied capacity,
- and $\overline{F_{vm}}$ represents the average instruction processing rate in a specific time interval for the VM.

These two parameters are related to the intrinsic characteristics of each VM. F_{vm} is measured in instructions per second. When this quantity is divided by the number of instructions, the result is expressed in units of time. For example, if a machine has the capacity to process two instructions per second and receives 20 instructions, it takes 10 s to process them. Also, D_{vm}^{tr} represents the time cost of assigning a task to the VM. The first part of Eq. (2), namely, $\frac{\overline{C_{vm}}}{\overline{F_{vm}}}$ is directly derived from Little's queuing law. To take into account the delay associated with assigning tasks, the second part is added to this equation. D_{vm}^{tr} is considered as the time cost associated with assigning tasks.

Since the network under consideration is an asymmetric network, the processing capacity must be defined precisely. The capacity of a VM refers to its ability to process computational tasks. The capacity of a VM is a combination of various resources such as random access memory (RAM), central processing unit (CPU), storage, and network bandwidth. To calculate the overall capacity, each of these resources is evaluated separately and the outputs are combined into a comprehensive model. The capacity of a VM determines its efficiency in executing various tasks and handling heavy processing loads. The exact capacity of a VM is calculated according to the following factors.

$$C_{vm} = k.e(|^{w_{ram}c_{ram}+w_{cpu}c_{cpu}+w_{disk}c_{disk}-w_{queue}Q_{vm}}|VM) \tag{3}$$

- where K is a scaling constant,
- w_{ram}, w_{cpu}, and w_{disk} are the weights of each resource (RAM, CPU, and storage) indicating their relative relevance in calculating the capacity.
- Also, c_{ram}, c_{cpu}, and c_{disk} are the remaining capacities of RAM, CPU, and storage, respectively,
- and Q_{vm} is the length of the task processing queue in the VM. This parameter represents the number of tasks waiting to be processed.

4.1.1. Impact of the task processing queue

The increase in the length of the Q_{vm} task queue due to resource busyness reduces the total processing capacity of the VM. This can slow down the processing speed and increase the latency of executing new tasks. In this designed model, increasing the RAM and CPU capacity leads to an increase in processing capacity. At the same time, increasing the processing queue length has a negative impact on processing capacity. It is expressed as an exponential nonlinear relationship considering the effects of simultaneous changes in several key factors and dynamically estimates the processing capacity. The constant k is added to the relationship for two reasons:

- Value adjustment: The actual values of the parameters may be too large or too small and need to be adjusted. Adjustment transforms the relationship into a reasonable and usable relationship on a real scale. The scaling constant k is used to fit the output of the relationship within a reasonable range.
- Unit standardization: This constant corresponds to the normalization of units in the relationship. This operation plays a crucial role when an exponential relationship is defined with different units of measurement (such as RAM in GB, CPU in GHz, and queue in number of tasks). The constant k helps to adapt the units and convert all the parameters to a common unit.

Before distinguishing between normal and overload, the amount of processing load distribution should be considered. To achieve a correct description of the network, the distribution should be based on the remaining capacity and the consumed processing capacity of each VM. The task entering rate is considered an independent variable. In a given time period, say t seconds, the number of tasks-entering the network should be determined. This quantity is determined as an average or by a function. Obtaining this information can approximately solve the problem and make the concept of load balancing meaningless. By definition, tasks are considered as events and their entry rate is a random phenomenon that has been used previously in the definition of the environment. For this purpose, the *Load Index* is used to evaluate this feature. This index indicates the amount of processing capacity used by each VM. Eq. (4) depicts how to calculate LI_{vm} based on processing capacity. Load Index is calculated as follows.

$$LI_{vm} = \frac{U_{vm}}{C_{vm}} * 100 \tag{4}$$

- where LI_{vm} is the percentage of the processing capacity used,
- U_{vm} refers to the actual utilization of the VM processing capacity,
- and C_{vm} represents the total processing capacity.

Based on the value of LI_{vm}, decisions can be made about processing load distribution and resource optimization. Such decisions can include changing the allocation of resources to VMs, or even adding new resources. The LI_{vm} value benefits system administrators to assess the load status of VMs and take optimization actions. This index can be used as a criterion to define load balancing among n virtual machines as follows:

$$LoadBalance = f(LI1, LI2, \ldots, LIn) \tag{5}$$

where LI_i represents the Load Index of VM_i.

$$Best\,LB = Min(Max\{LI_1\}.\mathrm{Max}\{LI_2\}.\cdots.Max\{LI_n\} \tag{6}$$

The max function represents the maximum load value for a VM. This means that the virtual machine with the highest load is considered. The min function aims to reduce the maximum load distributed among all VMs. In other words, the purpose is to find a load distribution strategy minimizing the maximum load distributed among all machines. This means preventing a VM from being overloaded. The purpose of using the max and min functions in Eq. (6) is to make the maximum LI value at each step tend to be the lowest possible value. The sum of the LI values alone cannot provide sufficient and accurate information about the load distribution status. The main objective of this equation is to ensure optimal resource utilization and prevent bottlenecks caused by high load on a particular VM. Minimizing the maximum Load Index makes the load distribution more balanced and can lead to improved performance and reduced latency in the network.

Effect of combinsing max and min in Eq. (6)

The combination of max and min functions in Eq. (6) plays a vital role in optimizing load balancing. The max function alone only measures the ratio of different resources without considering the actual size of the resources in the load balance. For example, if the Load Index of a VM is 0.1, it means that 10 % of the machine capacity is occupied. This statement lacks information about the actual quantity of processing tasks. Put differently, not considering the nominal capacity of the VM leads to calculating only the percentage of resource utilization. To provide better accessibility, in this study, the nominal capacity of the VMs is considered as the main criterion.

4.1.2. Impact of the standard deviation of LIs

By considering the standard deviation of LIs, we can evaluate the relative changes in the Load Index more accurately. When a new task is added to a VM with a processing requirement of n, the amount of change in the machine LI value and the difference from the average LI should be evaluated. The objectives of this procedure are as follows.

1. Impact of total capacity: Considering changes in the total capacity of VMs, it can calculate the impact of such changes on load balancing more accurately
2. Impact of the amount of change: Examining changes in *LI* indices takes into account the amount of deviation from the average in the load balancing calculation and increases the accuracy of the analysis. This method benefits to improve the accuracy of calculations and achieve a better balance in the use of processing resources in virtual systems.

To increase the accuracy, the equation is integrated with the standard deviation of the *LI* values and rewritten in the form of Eq. (7).

$$BestLB = Min(\alpha^{*}Max\{LI_1\}.Max\{LI_2\}.\cdots.Max\{LI_n\}) + (1-\alpha)^{*}\sigma(LI_1.LI_2\cdots..LI_n)) \tag{7}$$

Eq. (7) is the innovation of this research, derived through a simple mathematical argument based on axioms. The parameter α is determined as a weight for two objectives. The appropriate value of α depends on the system requirements. If $\alpha = 1$, the objective is only to minimize the maximum LI. Otherwise, the procedure focuses only on minimizing the standard deviation of the load index.

The value of α is determined based on the conditions and its level of heterogeneity. The more heterogeneous the environment, the greater the value of α should be to provide a more optimal and balanced distribution of the system load. As mentioned in the explanation of Eq. (7) about LI and its role in determining the load of machines, the adjustment of α plays an important role in the accuracy of the load balance in the system. Therefore, this parameter regulates the interaction between reducing the maximum load and reducing the load distribution among VMs to achieve the best network performance.

Now it is possible to provide an operational definition to distinguish between normal and overload modes. If Eq. (8) is satisfied, the load distribution is in the normal mode. Otherwise, overload has occurred. This operational definition uses the load index and system metrics to determine whether the system is operating optimally (normal mode) or has entered an overloaded mode due to increased load and resource incapacity. According to this definition, the system can automatically take actions to adjust and redistribute the load to prevent overload from occurring. This method helps to more accurately identify critical points and improve system stability under different conditions.

$$If\left(\sum_{VM=1}^{n}\overline{D_{vm}} < \overline{T}_{D} and Best_{LBt} < \overline{T}_{LB}\right) \textbf{then} Normalloaddistributionoccurs \textbf{else} Overloadoccurs \tag{8}$$

- where $\sum_{VM=1}^{n}\overline{D_{vm}}$ is the sum of normalized latencies or average latency for n virtual machines,
- $\overline{T}_D$ is the threshold of acceptable average latency in the network,
- $Best_{LBt}$ is the best load balance value up to time t,
- and $\overline{T}_{LB}$ represents the acceptable threshold for load balance (the maximum amount of load distribution relative to the remaining capacity).

According to this equation, if the total latency values of VMs are less than a defined threshold and the best LB is less than a certain limit, load balancing or system efficiency is improved. In the normal mode, the processing load is assigned to the least delayed VM with free capacity. This process is designed in such a way that Eq. (8) always holds true and overload does not occur. In this way, the system can optimally manage the load and stabilize its efficiency.

4.2. Overload detection in the proposed method

Overload in a network is detected by the following criteria.

1. Monitoring the load index (LI): If the LI value corresponding to one or more VMs approaches 1, the associated machines are approaching their maximum capacity and increasing the probability of overload.
2. Monitoring the processing latency: Measuring the average latency $\overline{T}_D$ in the network. Latency above a certain threshold can be an indication of overload
3. Monitoring the remaining capacity: Monitoring the remaining capacity of C_{vm} and C_{cpu}. Rapid reduction of the remaining capacity and reaching a certain threshold indicates the possibility of overload.
4. Checking the rate of incoming tasks: A sudden increase in the rate of incoming tasks into the system and the failure of the system in response to this upward trend increases the possibility of overload.
5. Task queue length analysis: A continuous increase in the queue length of processing tasks in VMs indicates high load and the possibility of overload.
6. Using load prediction models: Using analysis and prediction models, it is possible to examine the load trend and take preventive measures against overload.

7. Threshold definition: Defining specific thresholds for various criteria and monitoring threshold violations. Threshold violations can help detect overload conditions.

By combining these measures and continuously monitoring key metrics, it is possible to effectively identify and prevent overload conditions in the system. The aforementioned measures help to optimize system performance and maintain its stability. The proposed method in this study uses a combination of the above to detect overload in a system.

4.3. Task and VM migration process after overload

Load allocation in overload, especially in selecting the low-latency VM, is one of the major challenges in resource management. When the decision is made based on the lowest latency, some VMs will face overload. The important question is why the processing load is not considered at the time of decision-making to prevent overload. The main reason for this is the high processing cost and the possibility of network disruption. The following solutions are proposed for load management in overload mode.

1. Fast solution: In this method, the processing load is distributed more quickly. The disadvantage of this solution is the reduction in accuracy due to the high distribution speed. If this method leads to overload, the second solution is used.
2. Accurate solution: This method distributes the load more accurately. However, it requires more execution time

4.3.1. Managing VMs during overload

In overload mode, each VM can transfer all or part of its processing load to another VM. The VM can also be completely migrated to another physical server. This process, known as VM migration, is performed in two types: hot and cold migration.

- Hot migration: The VM transfers part of its processing tasks to another VM without downtime. The processing tasks are sent to the destination server in the form of information.
- Cold migration: The VM is completely shut down and all its processing tasks and attributes are transferred to the destination server.

4.3.2. Challenges in selecting the migration type

One of the main challenges of the proposed methods is determining the migration type and selecting the destination physical server. If the total number of servers in the network is S_r, only those servers satisfying the following equation are usable.

$$C_{Sr\ remain} C_{VM} \leq \quad (9)$$

- where $C_{Srremain}$ refers to the processing capacity or remaining resources of the server.
- Also, C_{VM} means the capacity required by the VM. In other words, this parameter indicates the amount of resources required by a VM to perform its tasks.

This equation is a condition for evaluating the ability to allocate VMs to servers. The remaining server capacity must be sufficient to provide the necessary resources for the VM. The remaining server processing capacity must be equal to or greater than the capacity required by the VM. Also, the destination server should not be overloaded after migration. If such an unfortunate situation occurs, the number of migrations and their costs will increase sharply.

4.3.3. Migration type determination strategy: Defining a confidence threshold

To determine the type of migration (cold or hot) and select the destination physical server, a confidence threshold is defined. For example, a threshold can be added to the remaining server processing capacity $C_{Srremain}$ in the form of a constant or a percentage of the VM capacity. This allows the VM to transfer its processing tasks to the destination server. This filter restricts the set S_r to a smaller subset called Sr_{proper} including servers suitable for migration.

4.3.3.1. Destination server selection strategy. To accurately select a destination server, it is not enough to consider only the latency and migration cost. Focusing solely on these two factors can lead to limited choices and network imbalance. For example, repeatedly selecting a small set of servers due to the nature of information transfer through network routing can lead to the following problems.

- Network bottleneck: High traffic slows down data transfer in the network.
- Network depreciation: Frequent use of a specific part of the network causes it to wear out more than other parts.

Avoiding problems such as bottlenecks and network depreciation requires intelligent and balanced load distribution between servers and the use of hot and cold migration in appropriate conditions. Applying appropriate confidence thresholds and filters can improve system performance and reduce costs and the number of migrations. In this study, the server is defined as a host resource from the perspective of the migrating VM and must have several characteristics as follows.

- Ability to run other VMs: The server must have sufficient processing capacity, including a combination of CPU, RAM, and HDD storage space to support the migrating VMs.
- Support for various enablers: The server must be able to support various enabler technologies.
- Enhanced security: The server must have strong security and protection standards to secure the data and processing operations of the VMs.
- Flexibility in utilizing hardware resources: The hardware and software compatibility of the server with the VM must be acceptable. This compatibility can include similarity of operating system, virtualization software, and other hardware features.
- Automated troubleshooting: The server must be able to provision resources and automate troubleshooting.
- Communication bandwidth: The communication bandwidth between the current server and the destination server must be sufficient to realize VM migration without disruption.
- Availability and uptime: The server's access and downtime must ensure the stability of VM operations.
- Storage capabilities such as RAID: The server must support reliable storage technologies such as RAID.
- Server location: The physical distance of the server from the VM can affect migration latency and network performance.

4.3.3.2. Prioritizing features. Inherently, no special priority is given to the features. If more important features are identified, the problem is transformed from an unsupervised problem to a well-defined structured problem. However, the environment faces several unknowns that must be addressed by decision-making methods. Subsequently, by a linear search and using a classifier or objective function, the problem can be solved based on the maximum or minimum value of the objective function. Then, the most similar server to the current server regarding hardware and software features is selected as the appropriate migration alternative. In addition, the selection of the server should not disrupt to the network traffic during the migration. To improve the migration

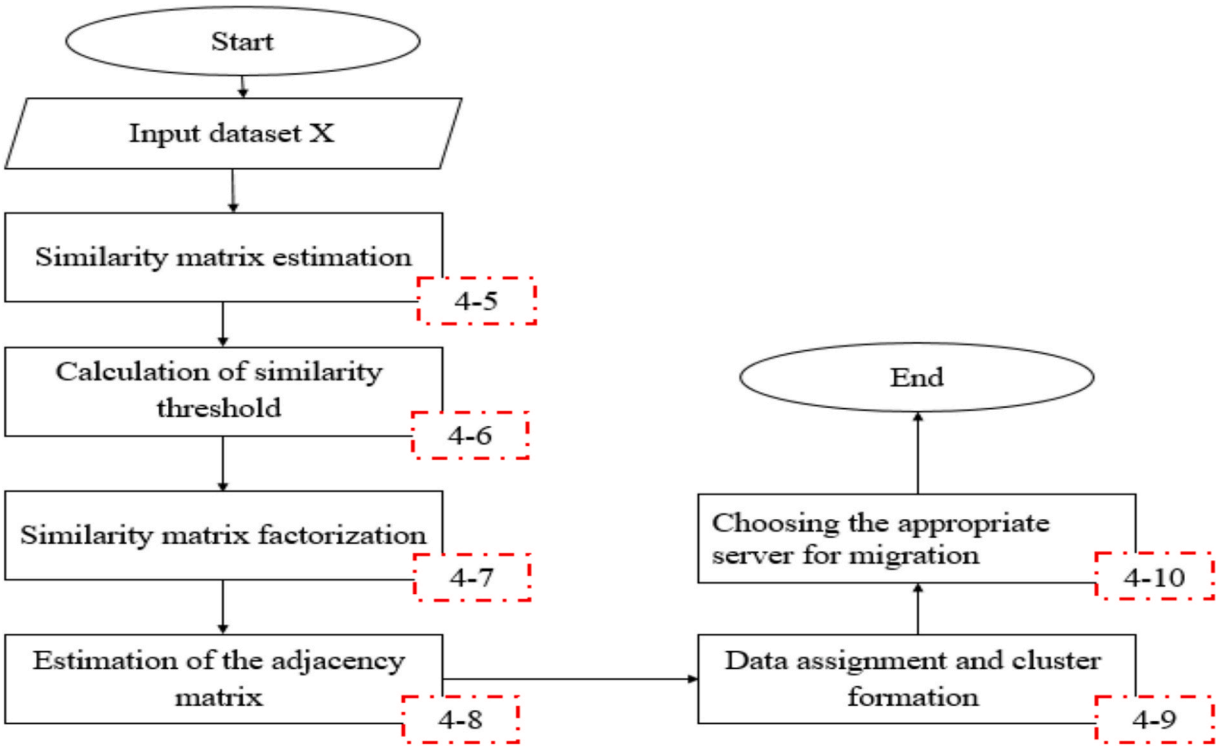

Fig. 4. The proposed Clustering method in CAAF-DSDN.

1: Function QR_ Algorithm (S) *// Introducing the main function called QR_Algorithm with Ssimilarity matrix input.*
2: Input: S similarity matrix (m×m) *// The input of the function is a similarity matrix S with dimensions (m×m)*
3: Output: Eigenvalues (λ) and eigenvectors (v) *// The output of the function will contain eigenvalues λ and eigenvectors v*
4: Initialize empty matrices Q and R. *// Two empty matrices Q and R are created to store the QR decomposition results.*
5: Set S as the matrix R *// Matrix S is regarded as matrix R*
6: For k:1 to max_iterations: (max_iterations is a predefined limit) *// An iteration loop is started for the maximum number of iterations (max_iterations).*
7: Perform QR decomposition on Sk to get Qk and Rk. *// In each iteration, QR decomposition is performed on Sk to obtain Qk and Rk matrices.*
8: Update Sk = Rk * Qk. *// Sk is updated and calculated as the product of Rk and Qk.*
9: End for *// The end of the iteration loop.*
11: Extract eigenvalues λ from the diagonal of R_k *// The eigenvalues of λ are extracted from the principal diameter of the Rk matrix .*
12: Initialize an empty matrix V of size m×m *// An empty matrix V with dimensions' m × m is created to store the eigenvectors .*
13: For each eigenvalue λi: *// For each eigenvalue λi a loop is started.*
14: Initialize a random vector $\boldsymbol{v_i}$ *// A random vector vi is defined to start the computation.*
15: For j:1 to max_iterations *// An iteration loop is started for the maximum number of iterations to compute the eigenvector.*
16: Compute vi = Sk * vi *// At each iteration, the eigenvector vi is updated by multiplying Sk by vi.*
17: Normalize vi *// The eigenvector is normalized to have unit length.*
18: End for *// End of iteration loop for special vectors.*
19: End for *// End of iteration loop for eigenvalues.*
20: Return (λ, v) *// Finally, the eigenvalues of λ and the eigenvectors of v are returned.*

21: End function

Fig. 5. The pseudocode of neighborhood matrix factorization in the proposed CAAF-DSDN method.

Table 6
Definitions of Variables and Functions Used in Algorithm 2 (Fig. 5).

Variables	Descriptions
S	The similarity matrix with dimensions (m × m).
Q, R	The matrices obtained from the QR decomposition, which are initialized and updated at each step.
$max_iterations$	The maximum number of iterations set for the convergence of the algorithm.
λ	A set of eigenvalues extracted from the matrix Rk.
$\vec{v}$	A set of eigenvectors calculated for each eigenvalue.
SK	The updated matrix, which receives a new value at each iteration and is used to compute the eigenvalues.
$\vec{v_i}$	The initial random vector, which is updated and normalized during the iteration process to calculate the final eigenvector.

process, a density-based clustering method is introduced by applying the following.

- Server similarity regarding the network topological structure: The similarity of the servers in terms of location and network connections is considered.
- Geographical importance: The physical distance of the server to the current server plays an important role in communication latencies.
- Need for flexibility: Migration methods should be flexible to respond to changing network and traffic needs.
- Reduce the probability of selecting servers in high-traffic areas: Servers located in high-traffic areas are less likely to be used to prevent the formation of bottlenecks in the network.

Algorithm 3. Cluster Generation Using the Neighborhood Matrix in the Proposed CAAF-DSDN Method

1: Function Create_cluster (N, X) *//Create a function named m with input (Neighborhood matrix (N) and data set (X))*

2: Input: H neighbore matrix (m×m), X dataset (m×n) *// Function inputs: (N): an m×m adjacency matrix indicating similarity between samples. (X): an m×n dataset containing m samples and n features.*

3: Output: B; *// The output of this function: (B): a matrix with dimensions (m*1) that shows which cluster each sample belongs to. This matrix stores the final clustering of the data.*

4: N`=sort (N); *// The matrix (N) is sorted in descending order based on the number of neighbors for each sample, prioritizing samples with the most neighbors for cluster allocation. The resulting ordered matrix is denoted as (N).*

5: For i:1 to m *// An iteration loop is executed for the data samples. This loop is executed for each sample (i) from 1 to (m) (the total number of samples).*

6: For j:i to m *// The inner loop to compare each instance (i) with other instances (j). This loop is executed from (i) to (m) so that all samples are checked in pairs.*

7: If ($b_j == 0$) *// This condition checks if the instance (j) has not yet been assigned to any cluster (if the value (b_j = 0)). If the sample is not assigned to a cluster, the condition is true.*

8: If ($n'_{ij} == 1$) *// This condition checks whether instance (j) is a neighbor of instance (i) or not. If the value of (n'_{ij} = 1), sample (j) is a neighbor of sample (i) and the condition is satisfied.*

9: If ($b_i == 0$) *// This condition checks if instance (i) is also not assigned to a cluster. If (b_i = 0) then instance (i) has not yet been clustered.*

10: $b_j = index\ of\ n'_i$ *// Here sample (j) is assigned to sample cluster (i). That is, the value (b_j) is equal to the cluster index (i).*

11: Else

12: $b_j = b_i$ *// If the sample (i) is already clustered (b_i =0), the sample (j) is also assigned to the cluster (i). This is done by initializing (b_j = b_i).*

13: End if

14: End if

15: End if *// Closing the bets and ending the inner loop.*

16: End for

17: End for *// End of outer loop for (i) that runs on all instances.*

18: For i:1 to m *// A new loop is run to finalize the clustering of the data, running from samples (i = 1) to (i = m).*

19: ***If count*** $((index\ of\ h'_i\ in\ B) \geq T_n\ \&\ b_i == 0)$ *// This condition checks whether the number of neighbors of the sample (i) in previous clusters is greater than or equal to the threshold (T_n) and whether (b_i = 0) (ie, the sample is not yet clustered).*

20: $b_i = index\ of\ h'_i$ *// If the previous condition is true, sample (i) is assigned to the cluster that has the largest number of its neighbors.*

21: ***Else If*** $(b_i == 0)$ *// If (b_i) is still equal to zero (that is, the sample is not assigned to any cluster), it is considered as outlier data.*

22: x by $index\ of\ h'_i$ is Outliers data *// In this step, sample (i) is labeled as outlier data.*

23: End if *// End of bet and loop.*

24: End for *// End of the loop to check and final allocation of clusters.*

25: Return (B) *// Finally, the function returns the matrix (B) representing the final clustering of samples.*

26: End function

Fig. 6. The Pseudocode of cluster generation using the neighborhood matrix in the proposed CAAF-DSDN method.

Algorithm 4. Clustering Procedure in the Proposed CAAF-DSDN Method

```
1: Function Match_clustering (X, α)   // This function clusters data using two inputs: (X) a data matrix with m rows (samples) and n columns (features), and (α) a coefficient used as an input parameter for algorithm settings.
2: Input: X dataset (m×n) , α Component coefficient
3: Output: B; // belong cluster matrix (m×1)   // The output of the algorithm is a matrix B that indicates the belonging of each sample to a specific cluster.
4:  Calculate S from eq(10)   // In this step, the matrix S is calculated based on a special equation (Eq. 10). This matrix may represent the similarity or distance between the data.
5:  [v, λ] = QR_Algorithm(S)   // The QR algorithm is used to decompose the matrix S. This algorithm achieves two eigenvector (v) and eigenvalue (λ) matrices that can be used to identify important structures in the data.
6:  Find the mode of distribution of matrix S   // In this step, the distribution mode of matrix S is calculated. This state can represent the accumulation of data or specific patterns in them.
7:  Calculate P matrix according to S distribution mode like eq(21)   // The P matrix is calculated based on the distribution mode of the S matrix. This step may help determine weights or probability values in clustering.
8:  Calculate Tc by eq(24)   // Here, the Tc value is calculated using Eq. 24. This value may serve as a threshold for detecting clusters.
9:  Calculate neighbor matrix H by eq(27)   // The neighbor matrix H is calculated based on Eq. 27. This matrix represents the neighborhood equations between samples in the data.
10:  B=Create_cluster (H, X)   // Using the neighbor matrix H and the data X, clusters are created and the result is stored in the matrix B.
11:  Return (B)   // In this step, the matrix B, which represents the cluster membership, is returned as the output of the function.
12:  End function
```

Fig. 7. The Pseudocode of clustering procedure in the proposed CAAF-DSDN method.

Table 7
Time complexity of the proposed CAAF-DSDN method.

Title	Interpretation	Time Complexity
Similarity Matrix Calculation	This means that the time required for computation is directly proportional to the square of the number of samples (m).	$O(m^2)$
Similarity Matrix Factorization	Factorization refers to decomposing the matrix into simpler forms, which is useful in data analysis.	$O(m^2)$
Similarity Matrix Distribution Calculation	It focuses on analyzing the similarities present in the data.	$O(m^2)$
Similarity Threshold Calculation	A threshold is set to detect similarity between samples.	$O(m^2)$
Neighborhood Matrix Estimation	This analyzes the neighborhood relationships between samples.	$O(m^2)$
Cluster Estimation	This includes the process of identifying and forming clusters based on the input data.	$O(m^2)$
Overall Complexity of the CAAF-DSDN Algorithm	Since all the above steps have similar time complexity and are executed sequentially.	$O(m^2)$

This clustering method should provide an accurate interpretation of the similarity of servers and be able to distinguish between good and bad servers. In addition, to better manage traffic and avoid problems caused by selecting servers in high-traffic areas, a weighted method based on adaptive algebra is introduced to prioritize and rank servers.

Table 8
Selected servers not located in high-traffic areas.

Description	Symbol
The assigned workload of the server is low.	C_s
The traffic from neighboring servers and controllers is low.	$Tr_s^{neighbor}$
The communication capacity of the server-to-server links is high.	$Li_s^{neighbor}$
The potential bottlenecks around the server are minimal.	Bt_s
The assigned workload to neighboring servers is low.	$C_s^{neighbor}$
The average previously recorded delay for the target server is as low as possible.	$\overline{D_s}$

Table 9
Network Architecture Analyzed in the Simulated Scenarios.

Simulator software	OMNeT++ 6.0.1
OS	windows
RAM	16 GB
CPU	Core i7 10510U
The topology at the controller layer	mesh
The topology at the server and controller layers	star
The topology at the server layer	random
Transmission medium	wire
Network type	Fog Distributed
Traffic program	CBR
Agent type	UDP
queue type	Queue/DropTail

4.4. Main challenges in density-based clustering

1. Selecting appropriate parameters: One of the most important challenges is the correct selection of parameters such as the similarity

Table 10
Parameters of the simulated scenarios in the proposed CAAF-DSDN method.

Scenario	Controller	Server	VM	Task	Scenario	Controller	Server	VM	Task
1	5	20	300	1000	42	20	100	500	10,000
2	5	20	300	5000	43	20	300	500	1000
3	5	20	300	10,000	44	20	300	500	5000
4	5	100	300	1000	45	20	300	500	10,000
5	5	100	300	5000	46	20	20	1000	1000
6	5	100	300	10,000	47	20	20	1000	5000
7	5	300	300	1000	48	20	20	1000	10,000
8	5	300	300	5000	49	20	100	1000	1000
9	5	300	300	10,000	50	20	100	1000	5000
10	5	20	500	1000	51	20	100	1000	10,000
11	5	20	500	5000	52	20	300	1000	1000
12	5	20	500	10,000	53	20	300	1000	5000
13	5	100	500	1000	54	20	300	1000	10,000
14	5	100	500	5000	55	100	20	300	1000
15	5	100	500	10,000	56	100	20	300	5000
16	5	300	500	1000	57	100	20	300	10,000
17	5	300	500	5000	58	100	100	300	1000
18	5	300	500	10,000	59	100	100	300	5000
19	5	20	1000	1000	60	100	100	300	10,000
20	5	20	1000	5000	61	100	300	300	1000
21	5	20	1000	10,000	62	100	300	300	5000
22	5	100	1000	1000	63	100	300	300	10,000
23	5	100	1000	5000	64	100	20	500	1000
24	5	100	1000	10,000	65	100	20	500	5000
25	5	300	1000	1000	66	100	20	500	10,000
26	5	300	1000	5000	67	100	100	500	1000
27	5	300	1000	10,000	68	100	100	500	5000
28	20	20	300	1000	69	100	100	500	10,000
29	20	20	300	5000	70	100	300	500	1000
30	20	20	300	10,000	71	100	300	500	5000
31	20	100	300	1000	72	100	300	500	10,000
32	20	100	300	5000	73	100	20	1000	1000
33	20	100	300	10,000	74	100	20	1000	5000
34	20	300	300	1000	75	100	20	1000	10,000
35	20	300	300	5000	76	100	100	1000	1000
36	20	300	300	10,000	77	100	100	1000	5000
37	20	20	500	1000	78	100	100	1000	10,000
38	20	20	500	5000	79	100	300	1000	1000
39	20	20	500	10,000	80	100	300	1000	5000
40	20	100	500	1000	81	100	300	1000	10,000
					41	20	100	500	5000

radius (∈) and the number of minimum points (MinPts) in a cluster. Inappropriate selection of these parameters can lead to inaccurate or incorrect results. Too small values for ∈ can lead to the identification of too many small clusters or outliers. Also, larger values for ∈ may lead to cluster merging and reduced accuracy

2. Data Ambiguity: Data ambiguity is another important and influential problem caused by noise, data collection flaws, or imprecision in the input data. Such ambiguity can affect clustering estimates and lead algorithms away from correctly detecting clusters

4.4.1. Proposed solutions to improve density-based clustering

Density-based clustering is one of the important approaches in data analysis. The grouping of complex data focuses on the density of the data. Such algorithms are especially useful in identifying clusters with irregular shapes and detecting outliers. Given the aforementioned limitations, the following solutions can help improve the performance of density-based clustering algorithms.

1. Adaptive algebra-based adaptive structure: Relying on adaptive algebra principles to automatically configure clustering parameters based on changes in input data can help solve the challenge of parameter selection. This structure dynamically models changes in data density and adjusts clustering parameters based on data density and distribution
2. Creating an adaptive adjacency matrix: One of the efficient methods to improve clustering is to use an adaptive adjacency matrix. Based on the distribution of data density in a given space, this matrix helps in determining adjacencies and similarities more accurately. It can also be used to better select outliers and optimize clustering results. Improving the accuracy of adjacency distribution estimation leads to better detection of clusters and boundaries by the model
3. Probability distribution modeling in clusters: Improving the estimation of data distribution in different areas of the data space can lead to more accurate identification of cluster boundaries. Using probability distributions to model data density and identify dense areas leads to more accurate models that adapt dynamically to different conditions

4.4.2. Advantages of the proposed method

- Enhance the prediction accuracy of dense areas: More accurate modeling of data density improves the detection of cluster boundaries and reduces the error in cluster separation.
- Reduce ambiguity in outlier detection: Improving the estimation of adjacency distribution leads to improved identification of outliers and reduces their interference in clusters.
- Increase computational efficiency: Using adaptive structure and adjacency matrix reduces computational complexity and increases the efficiency of the algorithm in dealing with big data.
- Better interaction with noisy and ambiguous data: The adaptive structure of algorithms and probabilistic models for processing noisy and ambiguous data leads to more accurate results.

Table 11
Type and characteristics of network nodes in the simulated scenarios.

Variable	Unit	Amount	Description
capacity server	MIPS (Million Instructions Per Second)	[200 300]	random
capacity task	MIPS (Million Instructions Per Second)	[5 20]	random
capacity vm	MIPS (Million Instructions Per Second)	[18 30]	random
number of processed units of tasks in a unit of time	MIPS^2 (Million Instructions Per seconds to the power of 2)	[1 2]	random
bandwidth	Mbps (Megabits Per Second)	[7 10]	random
server os		1,2	random(Linux, Windows)
vm software		1,2	random(VMware vSphere, Microsoft Hyper-V)
Uptime		[0 24]	random (down time in 24 h of a day)
delay allocation	ms(Millisecond)	[0 1]	
cost unit	MIPS*Mbps/m	1	
cost packing	MIPS/KB	1	
threshold of acceptable delay in server	ms(Millisecond)	200	
threshold of used capacity in vm	MIPS (Million Instructions Per Second)	80 %	
max width enviroment	m(meter)	1000	all nodes are locatted in this area
max height enviroment	m(meter)	1000	all nodes are locatted in this area
packet size	KB(Kilobytes)	1.5	
network type		wired	
protocol		udp	
max cluster		num server/5	
max iter		100	
simulation time	ms(Millisecond)	depend on scenarios	
enviromemt delay	ms(Millisecond)	100	

4.4.3. Improving the estimation of data distribution and its impact on the efficiency of density-based clustering

A model that can provide a more accurate estimate of the distribution of data can use the available information more efficiently and identify differences well. This provides conditions for detecting and interpreting patterns, equations, and changes in the data with greater accuracy. By improving the distribution estimate, the model receives more information about the diversity and dispersion of the data and makes more optimal decisions about the differences and equations between the data. This capability in the data clustering process can prevent interference and ambiguity in dense areas and determine the boundaries of clusters more accurately. In addition, a more accurate estimate of the adjacency distribution increases the flexibility of density-based clustering algorithms. By obtaining an accurate estimate of the data distribution at each point in the data space, the model can better understand the relationships and changes. Increasing the accuracy of the adjacency estimate helps the model adapt to the diversity and complexity of the data. Subsequently, in the face of various changes, the model can provide faster and more accurate responses. The trained model quickly identifies new patterns and features with a more accurate estimate of the data distribution and responds well to changes in the data density distribution. This feature leads to increased flexibility and improved model performance in the face of emerging data or conditions.

In density-based clustering algorithms, clusters are formed based on points located in high-density areas. Such points may be located in spaces with complex curves or patterns, or areas with high relative changes. Relying on information about the data density, density-based algorithms are able to detect and respond to areas with variable density and more diverse data. Such algorithms are particularly suitable for data with high variation and perform well in identifying heterogeneous areas with different densities.

The practical implementation of these ideas requires careful evaluation to determine their effectiveness in addressing the challenge of data ambiguity. Fig. 4 represents the clustering steps of the proposed method sequentially. To overcome the aforementioned challenges and achieve clustering, the proposed method presents a six-step approach called CAAF-DSDN:

1. Estimation of similarity matrix: In this step, the similarities between data are calculated using appropriate metrics.
2. Factorization of the similarity matrix: This step involves analyzing and separating the similarity matrix into fundamental components to identify hidden structures in the data.
3. Identifying the type of distribution in the adjacency matrix and estimating the distribution matrix: This step involves analyzing the distribution of data in different areas and estimating the distribution related to neighboring points.
4. Estimating the clustering threshold: In this step, the thresholds necessary for forming clusters are determined to assign points to appropriate clusters.
5. Estimating the adjacency matrix: This step involves calculating the data adjacency based on density and distance criteria.
6. Data assignment and cluster formation: Data are assigned to different clusters and clusters are formed based on the common characteristics of their members.

By redefining the similarity concepts in density-based clustering, the CAAF-DSDN method aims to develop a general rule for determining the degree of similarity between data. For example, by estimating the threshold based on decomposition rules inspired by adaptive Hermitian matrices, this method can theoretically provide an accurate estimate for the threshold of similarity and adjacency between two data.

4.5. Similarity matrix estimation

Accurate estimation of the similarity matrix in CAAF-DSDN can help reduce the effects of data ambiguity. With a more accurate similarity matrix, clustering algorithms can provide better criteria for forming clusters. Factorization of the similarity matrix helps to identify key and influential features in the data. This leads to an improved clustering process and the identification of meaningful differences between data. Identification of the distribution type in the adjacency matrix and estimation of the distribution matrix have been added to this method with the aim of better modeling of information and identifying differences in the data. To further optimize clustering and reduce interference, an accurate similarity threshold estimation method is added to the model. The created adjacency matrix provides a measurement for the similarity and difference between data in the form of a machine-understandable mathematical model. Combining the results from the previous steps with an accurate estimation of the matrices and the clustering threshold leads to significant improvements in the process of assigning data to clusters. This in turn leads to clearer cluster boundaries and improved formation of meaningful clusters. The proposed method is a new idea based on adaptive algebra to solve the problem of data ambiguity in clustering.

Before defining the clustering structure used, suppose that the clustering input is a set X represented as a $p \times q$ matrix containing scalar data. Also, p represents the number of features in each vector (data record) and q represents the number of vectors to be clustered. If x is

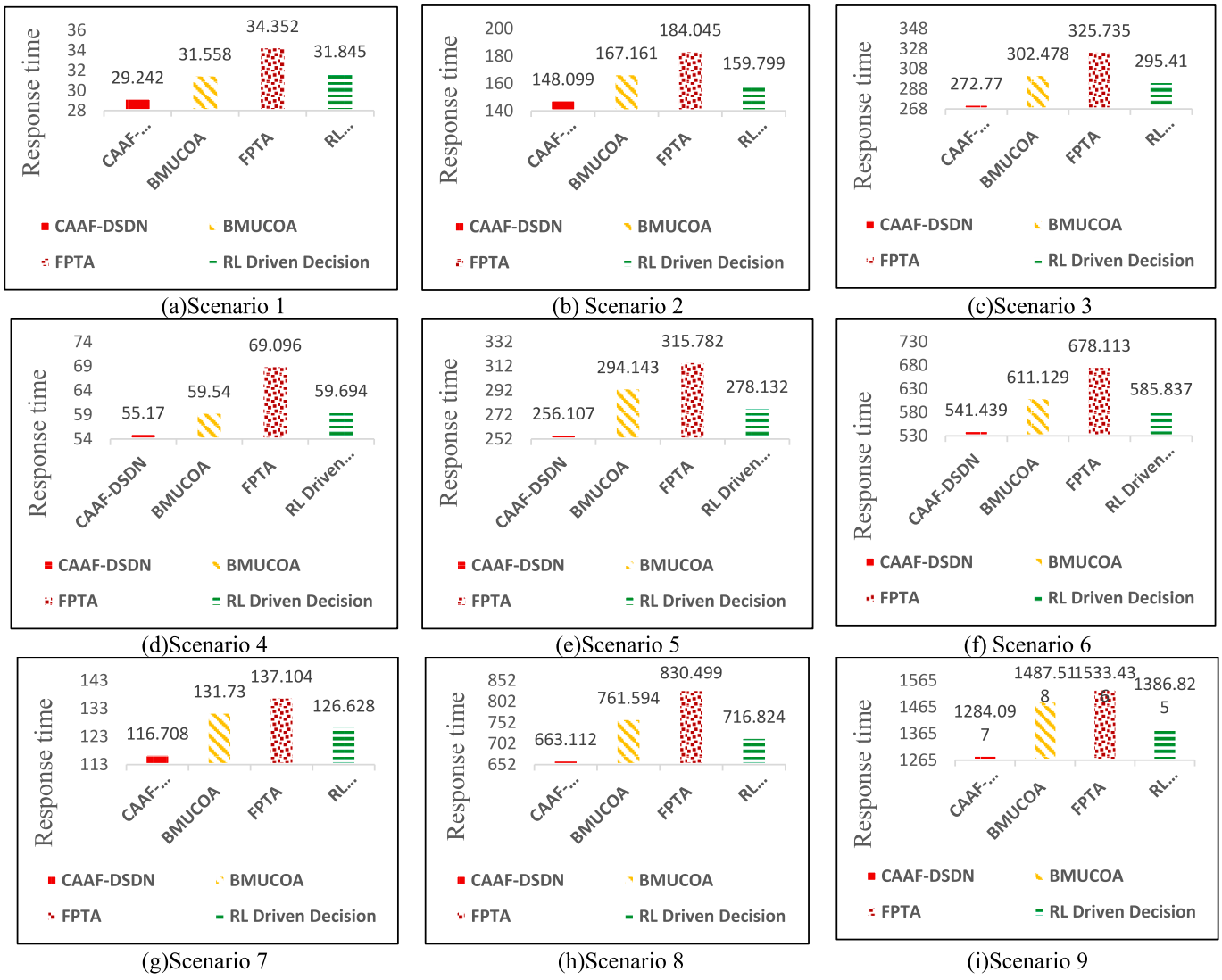

Fig. 8. Response time for 5 controllers and 300 VM in (a)Scenario 1, (b)Scenario 2, (c)Scenario 3, (d)Scenario 4, (e)Scenario 5, (f)Scenario 6, (g)Scenario 7, (h) Scenario 8 and (i)Scenario 9.

defined as a data, it can be expressed as Eq. (10) where each x_p represents the p^{th} feature in the data vector x.

$$\overrightarrow{x}p = \{x_1.x_2.\bullet\bullet\bullet.x_p\} \tag{10}$$

Where:

- $\{x_1.x_2.\bullet\bullet\bullet.x_p\}$: The data points, or elements of the set, are represented here as vectors of data.
- $\overrightarrow{x}$: represents the set of all data points from x_1 to x_p.

For a set with m member, Eq. (11) can be defined as follows.

$$\overrightarrow{x}Q = \{x_1.x_2.\bullet\bullet\bullet.x_q\} \tag{11}$$

Where:

- $\{x_1.x_2.\bullet\bullet\bullet.x_q\}$: The data points, or elements of the set, are represented here as vectors of data.
- $\overrightarrow{x}$: represents the set or vector consisting of all members from x_1 to x_q.

The algorithm has a matching criterion between two data $x_q.x_p$ as members of the data set x. According to the similarity principle, if we consider the data as vectors, the difference between the two vectors is determined by the length of the vectors and the angle θ between the two vectors. Therefore, the similarity of the two vectors $\overrightarrow{x_q}.\overrightarrow{x_p}$ can be calculated through Eq. (12).

Similarity $(\overrightarrow{x_q}.\overrightarrow{x_p}) = f(\|\overrightarrow{x_q}\|, \|\overrightarrow{x_p}\|, \theta)$.

$\|\overrightarrow{x_q}\| = \sqrt{x_q + x_q}$

$$\|\overrightarrow{x_p}\| = \sqrt{x_p + x_p} \tag{12}$$

where $\|\overrightarrow{x_q}\|$, $\|\overrightarrow{x_p}\|$ represent the lengths of the vectors $\overrightarrow{x_q}.\overrightarrow{x_p}$, respectively, and θ is the angle between them.

$$d_{qp} = \|\overrightarrow{x_q}\| \bullet \|\overrightarrow{x_p}\| \bullet \cos(\theta) = \overrightarrow{x_q} \bullet \overrightarrow{x_p} \tag{13}$$

Where:

- d_{qp}: It represents the distance or similarity between the two vectors $\overrightarrow{x_p}$ and $\overrightarrow{x_q}$.
- $\|\overrightarrow{x_q}\|$ و $\|\overrightarrow{x_p}\|$: They are, respectively, the length or norm of the vectors $\overrightarrow{x_q}.\overrightarrow{x_p}$.
- $\cos(\theta)$: Represents the cosine of the angle between the two vectors, which is used in the dot product of the vectors.

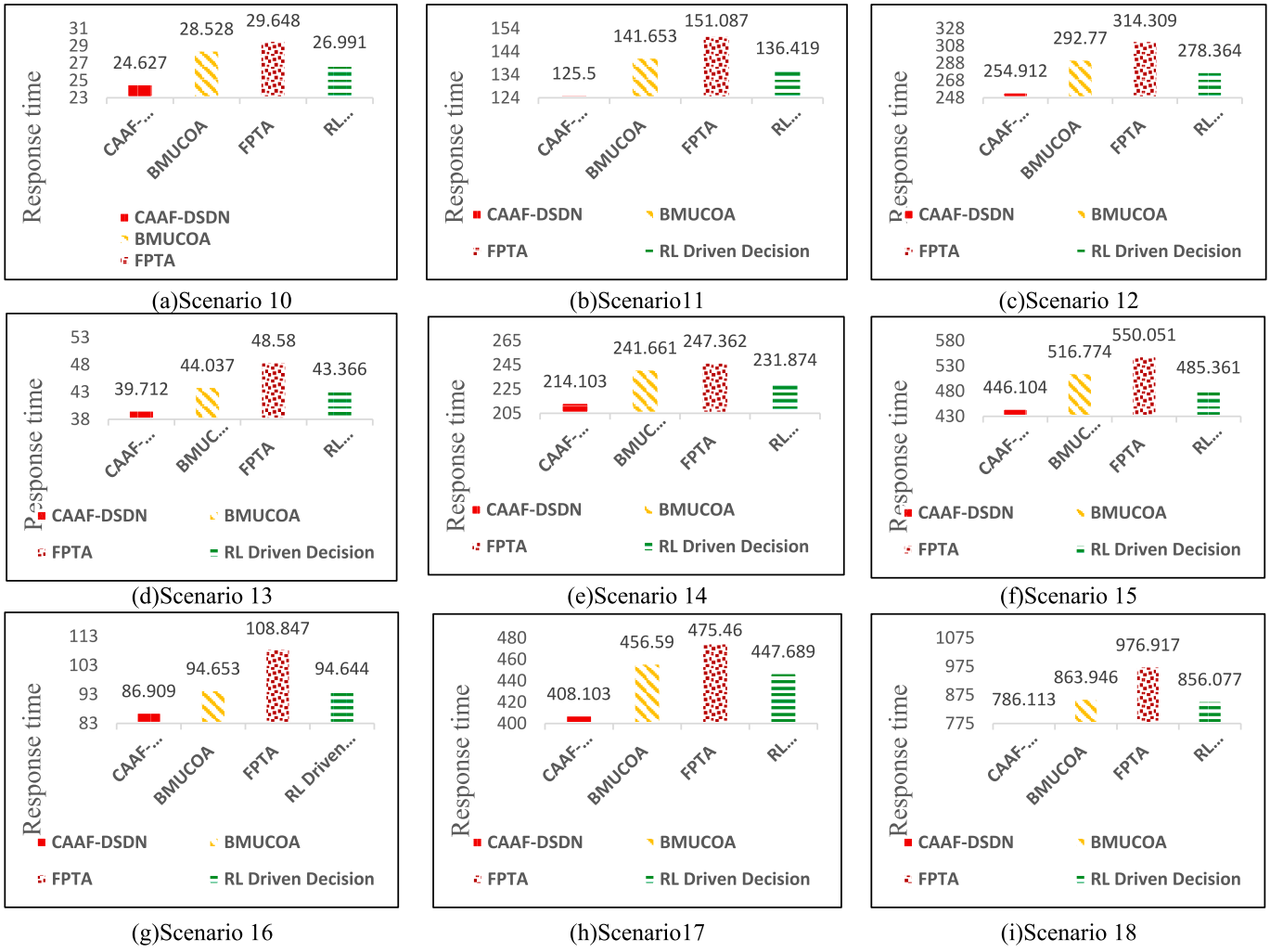

Fig. 9. Response time for 5 controllers and 500 VM in (a)Scenario 10, (b)Scenario 11, (c)Scenario 12, (d)Scenario 13, (e)Scenario 14, (f)Scenario 15, (g)Scenario 16, (h)Scenario 17 and (i)Scenario 18.

- $\overrightarrow{x_q} \bullet \overrightarrow{x_p}$ is the dot product between the two vectors $\overrightarrow{x_q}$,$\overrightarrow{x_p}$.

Eq. (12) and (13) are the main innovations of this study. Using mathematical equations and corresponding deductions, we concluded that such equations can be used to measure the similarity of two vectors. The core of these equations is based on the premise that if the angle between two data is smaller, the two data are likely to be closer to each other. However, increasing the length of two data increases the effect of the angle between the two data and subsequently reduces the similarity between the two vectors. The word "effect" here means reducing or increasing the similarity. In these equations, the operator (.) represents the inner product of two matrices, and the operator ||x|| represents the calculation of the length of the vector X based on the Euclidean distance. To calculate the similarity ratio between two vectors, the greatest possible similarity between the two vectors should be considered, that is when the two vectors are at an angle of zero degrees to each other. Then, the actual similarity is calculated using Eq. (12). Therefore, the similarity ratio of two data in matrix S is calculated as the similarity matrix of dataset X using Eq. (13). Eq. (14) is derived from the Cauchy-Schwarz inequality, which is one of the well-known concepts in mathematics. However, we have made some changes to it with our own deductions.

Cosine similarity, a metric used to measure the degree of similarity between two vectors in multidimensional spaces, is computed using the dot product and the normalization of the vectors. The formula for the cosine similarity between two vectors $\overrightarrow{x_p}$ و $\overrightarrow{x_q}$ is expressed as:

$$\text{Cosine Similarity}\,(\overrightarrow{x_p}\,\overrightarrow{x_q}) = \frac{\overrightarrow{x_q} \bullet \overrightarrow{x_p}}{\|\overrightarrow{x_q}\| \bullet \|\overrightarrow{x_p}\|} \tag{14}$$

Where:

- $\overrightarrow{x_p}$ and $\overrightarrow{x_q}$ are feature vectors (or data points) whose similarity needs to be calculated.
- $\overrightarrow{x_q} \bullet \overrightarrow{x_p}$ is the dot product between the two vectors.
- $\|\overrightarrow{x_q}\| \bullet \|\overrightarrow{x_p}\|$: represents the lengths or norms of the vectors $\overrightarrow{x_p}$ and $\overrightarrow{x_q}$, respectively.

The result of this formula is a value between −1 and 1, indicating the degree of similarity between the two vectors. A value of 1 indicates perfect similarity (the vectors are aligned in the same direction), while a value of −1 indicates complete dissimilarity (the vectors are aligned in opposite directions).

According to Eq. (14), the cosine similarity metric is primarily influenced by the angle between the two vectors, rather than their magnitudes (norms). As the angle between the vectors decreases, the similarity between the corresponding data points increases. Conversely, as the angle increases, the similarity approaches zero, indicating greater dissimilarity. It is important to note that the magnitudes (norms) of the vectors do not directly affect the cosine similarity, as the vectors are normalized before computing their dot product. Therefore, even if the

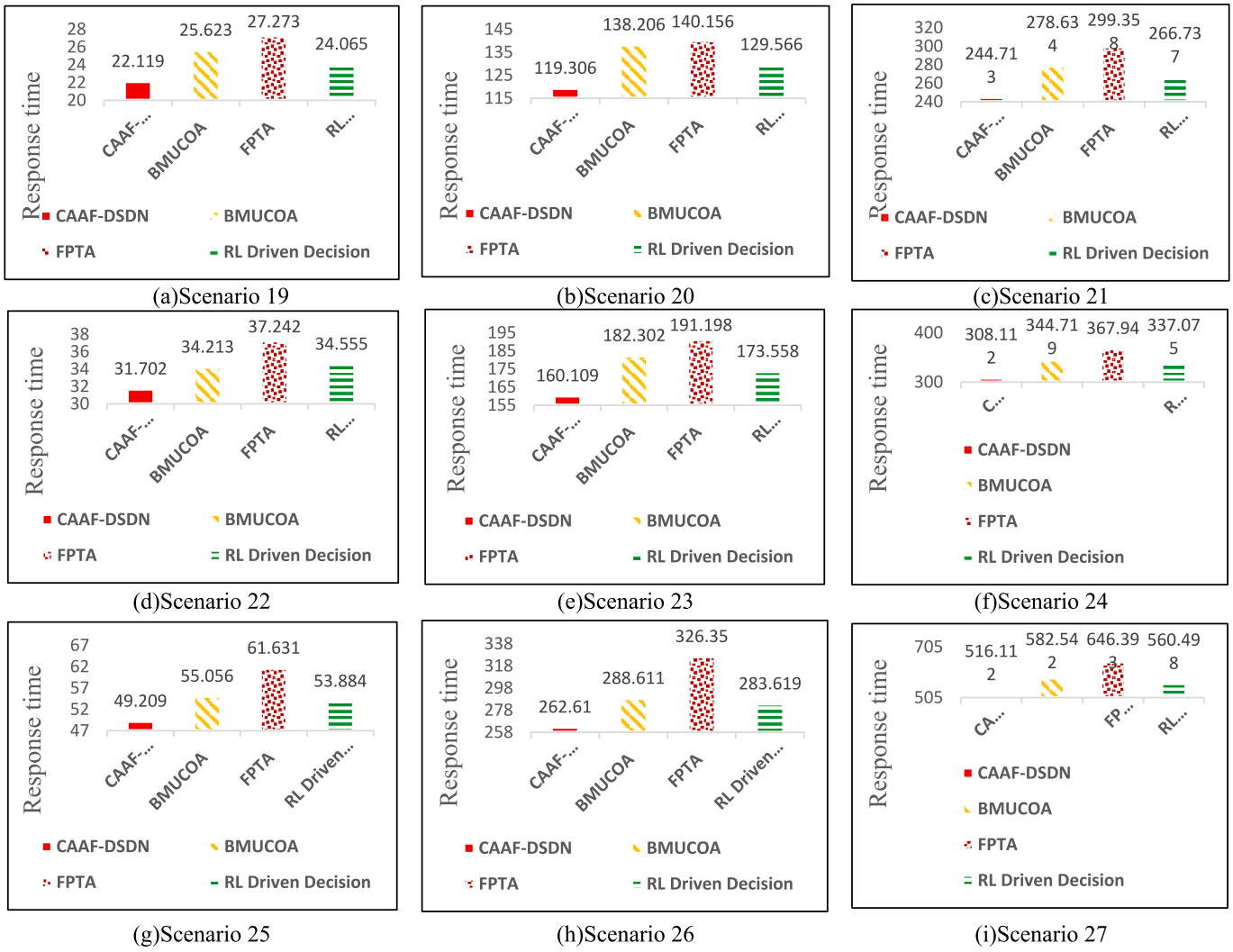

Fig. 10. Response time for 5 controllers and 1000 VM in (a)Scenario 19, (b)Scenario 20, (c)Scenario 21, (d)Scenario 22, (e)Scenario 23, (f)Scenario 24, (g)Scenario 25, (h)Scenario 26 and (i)Scenario 27.

vectors have different magnitudes, their similarity remains high as long as they are aligned in the same direction. However, as the angle between the vectors increases, the effective distance between the data points in the cosine similarity space increases, thereby reducing their similarity. Thus, the key determinant of similarity in this metric is the directional alignment of the vectors, not their absolute magnitudes.

4.6. Calculation of similarity threshold

To determine the similarity threshold in matrix S, the density of the distribution of values in the matrix must first be calculated. This density value is used to identify some matrix elements as similarities and some matrix elements as differences. Let T be the similarity threshold. Then for each value S_{ij} in matrix S, Eq. (15) must hold.

1. Density calculation: First calculate the density of the distribution of values. This can involve calculating the mean and standard deviation of the values in matrix S.

2. Threshold definition: Choose a similarity threshold T based on the density of the distribution and your specific needs. Such a threshold can be determined as follows.

$$T = \mu + \sigma \bullet k \tag{15}$$

where μ is the mean of the values in matrix S, σ is the standard deviation, and k is a regularization coefficient chosen based on your specific needs.

3. Determination of similarity or difference: Then, based on the threshold T, it is determined whether S_{ij} is considered acceptable similarity or difference.

$$\begin{cases} x_i \approx x_j if s_{ij} \geq T \\ x_i \not\approx x_j if s_{ij} < T \end{cases} \tag{16}$$

This process facilitates the determination of an appropriate threshold value for assessing similarity within the similarity matrix S, based on the density of the distribution of values. To validate this approach, it is essential to examine the effect of changes in distribution density on the similarity matrix S. As the distribution density increases, the likelihood that more values will surpass the threshold also increases. Consequently, when the distribution exhibits higher density, the maximum values in the matrix S are more likely to exceed the threshold. This finding suggests that adjusting the distribution density can be instrumental in selecting an optimal threshold value for identifying similarity or dissimilarity between vectors. Therefore, accurately estimating the distribution density and determining the appropriate threshold are critical steps in making more effective decisions for evaluating similarity and dissimilarity between vectors. The following methods and techniques are used to improve the similarity of data:

[6] Data preprocessing:

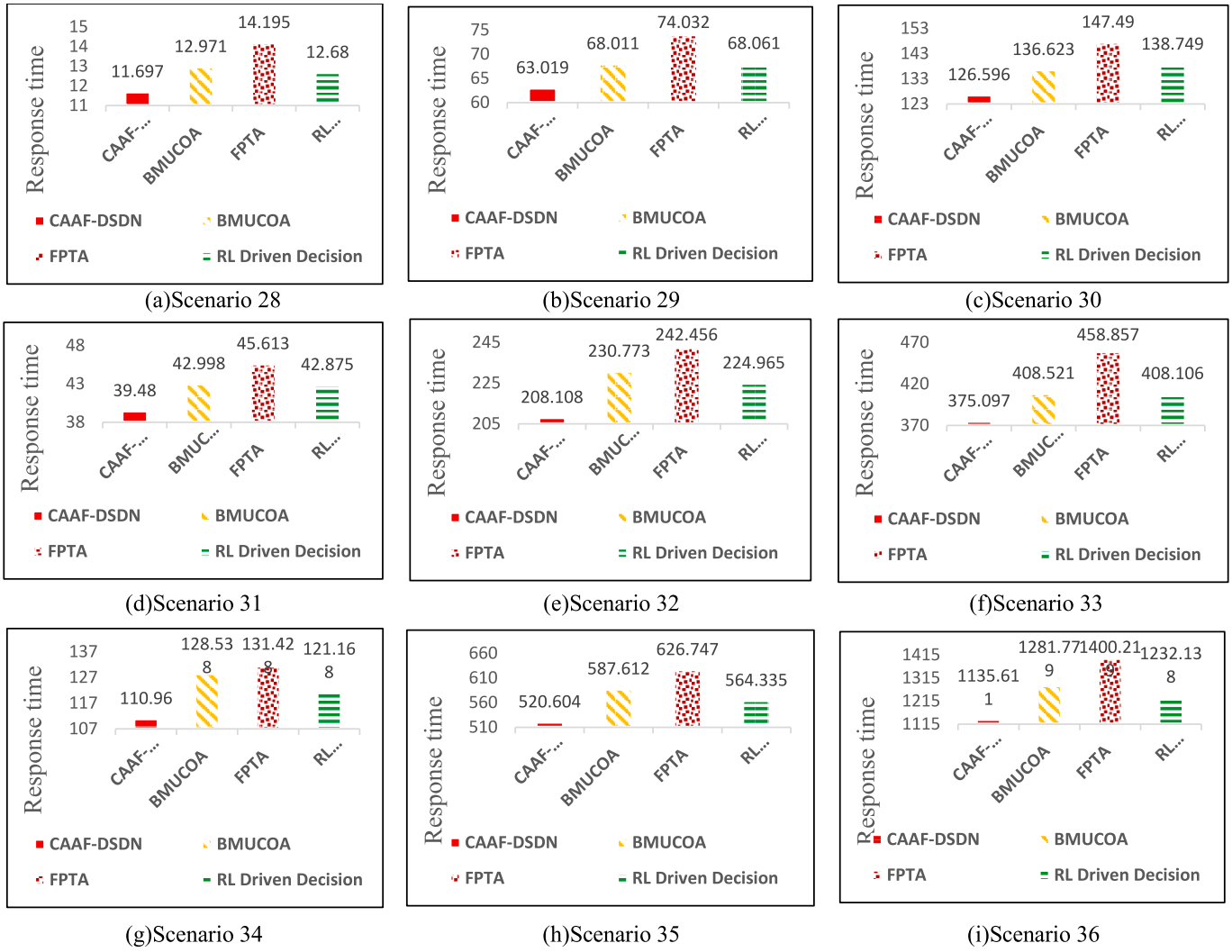

Fig. 11. Response time for 20 controllers and 300 VM in (a)Scenario 28, (b)Scenario 29, (c)Scenario 30, (d)Scenario 31, (e)Scenario 32, (f)Scenario 33, (g)Scenario 34, (h)Scenario 35 and (i)Scenario 36.

- Normalization: Scale the data to the same scale to reduce the effect of large values on the similarities.
- Data cleaning: Remove noisy or incomplete data to increase the accuracy of the analysis.

⁶ Feature selection:

- Dimensionality reduction: Use methods such as principal component analysis (PCA) or t-SNE to reduce the dimensionality of the data and identify more important features.
- Relevant feature selection: Keep only the features relevant to the problem to reduce clutter.

⁶ Using advanced similarity metrics:

- Similarity matrix: Use a similarity matrix to analyze equations between data. Test different metrics such as Euclidean distance, cosine, or machine learning-based metrics.
- Machine learning models: Use supervised or unsupervised learning models to improve data similarity.

⁶ Parameter tuning:

- Threshold tuning: Fine-tune similarity thresholds by testing different parameters.
- Algorithm tuning: Optimize the parameters of clustering or classification algorithms.

⁶ Using clustering techniques:

- Density-based clustering: Use algorithms such as DBSCAN or OPTICS to identify more complex structures.
- Hierarchical clustering: Use hierarchical techniques to identify equations between data at different levels.

⁶ Interactive analysis:

- Use visual analysis: Use data visualization techniques to better understand patterns and relations.

Combining these methods can effectively improve data similarity and obtain better results in analysis and modeling.

4.7. Similarity matrix factorization

Similarity matrix factorization is considered as one of the main innovations in this study. Such factorization based on mathematical equations provides the necessary tools to improve density-based clustering methods. In particular, the QR decomposition algorithm is used as a valid mathematical technique. This allows for changes to Eq. (14) and (15) and leads to improved results. To evaluate the density of the distribution in the similarity matrix S, the eigenvectors and eigenvalues of each row are calculated. The eigenvectors v_i and eigenvalues λ_i introduced in Eq. (17) play a key role in identifying the density features of the data and provide deeper insight into the structure of the data.

$$\left(\overrightarrow{v_1}.\overrightarrow{\lambda_1}\right), (\overrightarrow{v_2}.\overrightarrow{\lambda_2}). \bullet \bullet \bullet .(\overrightarrow{v_i}.\overrightarrow{\lambda_i}) \tag{17}$$

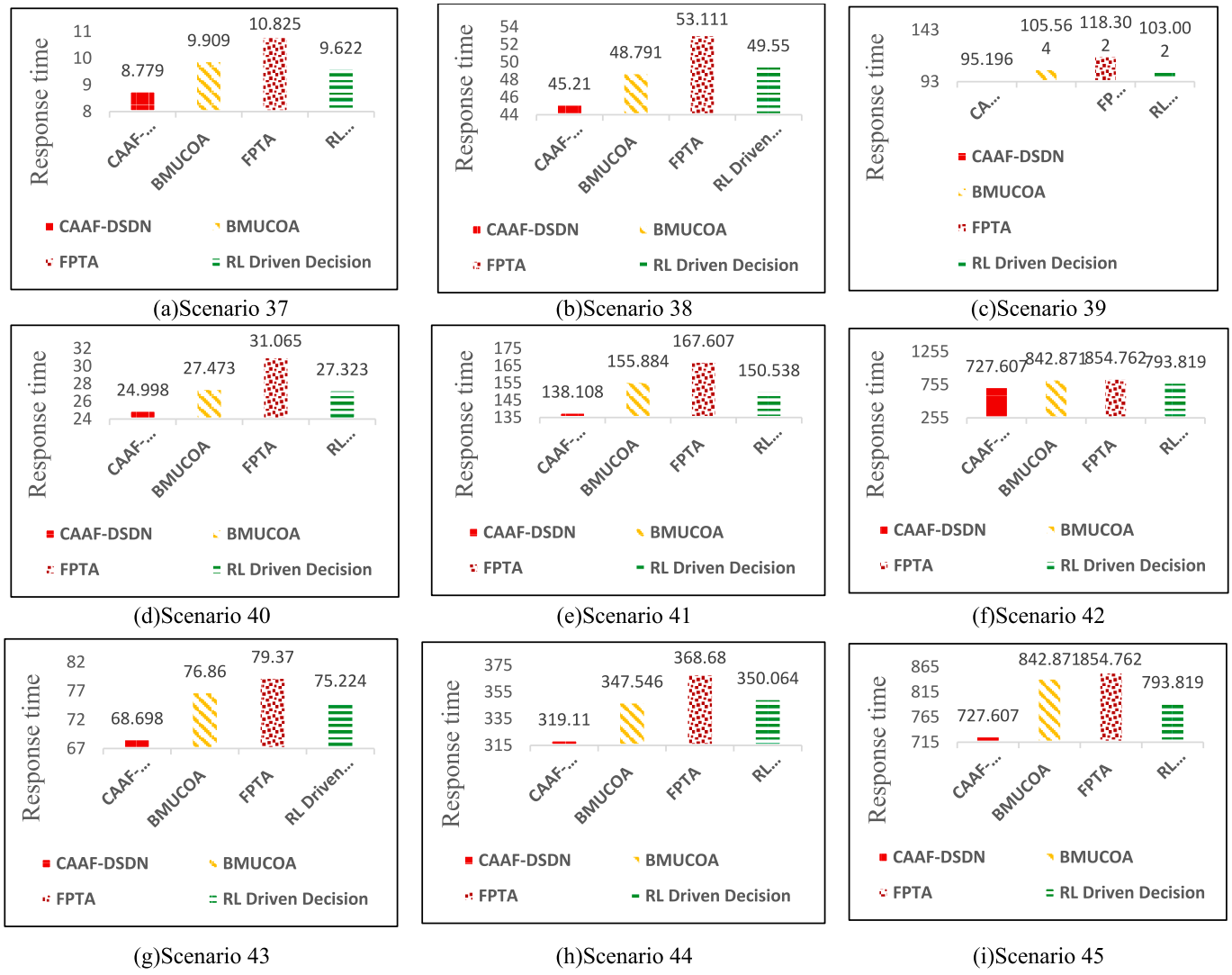

Fig. 12. Response time for for 20 controllers and 500 VM in (a)Scenario 37, (b)Scenario 38, (c)Scenario 39, (d)Scenario 40, (e)Scenario 41, (f)Scenario 42, (g) Scenario 43, (h)Scenario 44 and (i)Scenario 45.

where $\overrightarrow{v_i}$ and $\overrightarrow{\lambda_i}$ are the eigenvector and eigenvalue corresponding to each principal component of the similarity matrix S, respectively. These equations help us gain deeper insights into the density features of the data and achieve more effective improvements in the clustering process. In the context of the similarity matrix S, the eigenvalues $\overrightarrow{\lambda_i}$ and eigenvectors $\overrightarrow{v_i}$ encapsulate essential information about the structure and distribution of the dataset, revealing underlying patterns and relationships. These values can be used as criteria for identifying outliers, assessing cluster density, or analyzing data distribution in a high-dimensional space. According to the following equation, each eigenvector $\overrightarrow{v_i}$ is associated with an eigenvalue λ_i.

$$\overrightarrow{S_i v_i} = \overrightarrow{\lambda_i v_i} \tag{18}$$

This equation establishes the correspondence between the similarity matrix, eigenvectors, and eigenvalues and playing a crucial role in accurately analyzing the density and distribution of data in the clustering process. The reliability and accuracy of these values are evaluated through rigorous mathematical criteria and statistical validation tests, ensuring that the derived factors precisely characterize the properties of the similarity matrix S [2]. To obtain a precise solution for Eq. (18) and determine the eigenvalues λ and eigenvectors $\overrightarrow{v}$ of the similarity matrix s, the QR algorithm is systematically applied, as detailed in Algorithm 2.

4.7.1. Steps of the QR decomposition algorithm

In this section, the application of the QR decomposition algorithm to solve Eq. (18) is discussed. This algorithm is used to calculate the eigenvalues λ and eigenvectors $\overrightarrow{v}$. It is assumed that the similarity matrix S is a square matrix.

The steps of the QR algorithm are as follows:

1. Initial QR decomposition: Decompose the matrix S into two matrices Q and R such that QR=S, where Q is an orthogonal matrix and R is an upper triangular matrix. In this context
   - Q is an orthogonal matrix, which means that its columns form an orthonormal set, satisfying the condition $I = Q^T Q$.
   - R is an upper triangular matrix, which means that all elements below the main diagonal are zero.
2. Matrix update: Update the matrix S by Eq. (19), where R_K and Q_K are the results of the QR decomposition in the current iteration

$$S_K = Q_K \bullet R_K \tag{19}$$

Where:

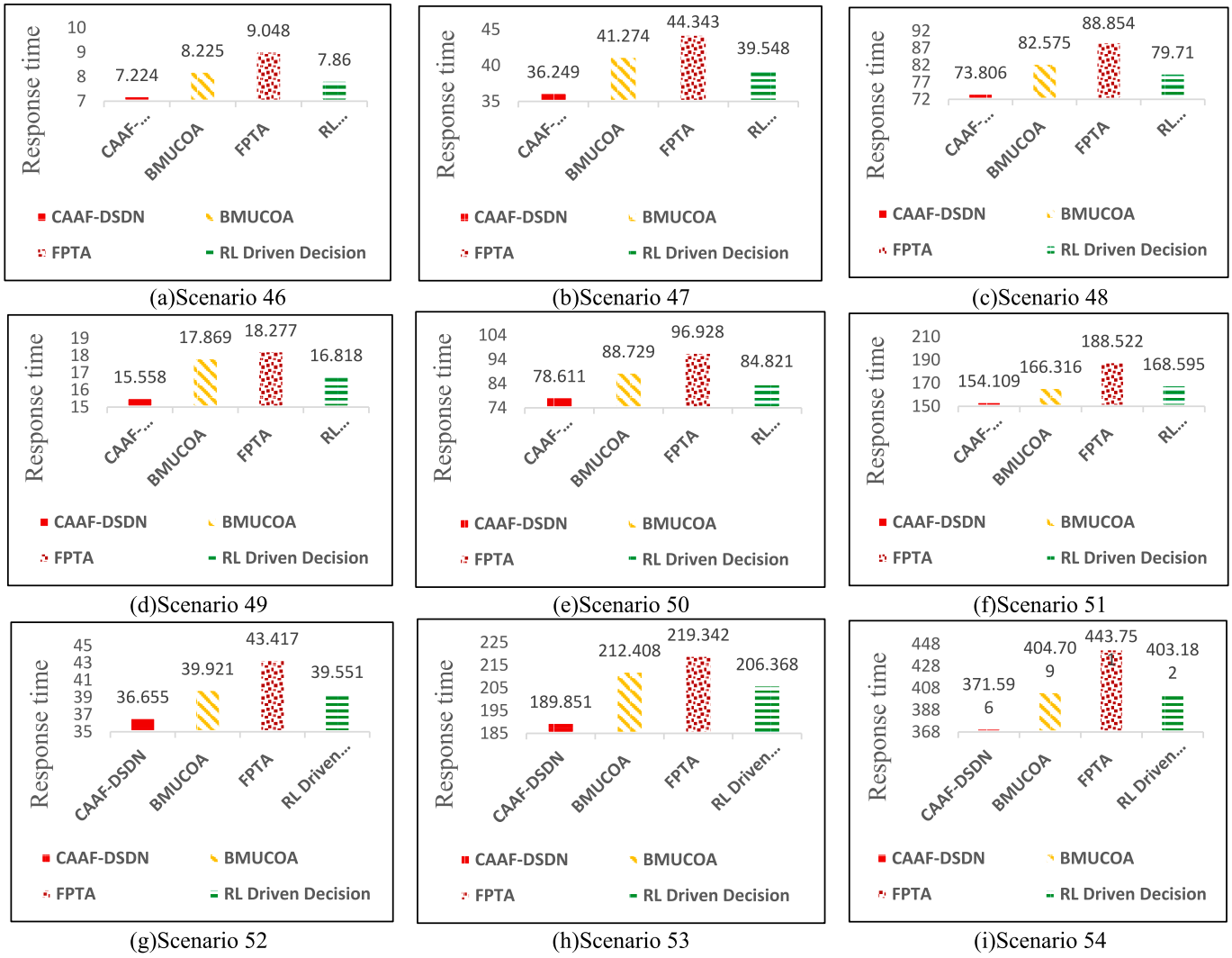

Fig. 13. Response time for 20 controllers and 1000 VM in (a)Scenario 46, (b)Scenario 47, (c)Scenario 48, (d)Scenario 49, (e)Scenario 50, (f)Scenario 51, (g)Scenario 52, (h)Scenario 53 and (i)Scenario 54.

- $R_k and Q_k$ represent the results of the QR decomposition at the k-th iteration.
- This process is performed as a simple multiplication of two matrices.

Eq. (19) demonstrates that the matrix S is updated in each iteration and transformed into a new form through the multiplication of Q and R. This iterative process eventually leads S_K to converge to an upper triangular matrix, where the eigenvalues of S appear along its main diagonal. Due to its numerical stability and high accuracy, this method is widely used in computational problems and eigenvalue calculations.

3. Process iteration: At each iteration, a new matrix S is obtained using the matrix R obtained from the previous step. This process continues until the variation between two consecutive matrices S is minimized

$$S_{new} = QR \tag{20}$$

Eq. (20) represents the QR decomposition of a matrix, where:

- S_{new} is a newly generated matrix derived from the original matrix S.
- Q is an orthogonal matrix, which means its columns are mutually perpendicular and satisfy the property $I = Q^T Q$.
- R is an upper triangular matrix, which means all elements below its main diagonal are zero.

This equation states that any square matrix S can be decomposed into the two matrices Q and R. In the subsequent steps of the algorithm, these matrices are used to compute eigenvalues and eigenvectors.

4. Eigenvalue calculation: After several iterations, the eigenvalues obtained from the diagonal of the matrix S are obtained. These values represent the essential characteristics of the similarity matrix s.
5. Eigenvector extraction: After calculating the eigenvalues, use the same steps to extract the eigenvectors. By inserting the eigenvalues into the equation $\overrightarrow{Sv} = \overrightarrow{\lambda v}$, the eigenvectors v can be obtained.
6. Analysis of the results: Analyze the obtained eigenvalues and eigenvectors to gain a better understanding of the structure and characteristics of the similarity matrix s.

As an effective tool in numerical analysis and computation, the QR algorithm provides the conditions for calculating eigenvalues and eigenvectors, leading to improved clustering and data analysis processes. Due to its fast convergence capability and high accuracy in calculations, this algorithm is used in many fields such as density-based clustering and data analysis. QR decomposition is effective in reducing the impact of anomalies in the data and improving clustering accuracy. It also helps analyze the data structure and identify patterns and hidden relationships in the data as an effective mathematical tool. This allows researchers

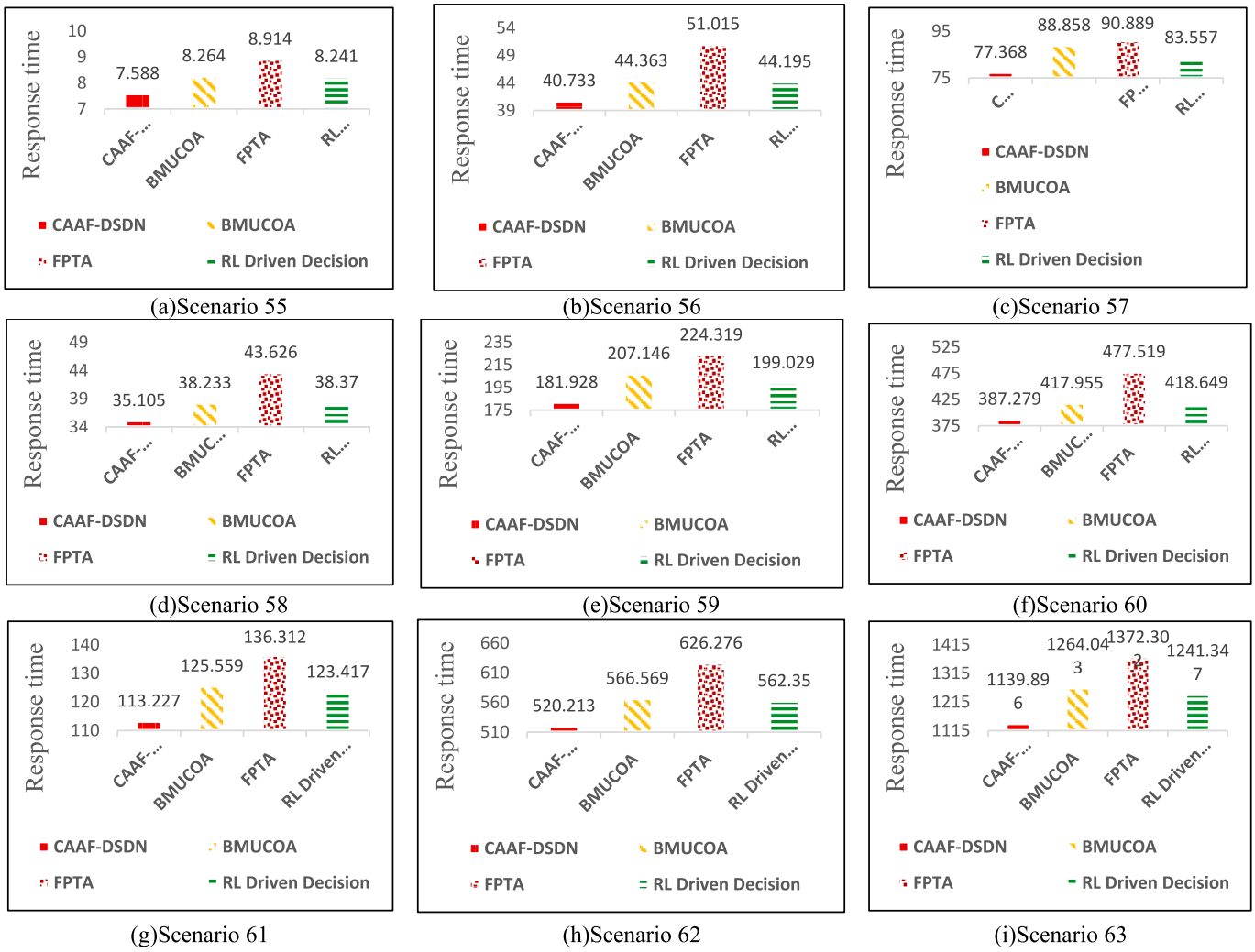

Fig. 14. Response time for 100 controllers and 300 VM in (a)Scenario 55, (b)Scenario 56, (c)Scenario 57, (d)Scenario 58, (e)Scenario 59, (f)Scenario 60, (g)Scenario 61, (h)Scenario 62 and (i)Scenario 63.

and experts to achieve better results in clustering and data analysis processes.

According to Fig. 5, QR factorization is performed on matrix R and matrix Q through an iterative process. The number of iterations of this algorithm is determined according to the problem conditions and the dimensions of the matrices at the time of simulation. The calculations are performed in two main parts. In the first part, the matrix S is decomposed into orthogonal and triangular matrices. In the second part, the eigenvalues and eigenvectors of the matrix Q are calculated. In the initial step, the matrix R is set equal to the matrix S. Then, new matrices Q and R are calculated and created for each iteration.

The QR matrices are calculated based on the properties of the orthogonal matrix. In an orthogonal matrix, the inner product of its two column vectors is equal to zero, which is expressed mathematically as follows:

$$I = Q^T Q \tag{21}$$

where Q^T is the transpose of the matrix Q and I is equivalent to the identity matrix. According to this property, the column vectors of the matrix Q are orthogonal and normalized.

$$Q^k = I - \frac{r^k e^k (e^k)^T}{||r^k||^2} \tag{22}$$

Eq. (22) is considered the key innovation of this study. The new clustering method presented in this section relies on the factorization rule. In Eq. (14),

- The value Q^k represents the calculated value of Q at the k^{th} step
- Also, r^k is the corresponding column in the matrix R at the k^{th} computation.
- e^k is the corresponding column in the unit vector $\vec{I}$,
- $||r^k||$ reflects the maximum value of the matrix R at the k^{th} computation.

Using Eq. (22), the value of Q can be calculated as Eq. (23).

$$Q = I - \sum_{1}^{n} \frac{r^k e^k (e^k)^T}{||r^k||^2} \tag{23}$$

The new matrix Q is the result of the QR factorization. The new matrix R is also extracted using the same process. According to Eq. (19) and (20), we have:

$$S = R_{new} Q_{new} \tag{24}$$

Eq. (24) represents the re-decomposition of matrix S after an update step, where:

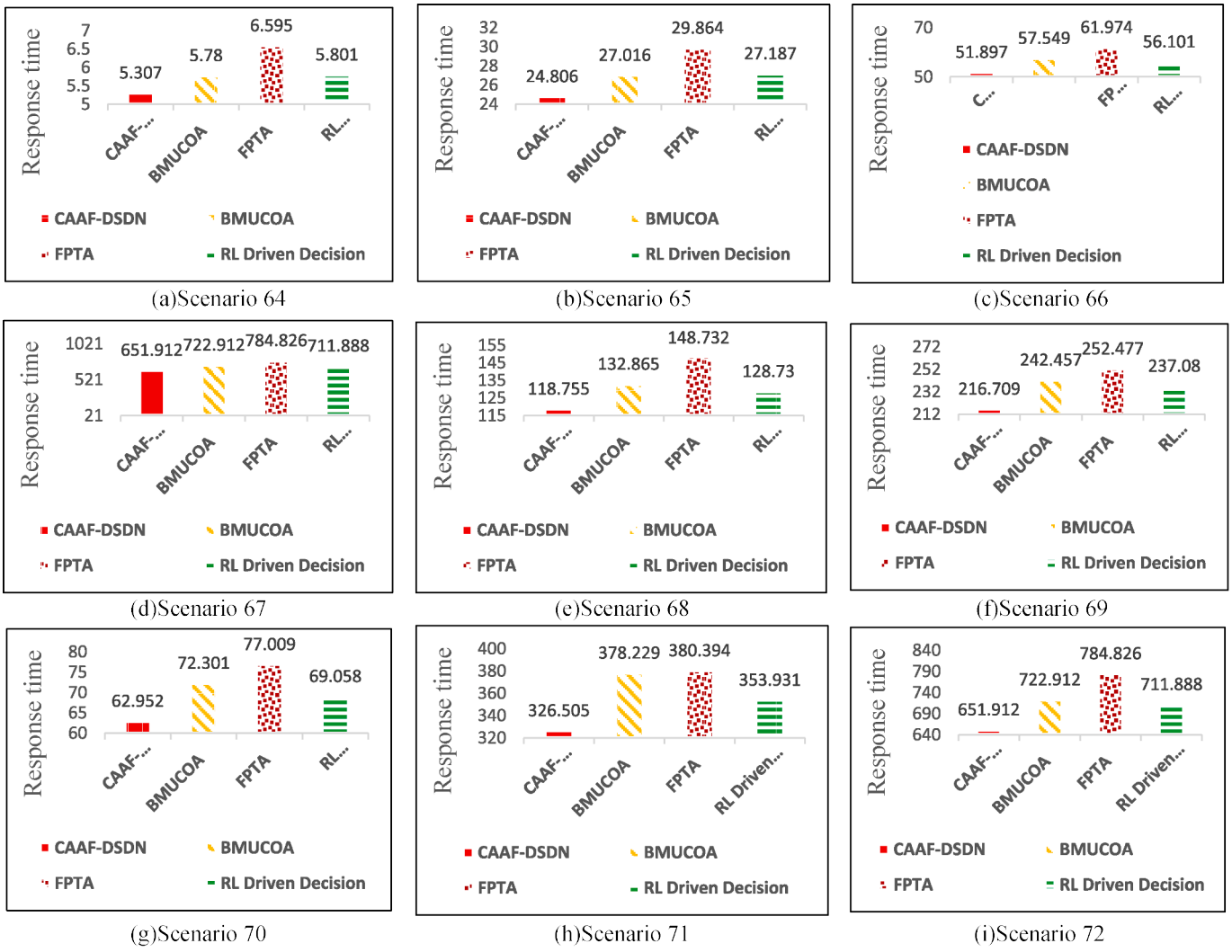

Fig. 15. Response time for 100 controllers and 500 VM in (a)Scenario 64, (b)Scenario 65, (c)Scenario 66, (d)Scenario 67, (e)Scenario 68, (f)Scenario 69, (g)Scenario 70, (h)Scenario 71 and (i)Scenario 72.

- R_{new} is the newly obtained upper triangular matrix from the QR decomposition.
- Q_{new} is the updated orthogonal matrix obtained in the current iteration of the algorithm.

This relation is employed in iterative processes to enhance computational accuracy and facilitate data analysis in clustering applications.

This shows that the matrix S can be effectively decomposed into new orthogonal and triangular matrices, which in turn leads to improved accuracy and efficiency in computational processes related to clustering and data analysis. The matrix Q is an orthogonal matrix, so the above Equation can be rewritten as follows:

$$Rv_i = \lambda_i v_i \tag{25}$$

According to Eq. (17), the eigenvalues λ are obtained from the main diagonal of the matrix R, because the matrix R has the same eigenvalues as the matrix S. The eigenvalues are calculated from the diagonal elements of the matrix R as follows:

$$\lambda = \begin{bmatrix} \lambda_1 & \cdots & R_{f1} \\ \vdots & \ddots & \vdots \\ R_{f1} & \cdots & \lambda_i \end{bmatrix} = \{R_{11}, R_{22}, \ldots, R_{ff}\} \tag{26}$$

Since S = QR and referring to Eq. (19), the eigenvalue matrix v is extracted from the matrix Q. In other words, each column of the matrix Q corresponds to an eigenvalue v.

$$S^k = R^k.Q^k \tag{27}$$

S^k represents the adjacency matrix calculated in the k^{th} step of the proposed method. In other words, this matrix is the result of performing operations on the matrix S in the k^{th} step. The same concept holds for the two matrices Q and R.

4.7.2. *Clustering threshold estimation*

The calculated eigenvalues and eigenvectors are necessarily ordered by size. This is because the eigenvectors associated with larger eigenvalues usually provide more important information about the similarity of the data. If we assume the distribution of the set S as a Gaussian distribution, to obtain an accurate estimate of the threshold, the probability distribution associated with the matrix S is needed. The relationship of the probability distribution in a set with a Gaussian distribution is expressed as follows:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} exp\left(\frac{-(x-\mu)^2}{2\sigma^2}\right) \tag{28}$$

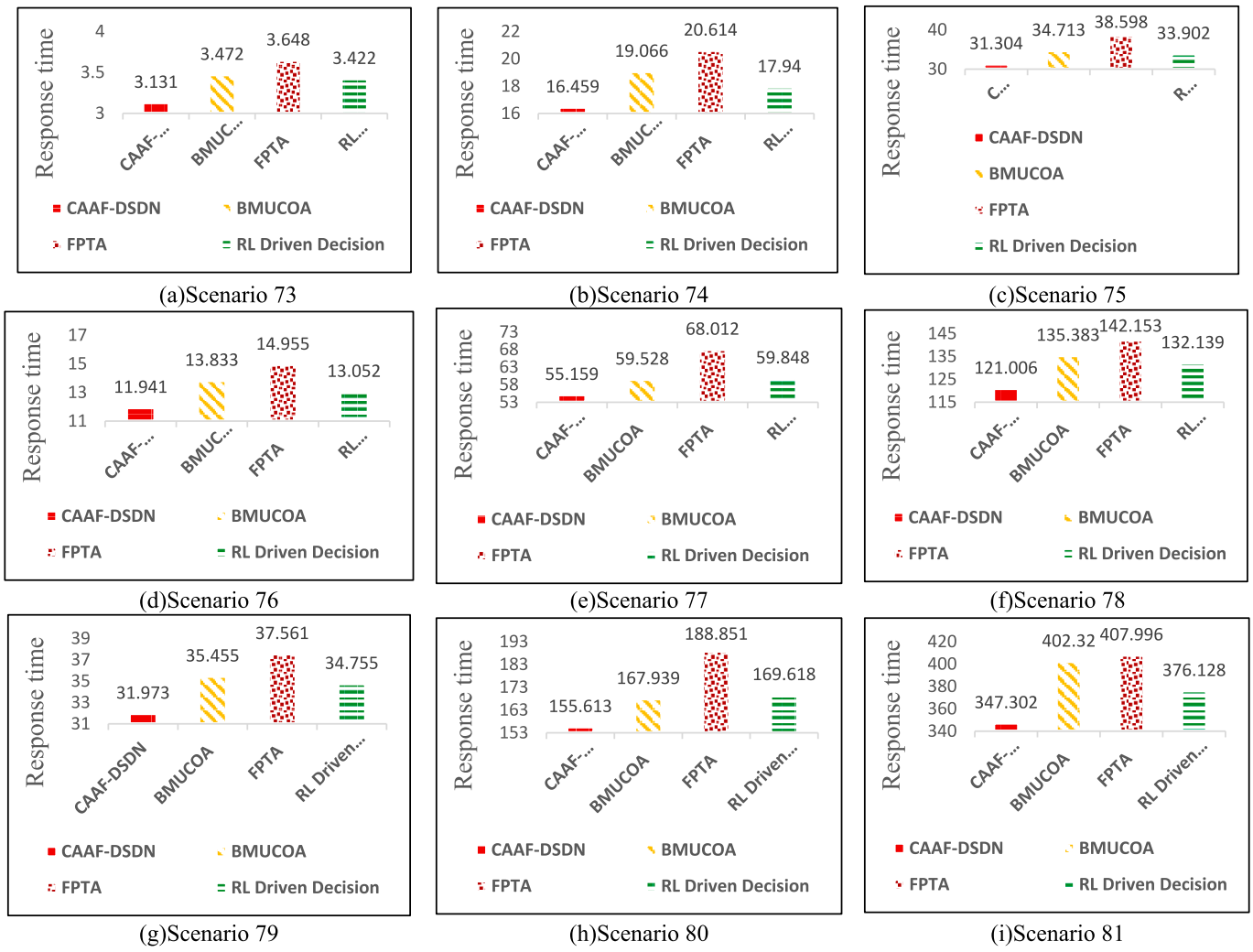

Fig. 16. Response time for 100 controllers and 1000 VM in (a)Scenario 73, (b)Scenario 74, (c)Scenario 75, (d)Scenario 76, (e)Scenario 77, (f)Scenario 78, (g) Scenario 79, (h)Scenario 80 and (i)Scenario 81.

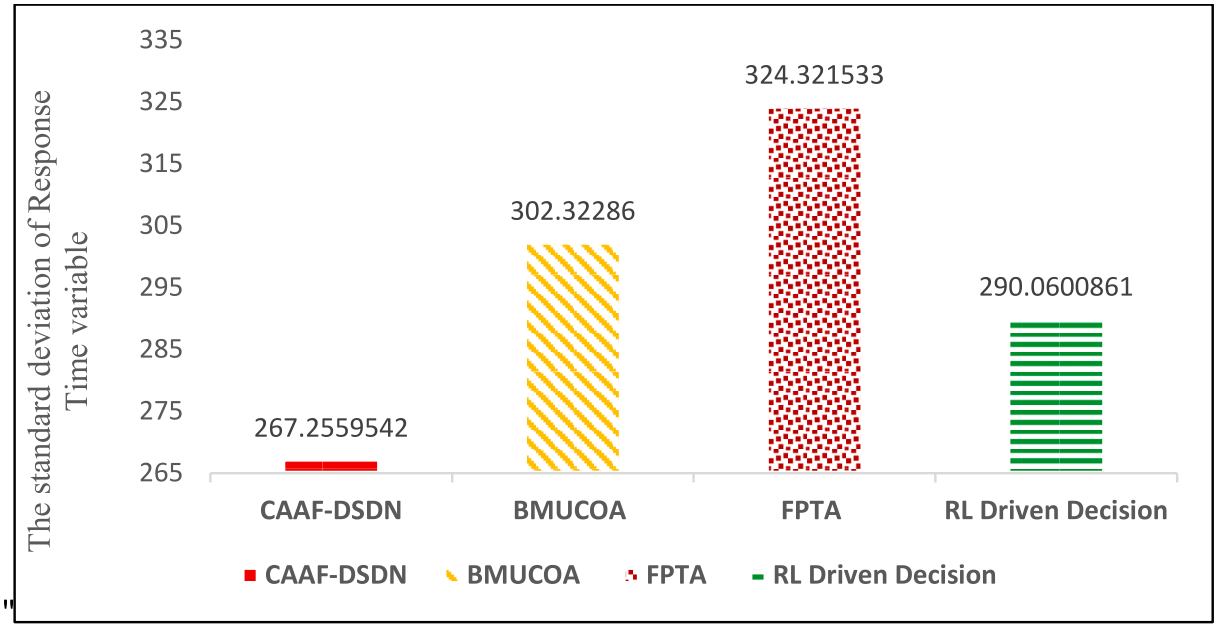

Fig. 17. The Standard Deviation of the Response Time Variable.

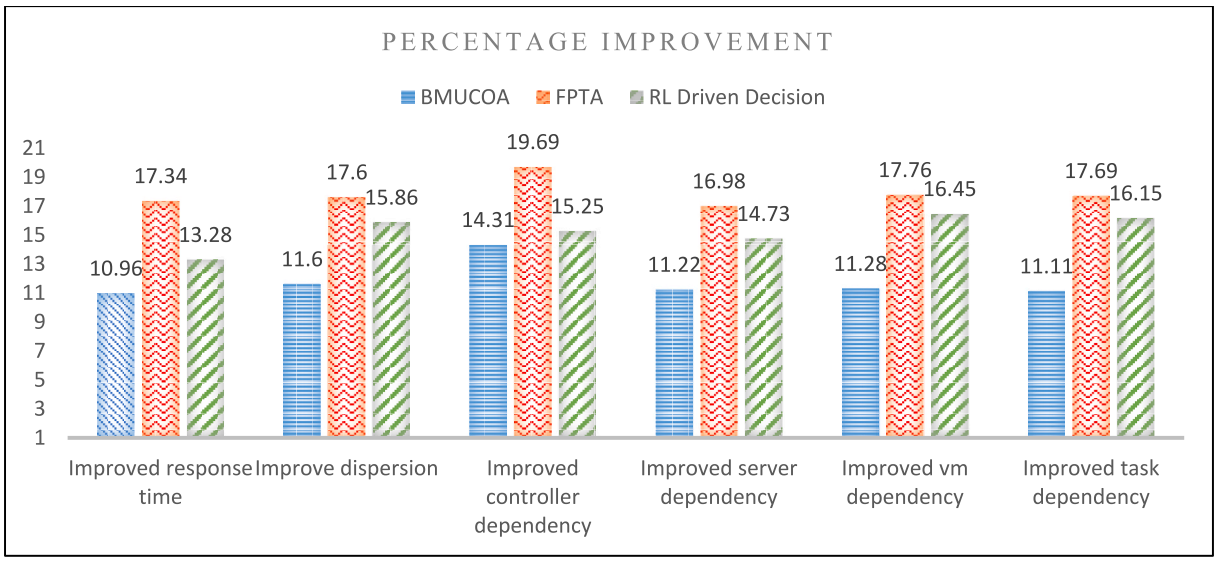

Fig. 18. Comparison of Improvements in CAAF-DSDN Proposed Method over BMUCOA, FPTA, and RL Driven Decision in Terms of Response Time.

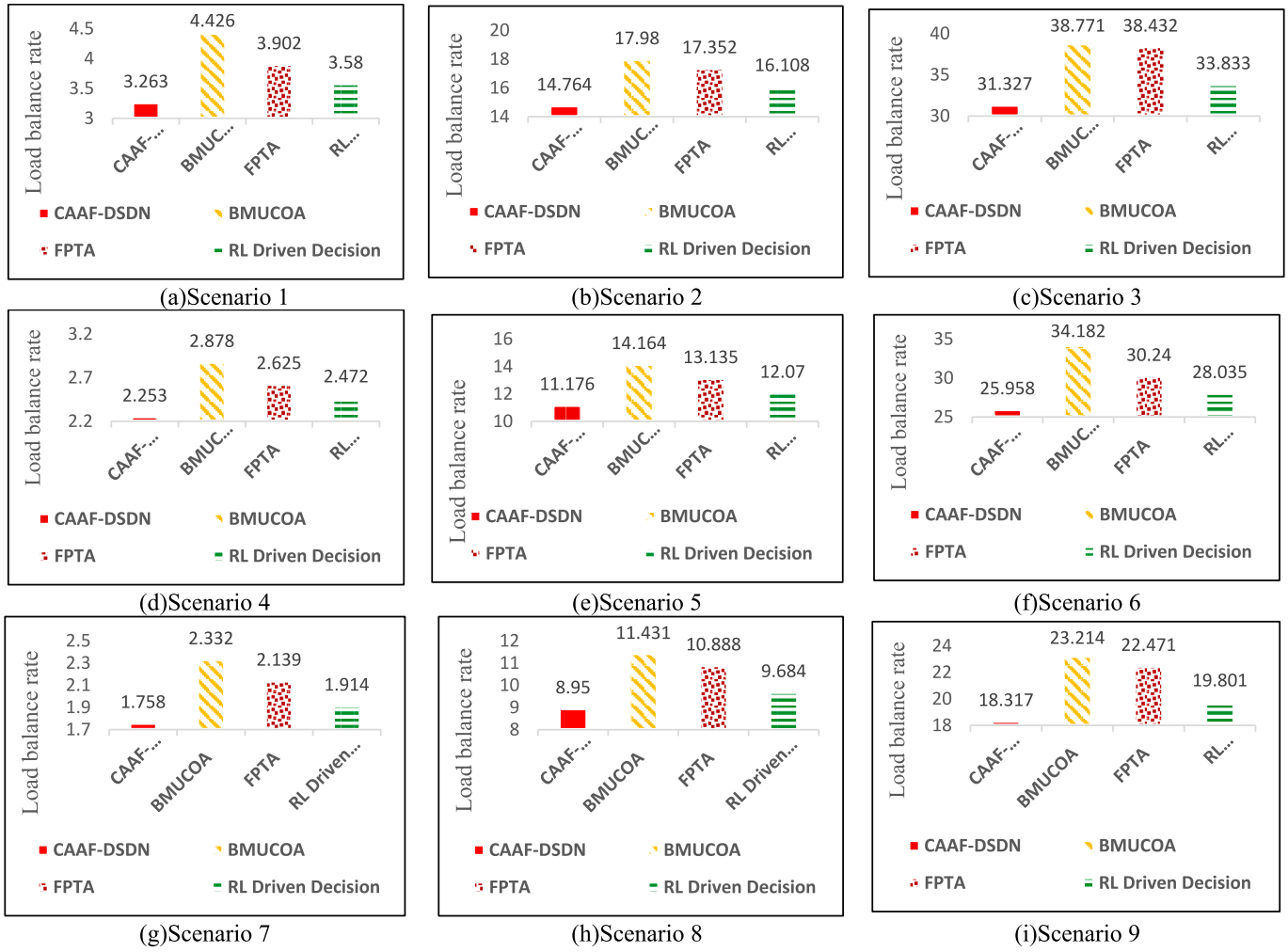

Fig. 19. Load balance rate for 5 controllers and 300 VM in (a)Scenario 1, (b)Scenario 2, (c)Scenario 3 (d)Scenario 4, (e)Scenario 5, (f)Scenario 6, (g)Scenario 7, (h) Scenario 8 and (i)Scenario 9.

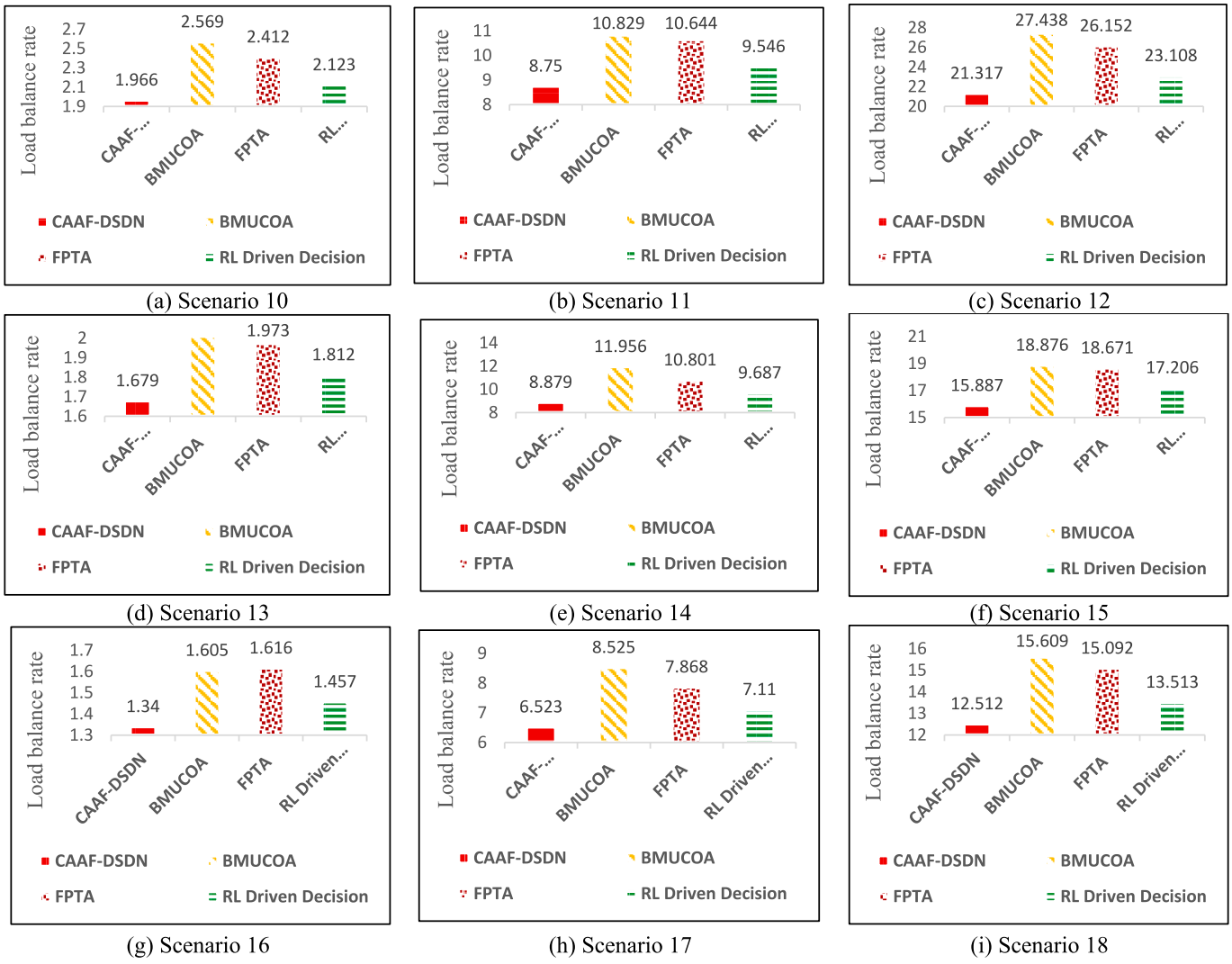

Fig. 20. Load balance rate for 5 controllers and 500 VM in (a) Scenario 10, (b) Scenario 11, (c) Scenario 12, (d) Scenario 13, (e) Scenario 14, (f) Scenario 15, (g) Scenario 16, (h) Scenario 17 and (i) Scenario 18.

- $p(x)$: The probability density function (PDF), which defines the likelihood of observing a specific value x within a normal distribution.
- μ: The mean or expected value of the distribution, indicating the central tendency around which the data is typically distributed.
- σ^2: The variance of the distribution, representing the extent to which the data points deviate from the mean μ. In other words, σ^2 quantifies the dispersion of the data. The standard deviation σ is the square root of the variance.
- *exp*: The exponential function, denoted as e^{-z}, which shapes the distribution by ensuring that probabilities are highest near the mean and decrease progressively as the distance from μ increases.

In summary, this equation represents the normal probability distribution for a random variable x that follows a normal distribution. The PDF of this distribution forms a bell-shaped curve, centered at μ, with its spread governed by σ^2.

- σ^2 represents the variance of the normal distribution, while σ is the standard deviation. By replacing the variable x with its corresponding value in the matrix S, Eq. (29) for the probability distribution s_{ij} based on the decomposition of the value $p(x)$ into its corresponding value p_{ij} is written as follows:

$$p_{ij} = P(s_{ij}) \tag{29}$$

$$p_{ij} = \frac{1}{2\pi\sigma^2} exp(\frac{-(\lambda_i - \lambda_j)}{2\sigma^2}) exp(\frac{-||v_i - v_j||^2}{2\sigma^2}) \tag{30}$$

- s_{ij}: The probability (or probability density) associated with the element *iandj* in the matrix S.
- p_{ij}: This represents the probability associated with the value x_{ij} in the matrix S.
- x_{ij}: The element *iandj* in the matrix S, denoted as x. This value is substituted for x in the original equation.

The remaining components follow the same explanations provided for Eq. (28). Eq. (30) is considered one of the innovations of this study. To calculate the threshold, the value p_{ij} is calculated. According to the analysis obtained from the data, when a data is transformed to a certain value, the definition of $\propto$ and its increase lead to an increase in the similarity threshold of the cluster C. If the distribution of the set S is considered in the form of other distributions such as exponential, logarithmic, and other distributions instead of the Gaussian distribution, Eq. (30) also changes based on the characteristics of the distribution in question. Such changes in the likelihood function depend on the specific characteristics of each distribution and must necessarily reflect the

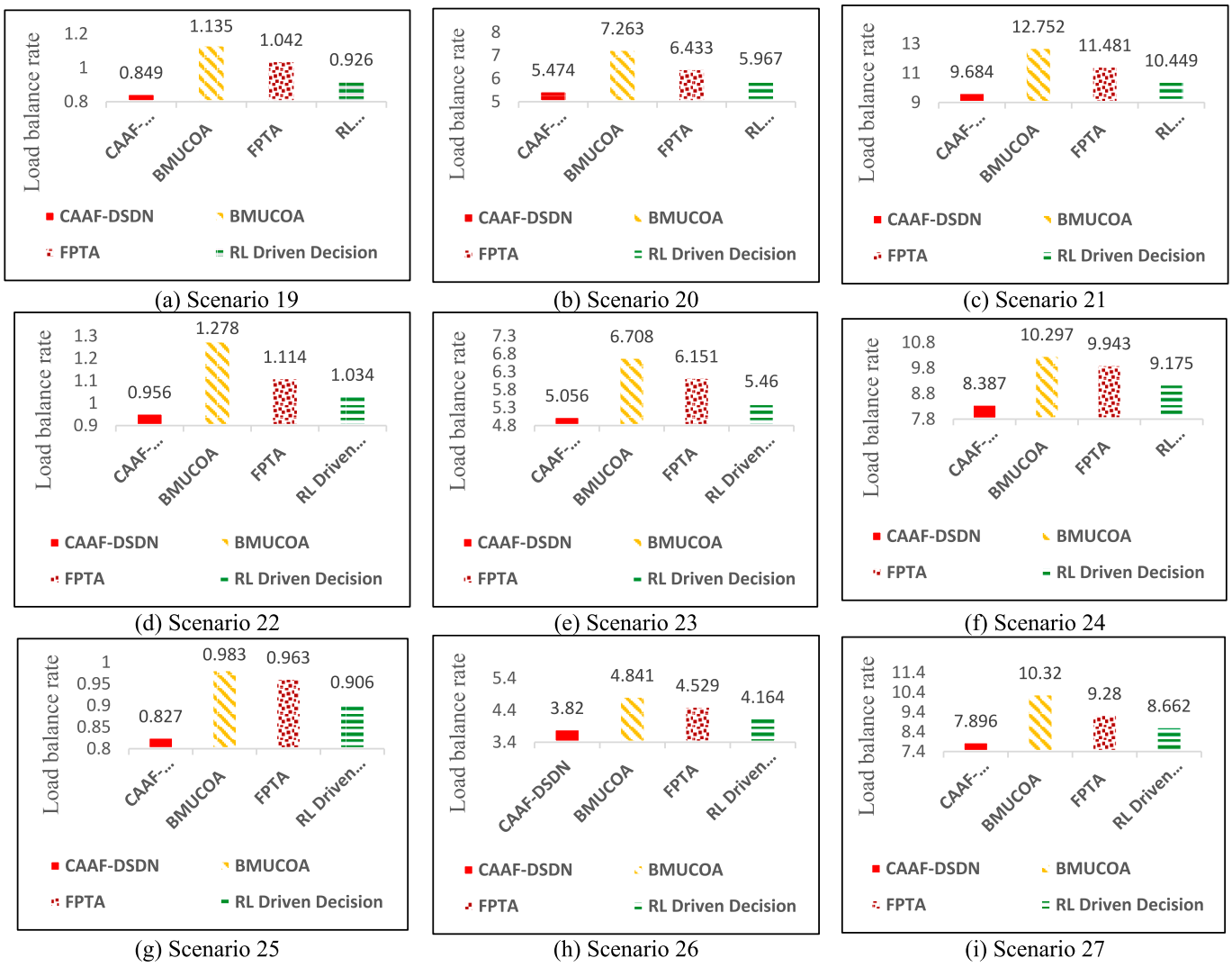

Fig. 21. Load balance rate for 5 controllers and 1000 VM in (a) Scenario 19, (b) Scenario 20, (c) Scenario 21, (d) Scenario 22, (e) Scenario 23, (f) Scenario 24, (g) Scenario 25, (h) Scenario 26 and (i) Scenario 27.

characteristics of that distribution.

4.8. Estimation of the adjacency matrix

According to the determination of the similarity threshold of the data in the cluster C based on the principles of adaptive algebra, the resulting threshold value depends on two factors: the average probability of occurrence of the value s_{ij} and the cumulative power of probability distribution s_{ij}. Considering this proportion, Eq. (31) is rewritten as follows. This type of rewriting emphasizes the simultaneous effect of the dependence of the mean and the cumulative power of probability distribution and considers the relationship between these factors in determining the similarity threshold of the data more accurately.

$$T_c \propto \sum_{i=1}^{m} \sum_{j=1}^{m} (p_{ij})^k \tag{31}$$

The power k in the distribution matrix represents a regularization parameter the value of which depends on the characteristics of the data and the objectives of the problem. To calculate the value of k, the values $\lambda = \{\lambda_1.\lambda_2.\ldots.\lambda_m\}$ are sorted in descending order in each row. The result of the sorting is denoted by λ'.

- T_c: The similarity threshold for the data in cluster C. This value serves as a measure to determine the degree of similarity among the data points within a specific cluster.
- p_{ij}: The probability of the occurrence of a specific value s_{ij} in the probability distribution matrix. This value indicates the extent to which a particular value appears within the data of the cluster.
- k: The power of the distribution matrix, acting as a tuning parameter. The value of this parameter is determined based on the data characteristics and the objectives of the problem.
- $\sum_{i=1}^{m}\sum_{j=1}^{m}$: This symbol represents the summation over all values in the probability distribution matrix, indexed by *iandj* .
- $\lambda = \{\lambda_1.\lambda_2.\ldots.\lambda_m\}$: The eigenvalue set, which is used to determine the value of k.
- λ': The sorted version of the eigenvalues λ, arranged in descending order. This sorting is used to select the optimal value of k.

The value of k is determined by examining the sorted eigenvalues λ'. This selection ensures that the similarity threshold T_c aligns with the data characteristics and is optimized for the given problem. Then, the value of k is determined using the condition in Eq. (32). This condition examines and evaluates the sorted values of λ' and determines when to choose a value of k to satisfy the objectives of the problem. This computational approach ensures that the power k is optimized and

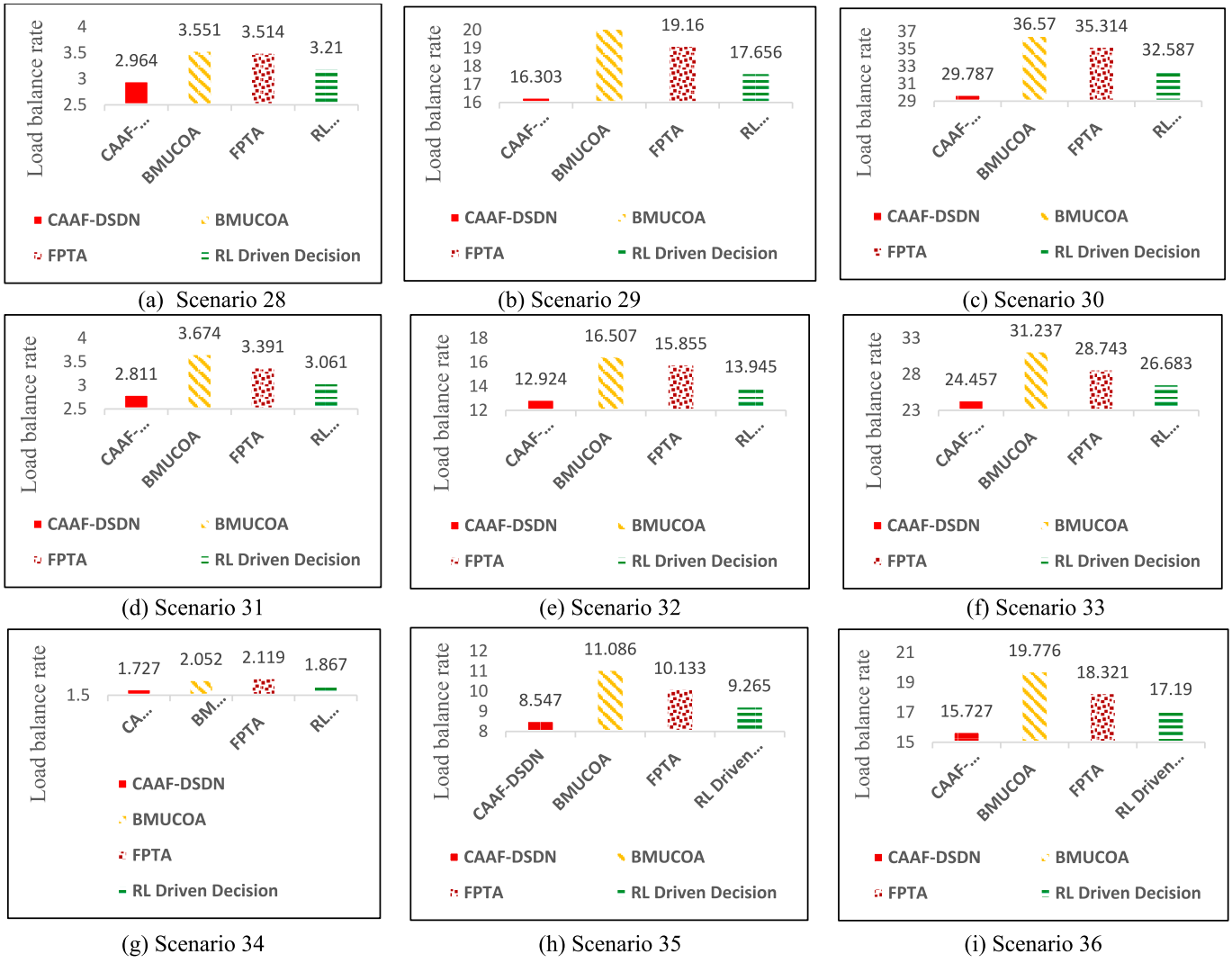

Fig. 22. Load balance rate for 20 controllers and 300 VM in (a) Scenario 28, (b) Scenario 29, (c) Scenario 30, (d) Scenario 31, (e) Scenario 32, (f) Scenario 33, (g) Scenario 34, (h) Scenario 35 and (i) Scenario 36.

exactly matches the characteristics of the data.

$$\sum_{i=1}^{k} \lambda_i' \geq T_c \tag{32}$$

Increasing the value of k transforms the cluster similarity threshold c into a sinusoidal curve. This means that if the probability of occurrence of the value s_{ij} is high, the cluster similarity threshold c decreases. Conversely, if the probability of occurrence of s_{ij} is low, the cluster similarity threshold c increases. To transform Eq. (32) into a more accurate equivalence Equation, a linear estimation coefficient needs to be added to get closer to the true value. Therefore, adding coefficient α changes Eq. (32) to Eq. (33). The coefficient α, as a modifier, helps to fine-tune the similarity threshold and obtain more optimal results. These modifications transform the Equation into a more dynamic and flexible model to more accurately model the effect of different probabilities.

$$T_c = \alpha \sum_{i=1}^{m} \sum_{j=1}^{m} (p_{ij})^k \tag{33}$$

The coefficient α is an adjustable parameter based on the convergence speed and data range. Increasing the value of α increases the similarity threshold of cluster C. In other words, two more similar samples are placed in the same cluster. To begin with, it is suggested to consider the value$\alpha = 0.001$. This initialization is based on the results of simulations. To reduce the influence of the initial value in the determining T_c (assuming that s follows a Gaussian distribution), the initial value of T_c is estimated by Eq. (34). This equation refers to calculating the average similarity in the similarity matrix and leads to reducing the effect of the initial value and increasing the accuracy of the estimation. This optimization method based on the Gaussian distribution leads to better adaptation to the nature of the data, improving the convergence speed of the model, and ensuring the final accuracy of the analysis.

$$T_c = \frac{1}{m^2} \sum_{i=1}^{m} \sum_{j=1}^{m} s_{ij} \tag{34}$$

As soon as the type of distribution in the S matrix is changed, Eq. (34) is also modified. For example, if the data follows another distribution such as exponential or Poisson, the similarity parameters and formulas are changed accordingly to maintain the accuracy and efficiency of the calculations. By default, if the distance between two data samples is less than or equal to the similarity threshold, the two samples are recognized as neighbors. In the context of adaptive algebra, the adjacency relationship between two samples is evaluated using Eq. (35). Considering the characteristics of the data distribution and the similarity threshold, this equation can specify the exact adjacency that matches the characteristics of the problem. This mechanism aims to better adapt to

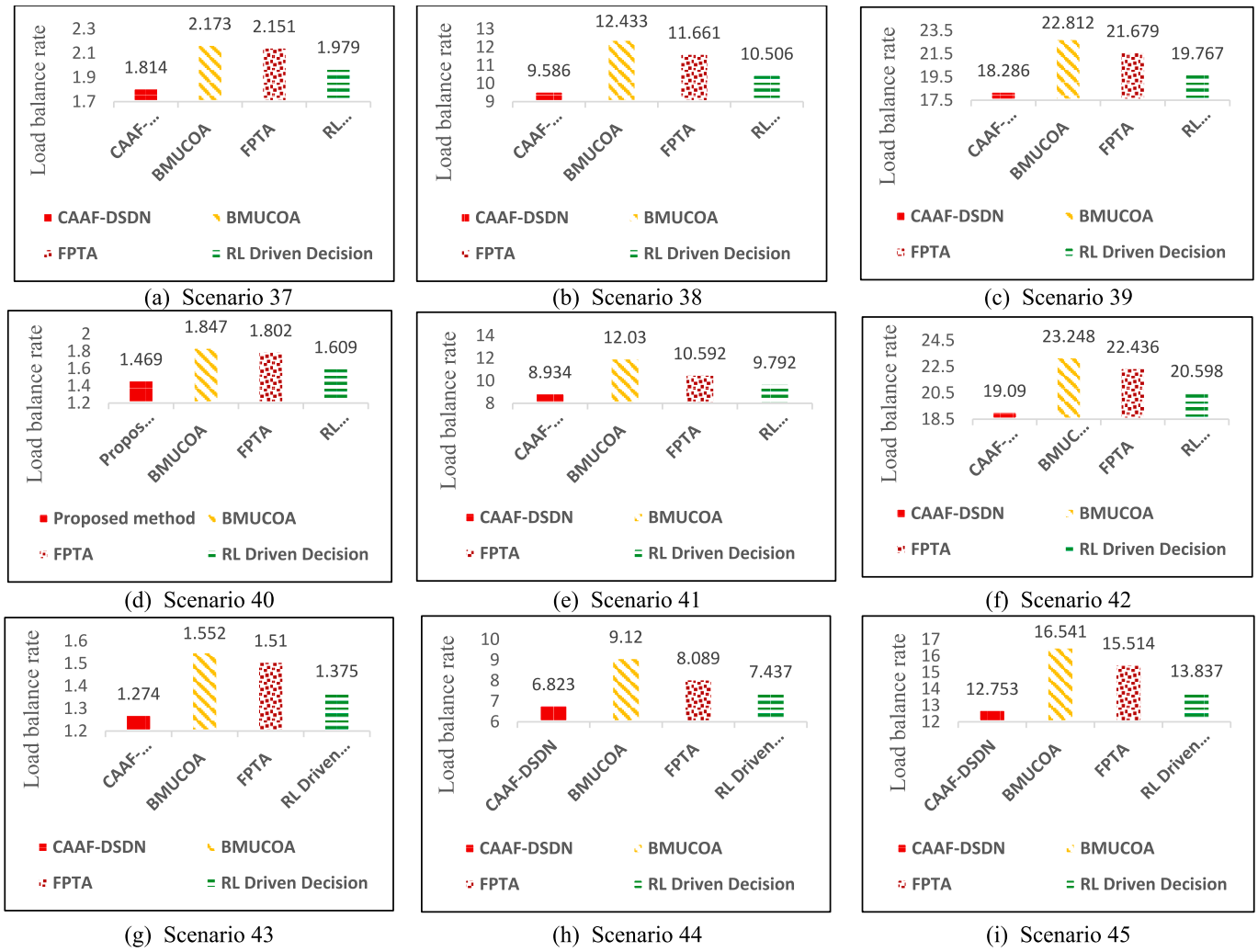

Fig. 23. Load balance rate for 20 controllers and 500 VM in (a) Scenario 37, (b) Scenario 38, (c) Scenario 39, (d) Scenario 40, (e) Scenario 41, (f) Scenario 42, (g) Scenario 43, (h) Scenario 44 and (i) Scenario 45.

nonlinear or complex data and more accurately analyze the clustering structure and adjacency between data.

$$N_{ij} = 1 + exp\left(\frac{-\left|\left(\left|x_i - x_j\right|\right)\right|^2}{s_{ij}^2}\right) \tag{35}$$

- N_{ij}: The similarity or proximity function between two samples $x_i and x_j$.
- $x_i and x_j$: The values of the two data samples being compared
- $\left|\left(\left|x_i - x_j\right|\right)\right|^2$: The squared Euclidean distance between the two samples, indicating the degree of difference or dissimilarity between them.
- s_{ij}^2: The scaling parameter of similarity, which determines the impact of the distance between samples on the proximity function.
- exp (·): The exponential function, which reduces the influence of large distances, playing a crucial role in determining similarity.
- exp (·) + 1: This component acts as a modifier, ensuring that the proximity function value is at least 1. This can be beneficial for computational stability in some models.

4.9. Data assignment and cluster formation

Based on the adaptive algebra and the obtained threshold, the adjacency of two data samples i and j is calculated with a relationship based on the similarity or distance between the two samples. The adaptive algebra evaluates the degree of similarity between the two samples using appropriate criteria and determines the adjacency by comparing this similarity with a specified threshold.

4.9.1. The neighborhood formula utilizing adaptive algebra

The following equation is used to determine the adjacency of two samples i and j:

$$N_{ij} = \begin{cases} 1 iff f_c(S_{ij}) \geq T_c \\ 0 else \end{cases} \tag{36}$$

- (S_{ij}) is the degree of similarity between the two samples i and j. This similarity is defined based on various criteria such as Euclidean distance, cosine similarity, or other similarity functions.
- $f_c(S_{ij})$ is a adaptive function calculating the relationship between data i and j in the form of a similarity measure. This function can be based on various criteria such as the kernel function or other similarity functions.
- T_c is the similarity threshold determined based on the data distribution or other problem parameters. If the similarity between i and j is greater than or equal to the threshold, the two data are considered to be neighbors.

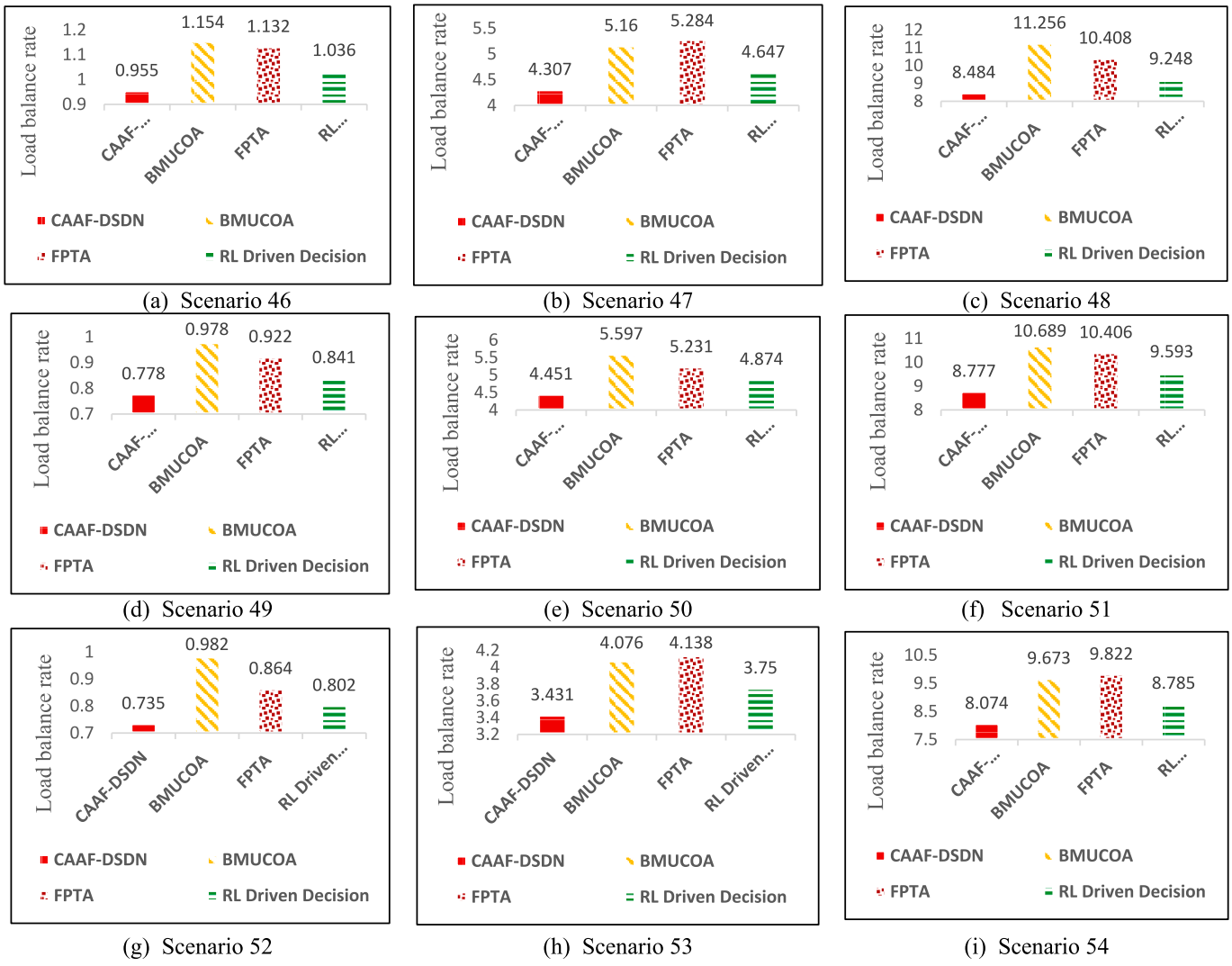

Fig. 24. Load balance rate for 20 controllers and 1000 VM in (a) Scenario 46, (b) Scenario 47, (c) Scenario 48, (d) Scenario 49, (e) Scenario 50, (f) Scenario 51, (g) Scenario 52, (h) Scenario 53 and (i) Scenario 54.

- N_{ij} is a binary variable equal to 1 if two samples i and j are adjacent, otherwise 0.

4.9.2. Computational steps

1. Calculate similarity or distance: First, calculate the similarity or distance between data i and j using an appropriate function such as Euclidean distance $d(i.j)$ or cosine similarity cos(θ)

   The distance or similarity between $(j, i) = .S_{ij}$

2. Compare with threshold T_c: Compare the calculated similarity or distance with the similarity threshold T_c. If the similarity value S_{ij} is greater than or equal to T_c, the data i and j are considered to be adjacent. Finally, the adjacency of two data is determined based on the value of N_{ij}. This adaptive method allows for accurate and efficient calculation of the adjacency between data according to the obtained similarity threshold and adaptive algebra. This process can be effective in clustering and data analysis problems. Eq. (37) represents the adjacency based on adaptive algebra. To enhance readability and improve the understanding of the algorithm's logic for the readers, all variables and functions used in algorithm 2, Fig. 5, are precisely explained in Table 6

$$N_{ij} = \begin{cases} 1 + exp\left(\frac{-|(|x_i - x_j|)|^2}{s_{ij}^2}\right) iff_c(S_{ij}) \geq T_c \\ 0 else \end{cases} \tag{37}$$

Where:

- $x_i and x_j$: The two data samples for which we aim to measure the similarity or proximity
- $|(|x_i - x_j|)|^2$: The squared distance between these two data points, which reflects the degree of difference between them.
- s_{ij}^2: The scaling parameter of similarity, which influences the sensitivity of the function to changes in distance.
- $exp\left(\frac{-|(|x_i - x_j|)|^2}{s_{ij}^2}\right)$: The exponential function, which reduces the impact of large distances and limits similarity to a controlled value.
- $f_c(S_{ij})$: A function applied to the value S_{ij}, which may represent a specific characteristic of the data (such as cluster density or correlation).
- T_c: A critical threshold that determines whether the value S_{ij} is large enough to consider the data points as neighbors.

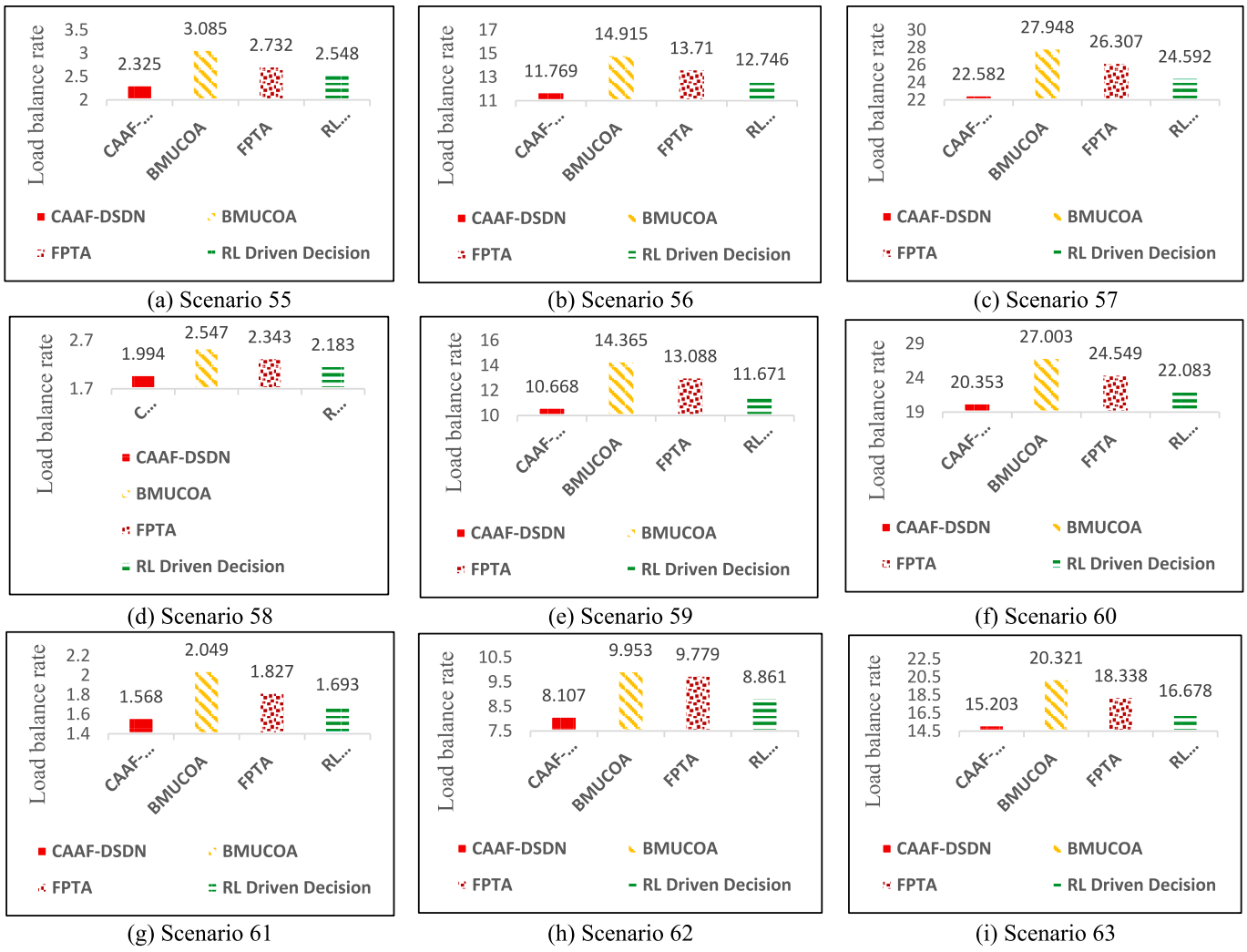

Fig. 25. Load balance rate for 100 controllers and 300 VM in (a) Scenario 55, (b) Scenario 56, (c) Scenario 57, (d) Scenario 58, (e) Scenario 59, (f) Scenario 60, (g) Scenario 61, (h) Scenario 62 and (i) Scenario 63.

It is important to note that if the value of $f_c(S_{ij})$ is greater than or equal to the threshold T_c, the value N_{ij} is computed using the exponential formula; otherwise, it is set to zero, indicating that the two samples are not considered neighbors. This equation is utilized in adaptive clustering algorithms, machine learning, and complex data analysis.

$$x_i \, is \, neighbour \, x_j \, if \, \forall g | \, N_{ij} \geq N_{ig} \tag{38}$$

In Eq. (38), x_i is considered a neighbor of x_j if:

$$\forall g | N_{ij} \geq N_{ig}$$

This condition ensures that the similarity measure N_{ij} between data points x_i and x_j is greater than or equal to that of any other data point x_g. In other words, x_j is identified as the most similar data point to x_i among all potential candidates. This criterion is fundamental in adaptive clustering algorithms and machine learning techniques, as it establishes a structured neighborhood relationship that facilitates accurate data grouping and pattern recognition.

Due to the presence of the conditional function, this model offers the capability for finer tuning and adapts well to nonlinear data. A value of zero for samples that do not meet the condition signifies that this method is also applicable for constructing neighborhood graphs or analyzing cluster structures. As the distance between two data samples increases, the value of the exponential function *exp* decreases. This decrease indicates a decrease in the degree of adjacency or similarity between the two data samples. As a result, the greater the distance between the data, the probability of adjacency and correspondence between them decreases. After completing the matrix *N*, the data are sorted in descending order based on the number of neighbors. Subsequently, the clustering process begins using Algorithm 3. Using the number of neighbors, this algorithm places data with the largest number of neighbors in the initial clusters. Gradually, other data are assigned to appropriate clusters. This method effectively determines the structure of the clusters according to the degree of data adjacency.

Fig. 6, is designed to cluster data based on adjacency and similarity matrix in the proposed CAAF-DSDN method. By sorting data based on adjacency and using similarity thresholds, this algorithm creates clusters to aggregate adjacent data into a group. Data with weak adjacency are considered as outliers. The proposed clustering method is summarized in the pseudo-code of Algorithm 4. Fig. 7, uses different steps to cluster data, including calculating the similarity matrix, analyzing data structures, and finally creating clusters based on adjacency relationships. The time efficiency of the proposed CAAF-DSDN algorithm is reviewed in Table 7.

4.9.3. Mathematical analysis and clustering accuracy

In addition to the time complexity analysis, mathematical analysis proves that CAAF-DSDN can reduce the average distance of samples within a cluster. This is done by increasing the accuracy of measuring the similarity between data and the effect of similarity distribution on

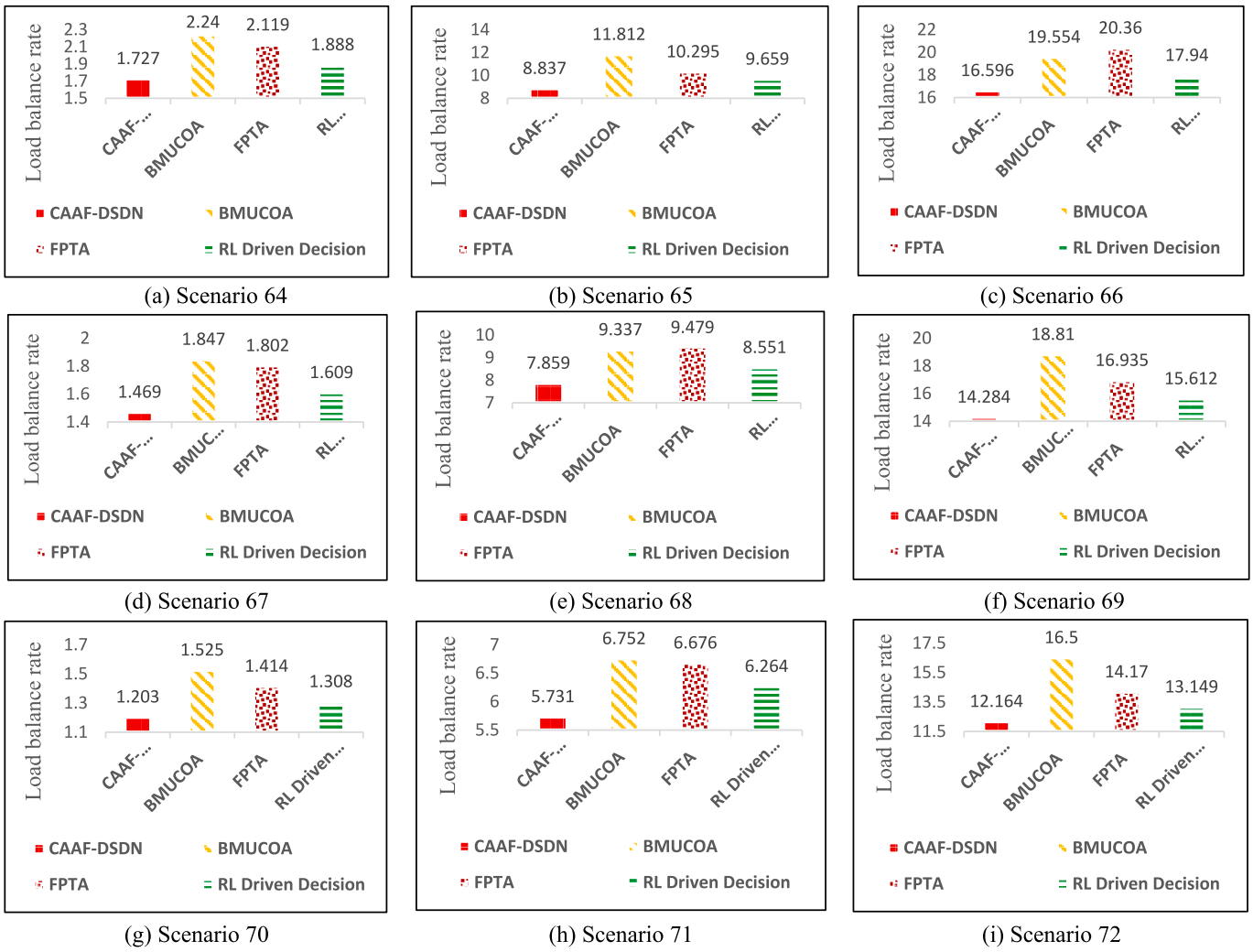

Fig. 26. Load balance rate for 100 controllers and 500 VM in (a) Scenario 64, (b) Scenario 65, (c) Scenario 66, (d) Scenario 67, (e) Scenario 68, (f) Scenario 69, (g) Scenario 70, (h) Scenario 71 and (i) Scenario 72.

determining the adjacency threshold. It also maximizes the difference between the data in each cluster and the data in other clusters. By understanding the similarity distribution in CAAF-DSDN, the algorithm can purposefully establish a more precise order for the similarity in the data structure and adjust the threshold values. Ultimately, this process helps to improve the separation of the boundaries between clusters. This advantage leads to the maximum difference between the data in one cluster and the data in other clusters. Relying on the aforementioned advantages, the clustering performance regarding the Silhouette score is significantly improved.

4.10. Choosing the appropriate server for migration

The clustering output (Sections 4.1 to 4.9) consists of assigning servers to the most similar clusters. If the current server is considered the source server, the cluster containing the current server can develop a list of servers with the lowest possible latency for the selector. The VM can select a server from the servers in the cluster as the destination server to perform the migration. Since servers in a DSDN network lack comprehensive information about the network state, the process of selecting and listing servers must be performed in the controllers. The selected server should be as far away from the high-traffic areas of the network as possible. Therefore, the server should be selected according to Table 8.

All the equations used in this section are considered innovations of this study. Also, mathematical principles are applied to estimate the normal cone to select the most optimal server, and new equations are developed to perform cold and hot migrations.

To select the server with the least traffic, a normal cone-based estimation is used. Therefore, the mentioned variables can be considered as distribution parameters as follows.

$$F(Sr)\propto\frac{1}{(w_{cs}\,C_s+w_{Tr}\,Tr_s^{neighbor}+w_{Li}\,Li_s^{neighbor}+w_{Bt}Bt_s+w_{cne}\,C_s^{neighbor}+w_{Ds}\overline{D_s})} \tag{39}$$

Due to the inverse effect of all the mentioned variables, the coefficients of all variables in the power of the normal cone estimation function are defined as negative. Therefore, this equation is expressed as follows:

$$F(Sr)\propto exp(-w_{cs}C_s-w_{Tr}Tr_s^{neighbor}-w_{Li}Li_s^{neighbor}-w_{Bt}Bt_s-w_{cne}C_s^{neighbor}-w_{Ds}\overline{D_s}) \tag{40}$$

To convert Eq. (40) into an equivalence equation, the importance coefficient of each case of the normal cone estimation is added to the equation in the form of the normalized difference value between the server *sr* and the current server. Thus, Eq. (41) is expressed as follows.

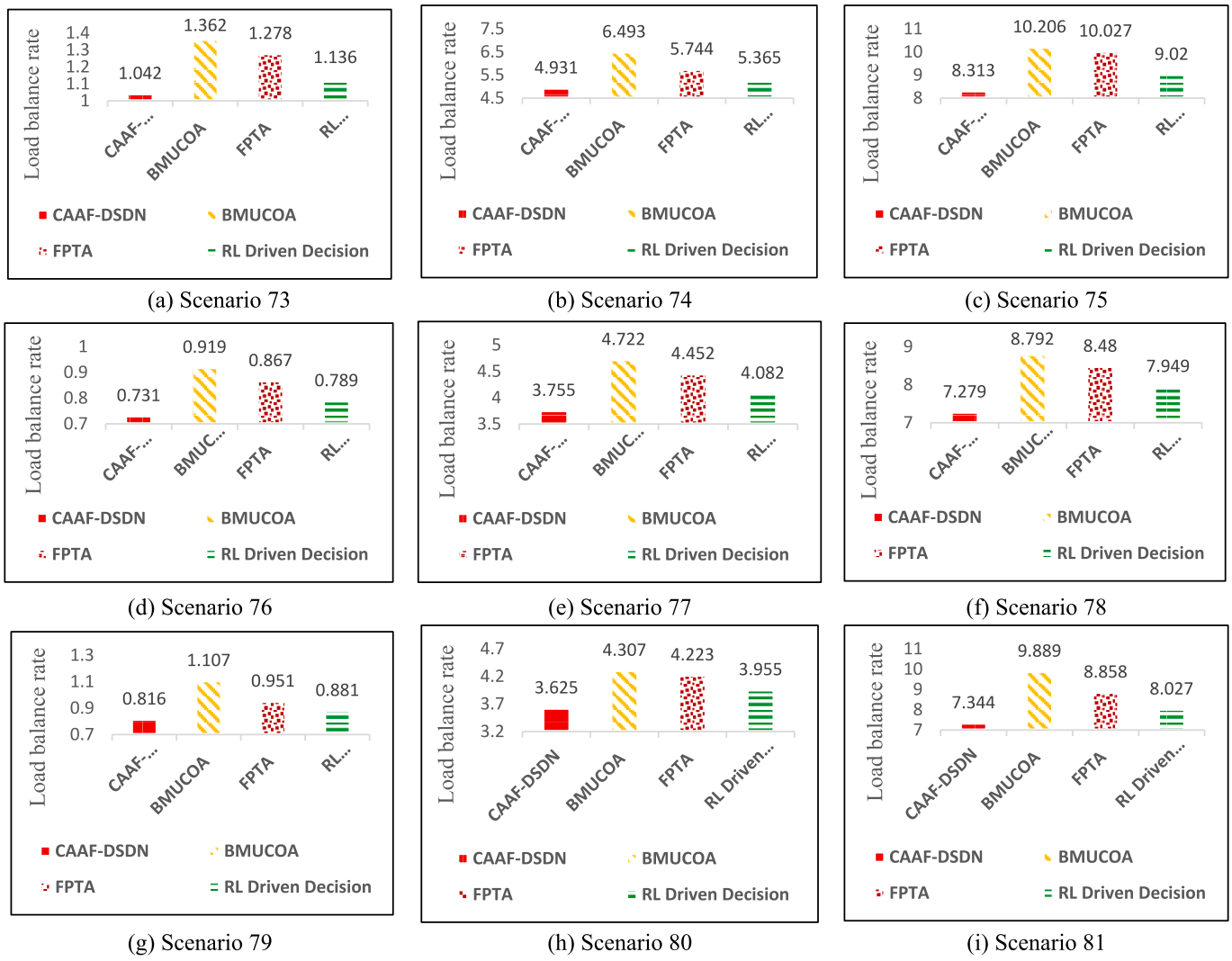

Fig. 27. Load balance rate for 100 controllers and 1000 VM in (a) Scenario 73, (b) Scenario 74 (c) Scenario 75, (d) Scenario 76, (e) Scenario 77, (f) Scenario 78, (g) Scenario 79, (h) Scenario 80 and (i) Scenario 81.

$$F(Sr) = exp(\delta_{sr}(-w_{cs}C_s - w_{Tr}Tr_s^{neighbor} - w_{Li}Li_s^{neighbor} - w_{Bt}Bt_s - w_{cne}C_s^{neighbor} - w_{Ds}\overline{D_s})) \tag{41}$$

The weights of the variables are selected based on the problem conditions. Also, the value of δ_{sr} is calculated according to Eq. (42).

$$\delta_{sr} = \theta(\sqrt{\sum_{i=1}^{N_{feature}} (Fe_{curent}^i - Fe_{sr}^i)^2}) \tag{42}$$

- where, $N_{feature}$ indicates the number of effective features of each server.
- Also, Fe_{curent}^i is the i^{th} feature of the current server.
- Fe_{sr}^i represents the i^{th} feature of the Sr server.
- In addition, θ represents the max_min normalization equation and maps all distances between the existing servers to the range 0 to 1.
- Finally, the server with the lowest $F(S_r)$ value in the current cluster is selected as the most optimal server.

The next challenge in this process is the type of migration to the selected server. The migration should cause the lowest possible latency.

- If the latency caused by the task migration is less than the overall VM latency, hot migration should be performed.
- Otherwise, cold migration is a more appropriate choice. If Eq. (43) is the cost of cold migration, this type of migration is more appropriate.

$$Cost_{sr}^C = C_{vm}\frac{Cost_{unit}L_{curenttoSr}}{min(BW_{curenttoSr})} \tag{43}$$

- where $\text{Cost}_{\text{unit}}$ is the cost of transferring a task over a link with a bandwidth of 1 byte per meter.
- Also, $\text{L}_{\text{curent to Sr}}$ is the length of the link between the current server and the server Sr.
- min $(\text{BW}_{\text{curent to Sr}})$ calculates the minimum bandwidth between the current server and the server Sr. Eq. (44) is used to calculate the cost of hot migration.

$$Cost_{sr}^H = C_{task}\frac{Cost_{unit}L_{curenttoSr}}{min(BW_{curenttoSr})} + C_{task}Cost_{packing} \tag{44}$$

- where, C_{task} is the amount of capacity occupied by the migrating task in the VM.
- Also, $\text{Cost}_{\text{packing}}$ represents the cost of preparing to send a unit of tasks to the current server.

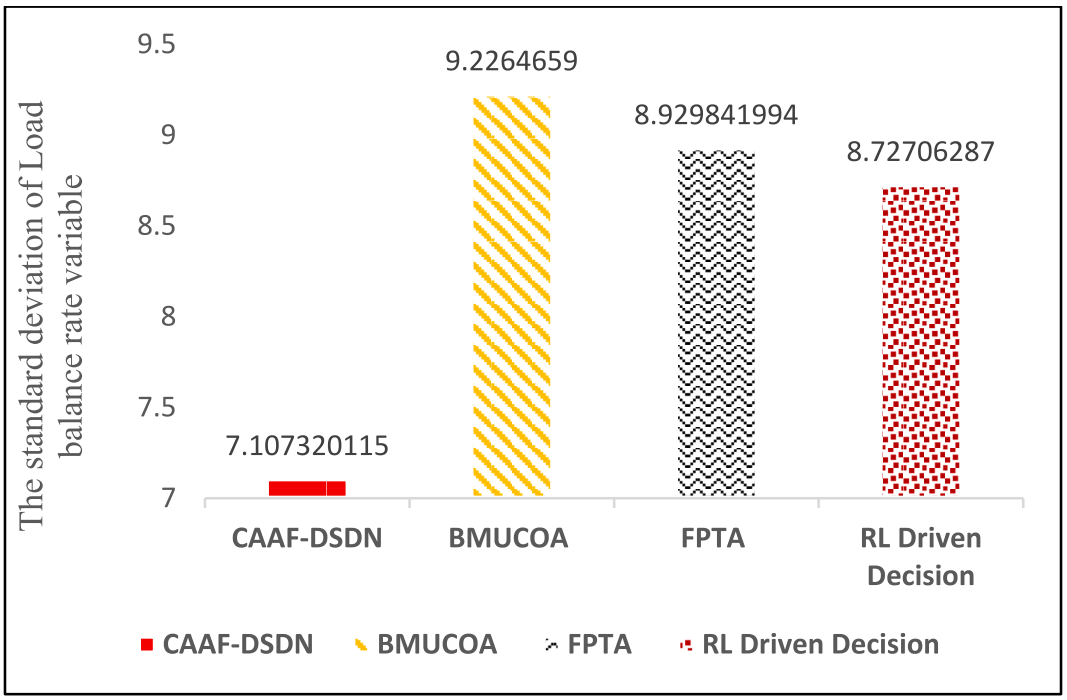

Fig. 28. Standard Deviation of the Load Balance Rate Variable.

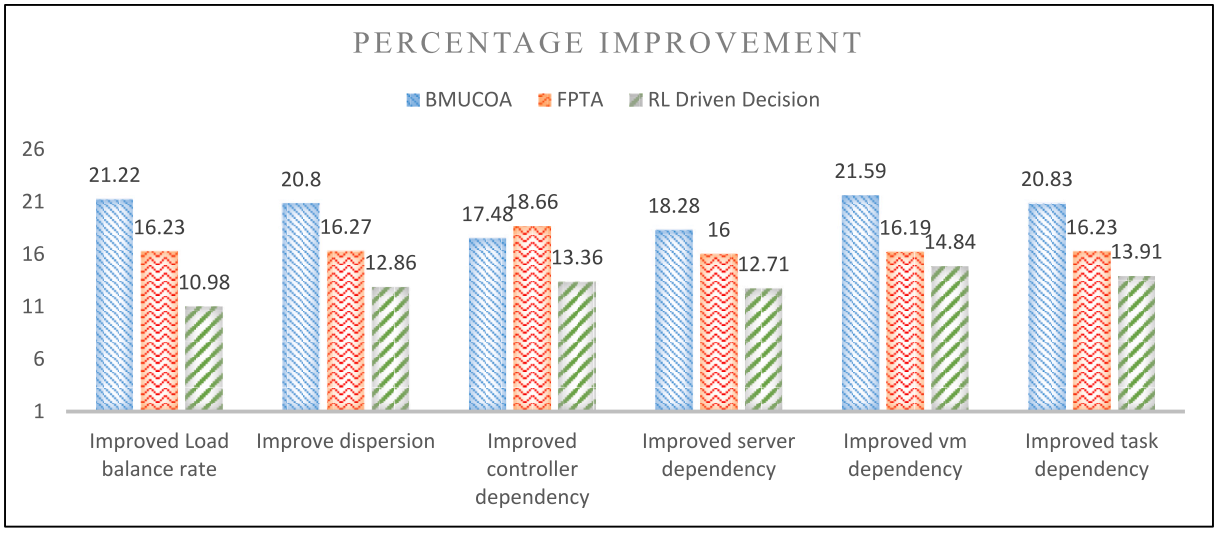

Fig. 29. Comparison of Improvements in the Proposed Method CAAF-DSDN over BMUCOA, FPTA, and RL Driven Decision in Terms of Load Balance Rate.

In light of the above, if $Cost_{sr}^{C} < Cost_{sr}^{H}$, then the process performed is a cold migration, otherwise, a hot migration occurs. The proposed method is fully represented in Fig. 2. The question is whether re-clustering upon the arrival of a new task is cost-effective.

Should the conditions for initiating re-clustering be more stringent? It should be said that this process is not performed for each task separately. Only a group of incoming tasks require re-clustering. This can be considered as a metaphor for the changing state of the environment.

5. Simulation results and discussion

In this section, to evaluate the efficiency of the CAAF-DSDN structure and achieve the load balancing objectives in Fog DSDN networks, this method is compared with three methods: BMUCOA (Sharma & Seth, 2024), FPTA (Huandong Wang et al., 2017), and RL Driven Decision (Alqam et al., 2023). After implementing the aforementioned methods on the default Fog DSDN network and analyzing the data obtained from each method, the performance of CAAF-DSDN regarding various aspects such as response latency to processing tasks, optimality of load distribution in the Fog DSDN network, and network latency is investigated.

5.1. Simulation platform and settings

The network structure under study in the simulated scenarios is presented in Table 9.

5.2. Simulation parameters

In this section, the effect of four independent variables, namely the number of controllers, the number of servers, the number of VMs, and

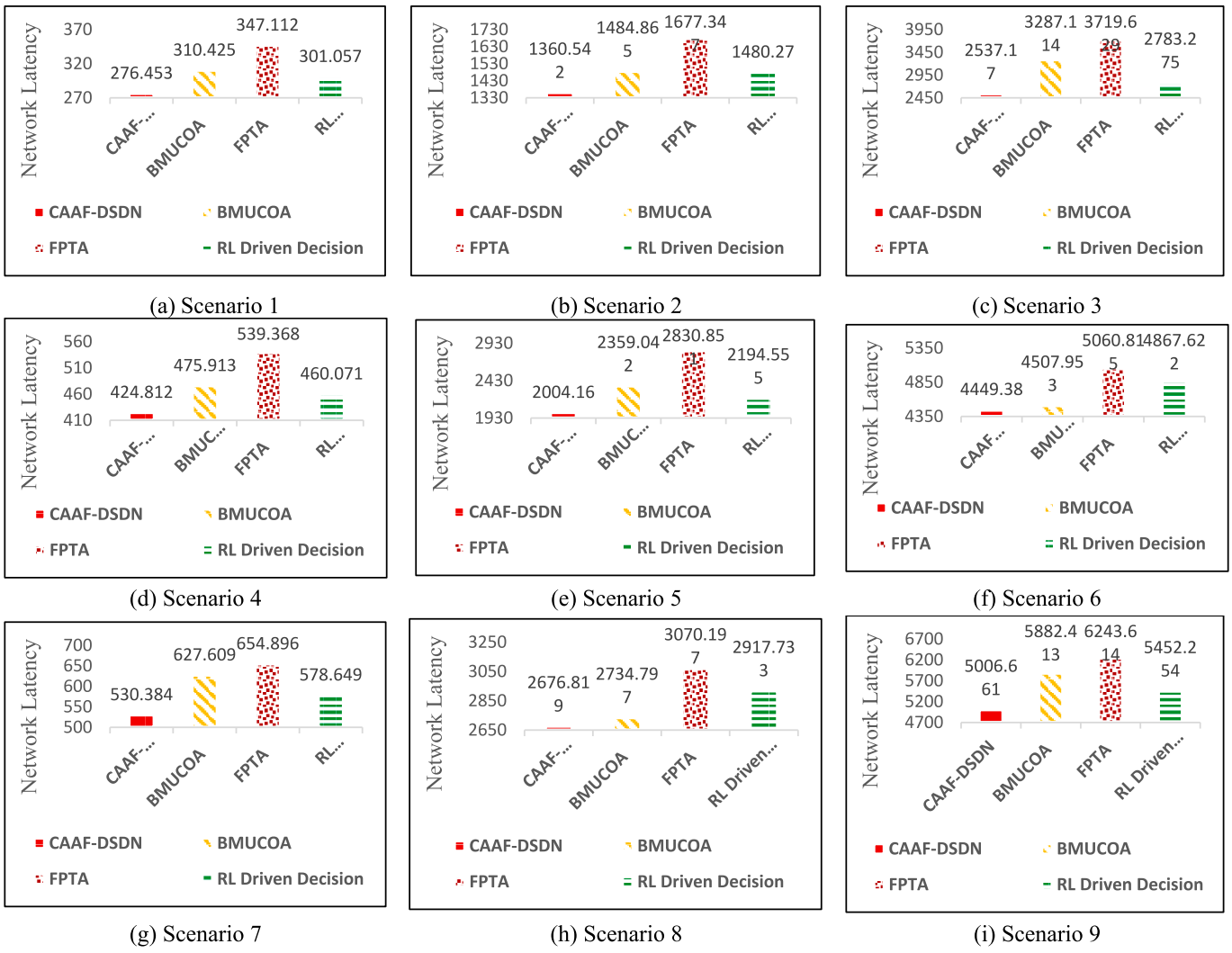

Fig. 30. Network Latency for 5 controllers and 300 VM in (a)Scenario 1, (b)Scenario 2, (c)Scenario 3, (d)Scenario 4, (e)Scenario 5, (f)Scenario 6, (g)Scenario 7, (h) Scenario 8 and (i)Scenario 9.

the number of processing tasks on three dependent variables, including network latency, load balancing rate, and response time, is examined in 81 simulated scenarios. The details of each scenario are presented in Table 10. The proposed method is designed in the form of a three-layer distributed Fog DSDN architecture, including three types of nodes, namely controllers, SW nodes, and VM. Table 11 shows the type and characteristics of the nodes used in the simulation. Each scenario is simulated 10 times and the average results are presented in the form of plots. The following criteria are evaluated to analyze and compare the proposed method with similar methods.

- Network latency: Network latency refers to the time taken to transfer data from one point to another in the network. This time is measured in milliseconds and is affected by various factors such as distance, network congestion, and connection quality. High latency can lead to reduced response time and poor performance in network-dependent applications such as online gaming or video streaming.
- Load balance rate: Load balancing refers to the process of distributing traffic across multiple servers to improve the performance and reliability of a service or application. Load balance rate refers to the proportion of demand or workload that is distributed evenly across a distributed system or network, such as a cluster of servers. This distribution prevents excessive component damage and overload and allows for optimal performance. Load balancing can be achieved through various techniques such as round-robin scheduling, weighted distribution, or dynamic load balancing based on real-time measurements. The load balance rate measures the effective distribution of workload among available resources.
- Response Time: The time elapsed from the entry of a task into the queue until the result is received at the requesting node is called response time, which is measured in seconds.

5.3. The response time

Figs. 8 to 16 depict the average improvements in response time as the independent variables increase across Scenarios 1 to 81. The results indicate that the CAAF-DSDN algorithm consistently outperforms the BMUCOA (Sharma & Seth, 2024), FPTA (Huandong Wang et al., 2017), and RL Driven Decision (Alqam et al., 2023) methods. Specifically, CAAF-DSDN achieves reductions in average response time of 10.96 %, 17.34 %, and 13.28 %, respectively.

Additionally, Fig. 17 illustrates the standard deviation of the response time for CAAF-DSDN compared to the other three methods. These results suggest that CAAF-DSDN offers greater stability, with a lower variability in response time—11.6 %, 17.6 %, and 15.86 % less than BMUCOA (Sharma & Seth, 2024), FPTA (Huandong Wang et al.,

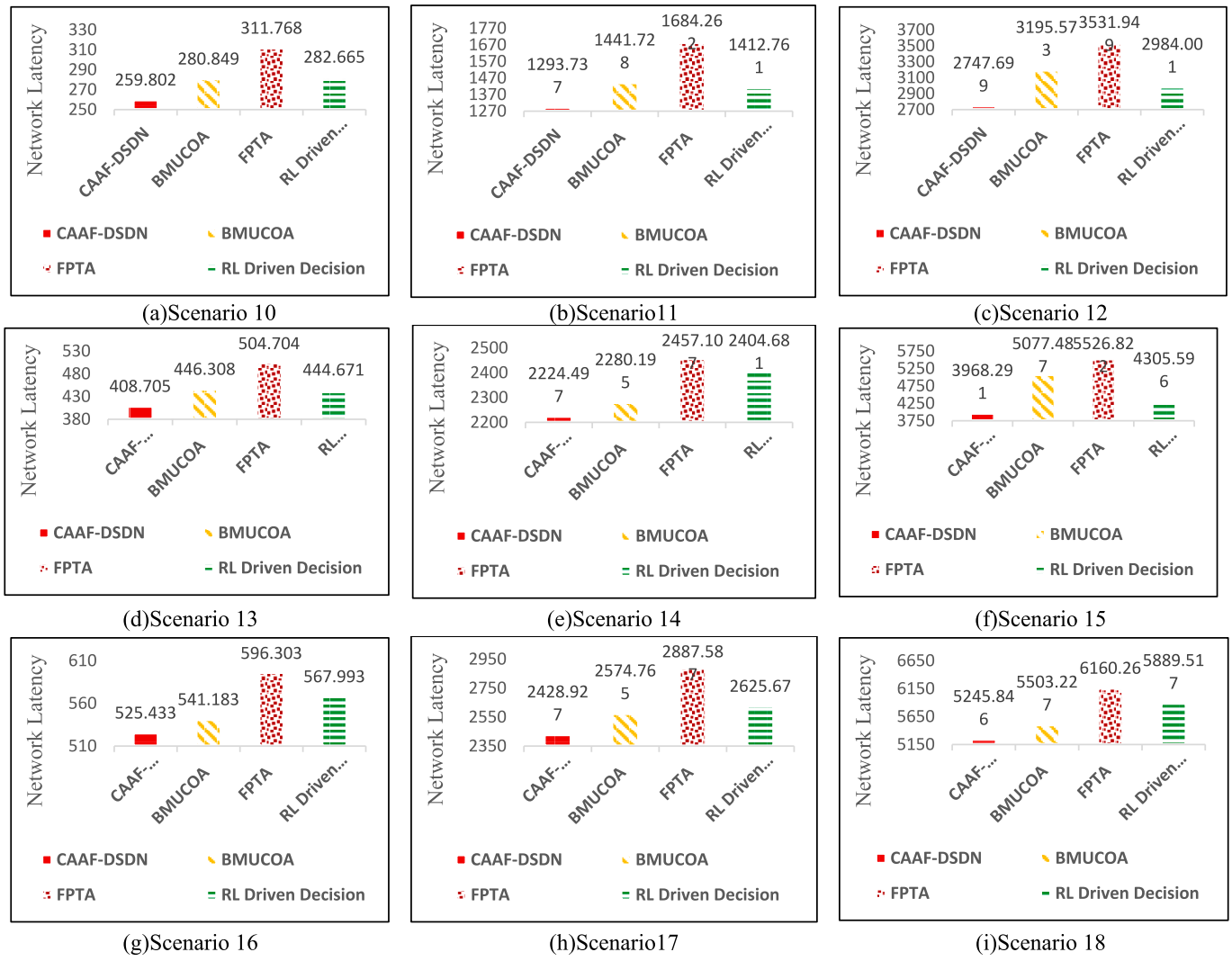

Fig. 31. Network Latency for 5 controllers and 500 VM in (a)Scenario 10, (b)Scenario 11, (c)Scenario 12, (d)Scenario 13, (e)Scenario 14, (f)Scenario 15, (g)Scenario 16, (h)Scenario 17 and (i)Scenario 18.

2017), and RL Driven Decision (Alqam et al., 2023), respectively. This stability is crucial in dynamic environments where maintaining consistent performance is a key factor.

The impact of increasing the number of controllers on response time was also analyzed. In all methods, the number of controllers directly influences response time, but the effect is less pronounced in CAAF-DSDN. Specifically, the reduction in response time is 14.31 %, 19.69 %, and 15.25 % less in CAAF-DSDN compared to BMUCOA (Sharma & Seth, 2024), FPTA (Huandong Wang et al., 2017), and RL Driven Decision (Alqam et al., 2023), respectively. This suggests that CAAF-DSDN's improvements are driven more by the algorithm's inherent efficiency than by changes in network structure or the number of controllers.

Fig. 18 further elaborates on the variables influencing response time. The findings reveal that CAAF-DSDN demonstrates a lower dependency on network structure compared to the other methods. The significant improvements in response time observed in CAAF-DSDN are attributed to three main factors:

Algorithm Optimization: CAAF-DSDN optimizes the handling of input data and variables, leading to more efficient processing and reduced response times. By enhancing data processing and path selection, the algorithm improves overall system efficiency.

Reduction in Computational Complexity: By minimizing computational complexity, the algorithm reduces the time required for processing information. Techniques such as clustering, data compression, and artificial intelligence-based optimizations contribute to this reduction, enabling faster decision-making and task execution.

Efficient Resource Management: CAAF-DSDN makes optimal use of network resources, including controllers, bandwidth, and traffic management. By efficiently allocating resources and minimizing unnecessary network traffic, the algorithm enhances performance and reduces response times.

In conclusion, the results across the figures clearly demonstrate the superior performance of CAAF-DSDN, not only in terms of reducing response time but also in maintaining stability across different network conditions. The combination of algorithmic optimization, reduced computational overhead, and effective resource management allows CAAF-DSDN to outperform existing methods, making it a robust solution for workload distribution and virtual machine migration in large-scale network environments.

Furthermore, the findings underscore the potential of CAAF-DSDN in addressing the challenges posed by dynamic and large-scale network environments. While other methods demonstrate improvements in response time, CAAF-DSDN's ability to achieve these reductions with lower dependency on the network structure and number of controllers highlights its adaptability.

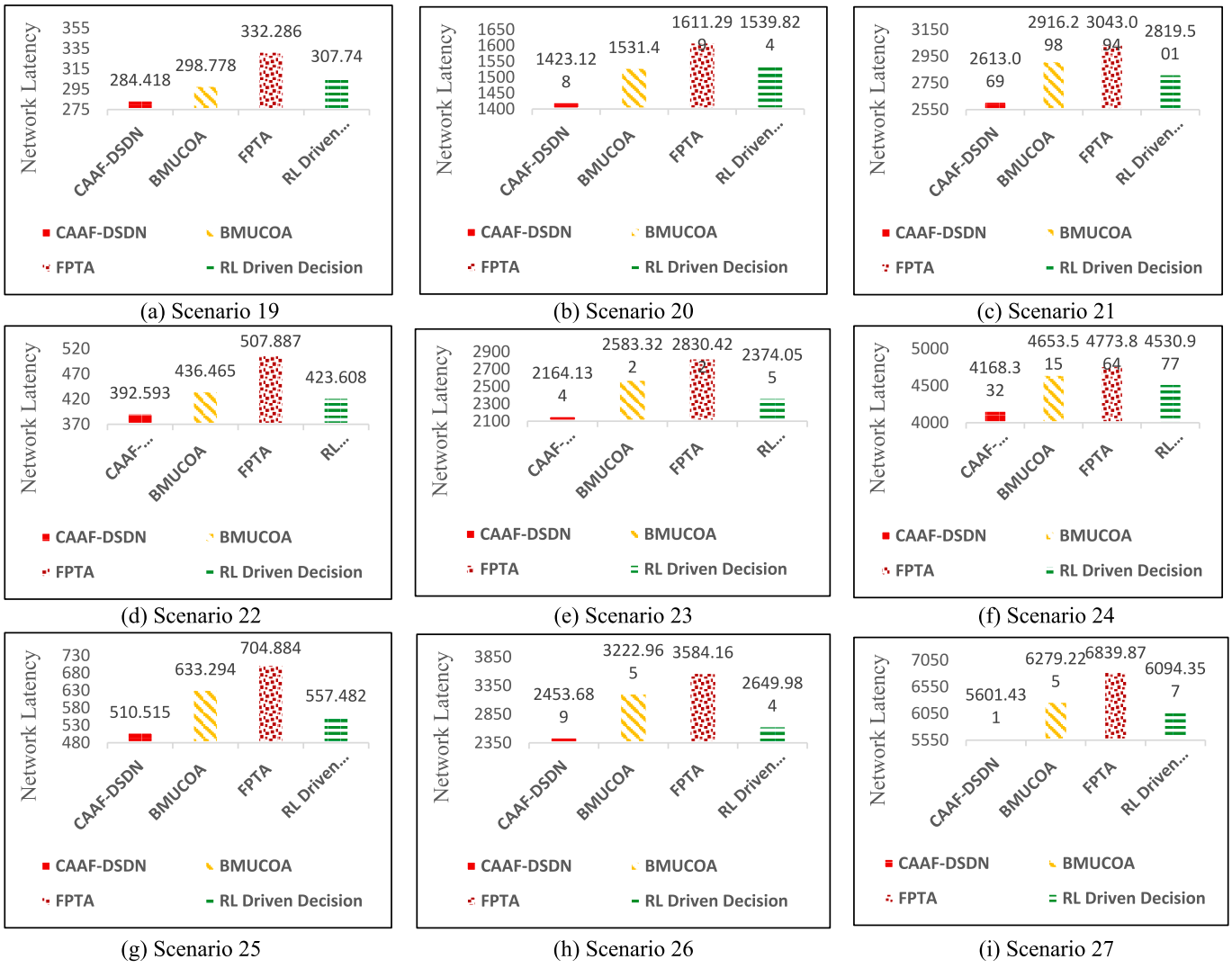

Fig. 32. Network Latency for 5 controllers and 1000 VM in (a) Scenario 19, (b) Scenario 20, (c) Scenario 21, (d) Scenario 22, (e) Scenario 23, (f) Scenario 24, (g) Scenario 25, (h) Scenario 26 and (i) Scenario 27.

This characteristic is particularly advantageous in environments where network conditions are subject to constant fluctuation. Additionally, the algorithm's efficiency in minimizing resource consumption while maintaining stable performance demonstrates its scalability for large-scale deployments. The ability to effectively balance computational complexity, network traffic, and resource allocation ensures that CAAF-DSDN remains an optimal solution even under high-demand conditions, thereby offering significant potential for real-time applications that require both flexibility and reliability in performance.

5.3.1. Performance improvement during migration

The performance of CAAF-DSDN during migration is improved by selecting a server with the highest compatibility with the VM source server. The CAAF-DSDN method can reduce the average distance of samples within a cluster. This distance reduction leads to increased clustering accuracy and improves the overall performance of the method. Another reason for improved performance in migration is the ability to choose between hot and cold migration. In critical network conditions, choosing the least expensive method of transferring data to low-traffic areas improves migration performance. Also, the information obtained from the clustering process and traffic distribution helps the network to make smarter decisions in load management. This includes rerouting requests and optimally allocating resources to services, which leads to reduced response time.

5.3.1.1. Flexibility and optimization: Little's law and reduced time complexity in improving CAAF-DSDN performance.

- Due to its high flexibility, CAAF-DSDN as an unsupervised method can respond more effectively to changes in data. Such flexibility allows the model to perform better in the face of new data or changing conditions. This capability is very crucial in dynamic and changing environments. As a result, CAAF-DSDN is less dependent on the environment than other methods. On the other hand, using Little's law as an efficient method for queue management provides the conditions for calculating a equation between latency and processing load in a VM. This law clearly explains the equationship between the average latency and the average processing load and helps in selecting VMs with the lowest latency.

- Reduction of response time by reducing time complexity: According to the time complexity analysis, CAAF-DSDN has a time complexity of O(n2) and is more efficient compared to BMUCOA, FPTA, and RL Driven Decision. The lower time complexity means that CAAF-DSDN processes tasks faster and provides better response time.

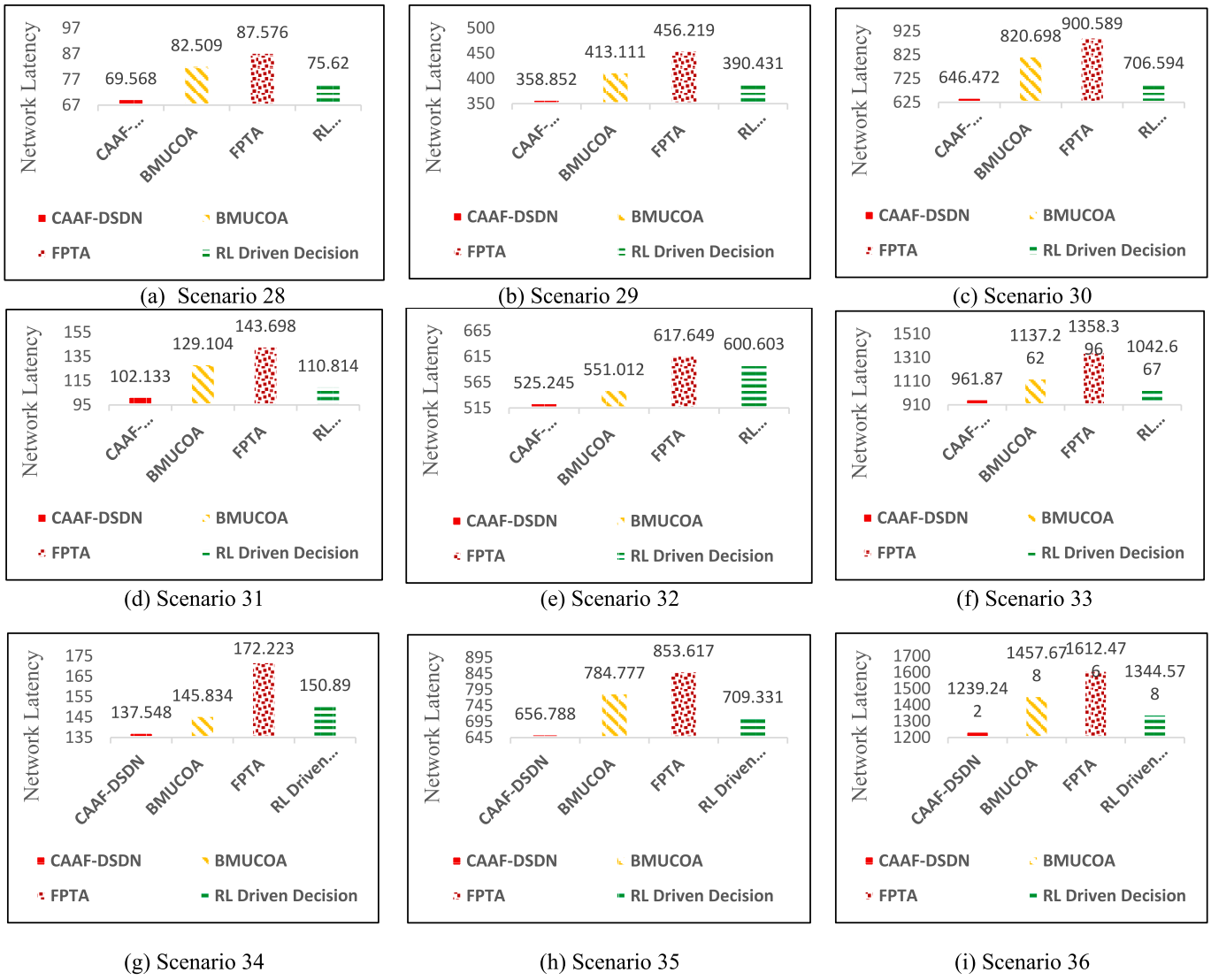

Fig. 33. Network Latency for 20 controllers and 300 VM in (a) Scenario 28, (b) Scenario 29, (c) Scenario 30, (d) Scenario 31, (e) Scenario 32, (f) Scenario 33, (g) Scenario 34, (h) Scenario 35 and (i) Scenario 36.

5.4. The load balancing rate

According to Figs. 19 to 27, the CAAF-DSDN algorithm exhibits a significant improvement in the average load balancing rate when compared to the BMUCOA (Sharma & Seth, 2024), FPTA (Huandong Wang et al., 2017), and RL-Driven Decision (Alqam et al., 2023) methods. Specifically, the load balancing rate for CAAF-DSDN has improved by 21.22 %, 16.23 %, and 10.98 % compared to these methods, respectively. This substantial improvement underscores CAAF-DSDN's ability to more efficiently distribute workloads across virtual machines and servers, ensuring optimized resource utilization in dynamic network environments.

Moreover, Fig. 28 presents the standard deviation of the load balancing rate, which provides additional insight into the consistency of performance. Based on the results, it is evident that CAAF-DSDN demonstrates superior stability, with a reduction in variability of 20.8 %, 16.27 %, and 12.86 % compared to BMUCOA (Sharma & Seth, 2024), FPTA (Huandong Wang et al., 2017), and RL-Driven Decision (Alqam et al., 2023), respectively.

This indicates that CAAF-DSDN not only improves load balancing but also maintains a more consistent level of performance across varying conditions. Additionally, the impact of increasing the number of controllers on the load balancing rate has been explored. As expected, increasing the number of controllers enhances the load balancing rate in all methods, yet this improvement is less pronounced in CAAF-DSDN. Specifically, CAAF-DSDN shows an improvement of 17.48 %, 18.66 %, and 13.36 % less than BMUCOA (Sharma & Seth, 2024), FPTA (Huandong Wang et al., 2017), and RL-Driven Decision (Alqam et al., 2023), respectively.

This suggests that CAAF-DSDN's load balancing mechanism is less sensitive to changes in the number of controllers, providing greater stability in more dynamic and large-scale environments. Furthermore, while the addition of controllers improves load balancing, the improvement in CAAF-DSDN is notably less volatile than in the other two methods, reflecting its robustness in maintaining performance even as the system scales.

Fig. 29 illustrates the variables considered in this analysis, reinforcing the findings that CAAF-DSDN leads to a clear improvement in load balancing efficiency. The increase in load balancing within CAAF-DSDN can be attributed to the effective optimization of resource distribution, where the algorithm efficiently allocates network resources across the infrastructure, ensuring that no single node is overloaded. This enhanced load balancing stability positions CAAF-DSDN as a promising solution for managing complex, high-demand network

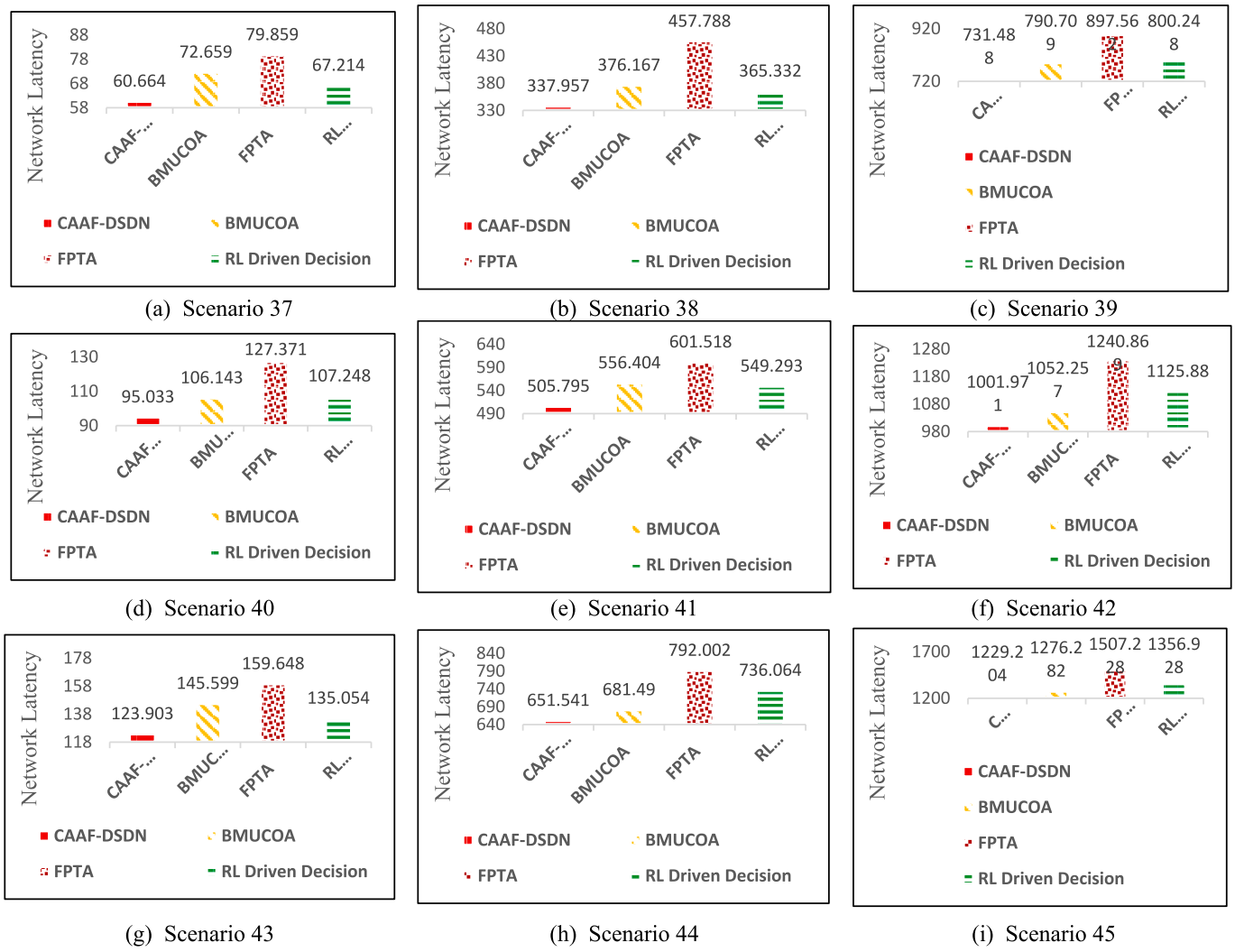

Fig. 34. Network Latency for 20 controllers and 500 VM in (a) Scenario 38, (b) Scenario 39, (c) Scenario 40, (d) Scenario 41, (e) Scenario 42, (f) Scenario 43, (g) Scenario 34, (h) Scenario 44 and (i) Scenario 45.

environments, where maintaining optimal performance and avoiding resource contention are critical for long-term scalability and reliability.

Due to the following two factors, CAAF-DSDN has managed to improve and stabilize the performance in load distribution.

- Clustering of similar servers: By clustering servers based on their capacity and common features, CAAF-DSDN optimizes resource allocation. Such clustering causes VMs to migrate to the servers most compatible with their requirements. As a result, the servers are loaded more evenly. By considering various variables in the clustering process, this method accurately analyzes the workload and allocates tasks while maintaining load balance across the network. Also, the dynamic selection of VMs based on the current load balance protects the network from critical situations.
- Reduce traffic on congested servers: According to simulation results, servers with higher processing loads generate more traffic in the surrounding network. By prioritizing low-traffic areas and providing an overview of the network load status, CAAF-DSDN creates a more even load distribution. This reduces the standard deviation of the ratio of the filled capacity to the total capacity of the servers and improves load balancing.

In general, by relying on density-based clustering and selecting VMs with the lowest L_i value, the processing load is distributed more evenly among the servers. Such an even load distribution helps to reduce the difference between the filled capacities of the servers. As a result, the standard deviation of the ratio of the filled processing capacity to the total processing capacity of the server is reduced. In addition, CAAF-DSDN tries to minimize the highest L_i value. This reduces the difference between L_i values across VMs, which in turn reduces the standard deviation and improves network load balancing. This increases overall system efficiency and distributes the load more evenly across servers.

5.5. The network latency

In this section, the proposed method is evaluated regarding network latency. This test tells us whether the network has succeeded in eliminating bottlenecks or latency factors in its operation or not. According to the results in Figs. 30 to 38, CAAF-DSDN has been able to reduce the total network latency by 11.3 %, 19.6 %, and 14.56 % compared to BMUCOA(Sharma & Seth, 2024), FPTA(Huandong Wang et al., 2017), and RL Driven Decision(Alqam et al., 2023), respectively. The standard deviation of the load balance rate in the three studied methods is represented in Fig. 39. These figures indicate the greater stability of CAAF-DSDN performance.

The stability of the proposed method is 11.22 %, 18.841 %, and 15.53 % higher compared to BMUCOA (Sharma & Seth, 2024), FPTA (Huandong Wang et al., 2017), and RL Driven Decision(Alqam et al.,

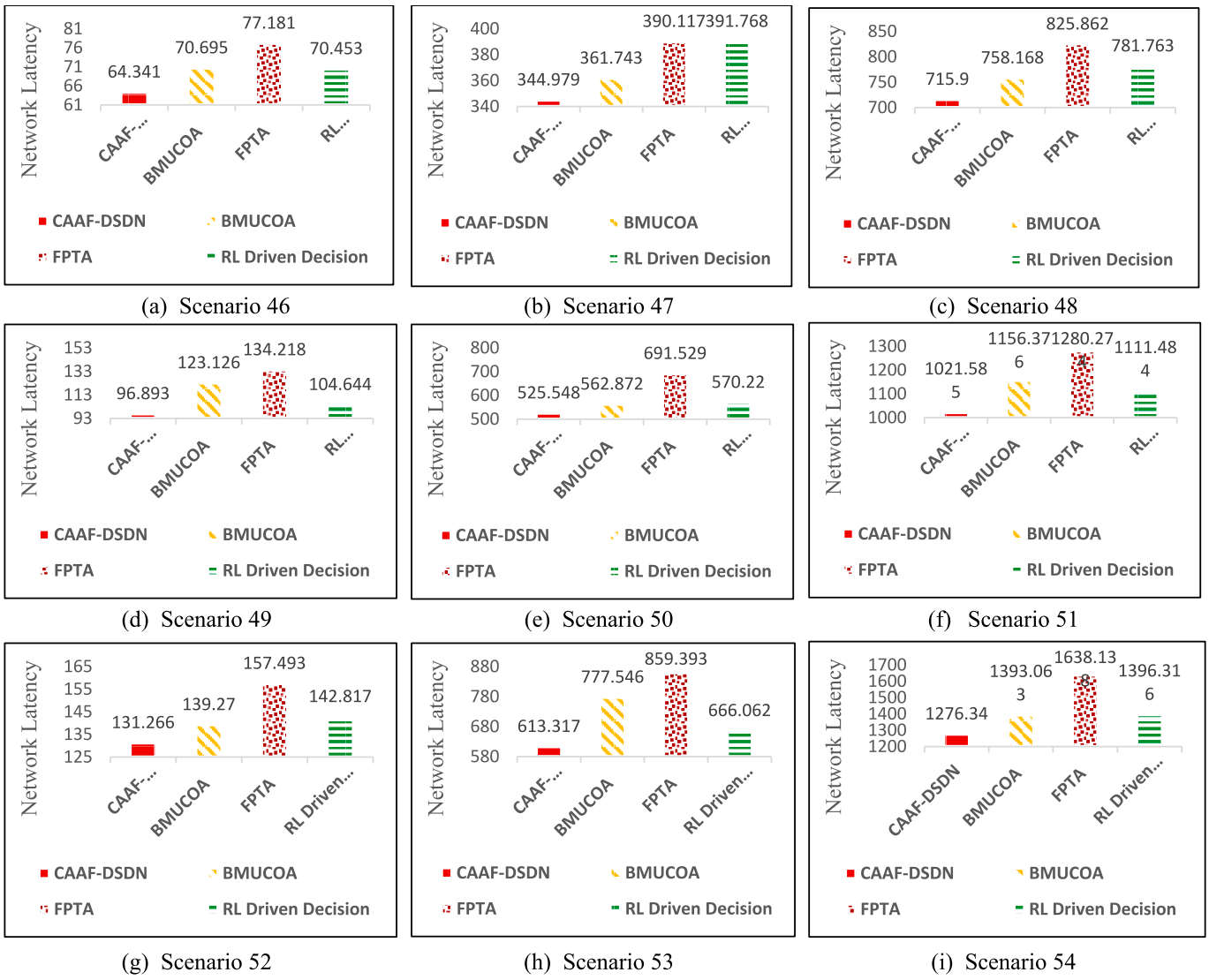

Fig. 35. Network Latency for 20 controllers and 1000 VM in (a) Scenario 46, (b) Scenario 47, (c) Scenario 48, (d) Scenario 49, (e) Scenario 50, (f) Scenario 51, (g) Scenario 52, (h) Scenario 53 and (i) Scenario 54.

2023), respectively. Considering the dependence of the total network latency on four independent variables including the number of controllers, the number of servers, the number of VMs, and the number of tasks in the plot of Fig. 40, CAAF-DSDN shows less dependence on network conditions compared to BMUCOA(Sharma & Seth, 2024), FPTA (Huandong Wang et al., 2017), and RL Driven Decision(Alqam et al., 2023). This confirms the improved performance and flexibility of the proposed method in dealing with network changes.

The CAAF-DSDN method offers several significant advantages over the methods under consideration, providing substantial improvements in network load distribution and latency management. These advantages are as follows:

1. Density-based Clustering: CAAF-DSDN utilizes density-based clustering to divide servers into clusters based on traffic density. By identifying high-density clusters, this method avoids selecting servers located in congested areas, thereby reducing the load on these servers and ultimately decreasing network latency. This approach ensures that high-traffic zones are efficiently managed, preventing network bottlenecks
2. Low-latency VM Selection: In assigning the processing load, CAAF-DSDN selects the lowest-latency virtual machine (VM) with available capacity. This strategy minimizes waiting time and latency in data processing, which enhances overall system performance and responsiveness. It effectively supports faster task allocation and execution, ensuring that the system remains efficient under varying load conditions
3. VM Migration: When server overload occurs, the system facilitates VM migration to other servers. This migration can be either hot or cold. Hot migration transfers processing tasks without shutting down the VM, which minimizes the latency typically associated with migration processes. As a result, this feature improves the user experience by ensuring minimal disruption in service
4. Dynamic resource allocation based on network Conditions: CAAF-DSDN dynamically allocates processing resources according to current network conditions, optimizing performance in real-time. This adaptive resource management helps reduce latency in both data transmission and processing, ensuring that network efficiency is maximized. This dynamic allocation of resources contributes to overall improvements in service delivery
5. Reduction of response Time: Adaptive clustering is employed to select servers in low-traffic areas during peak periods. In normal operating conditions, traffic distribution is managed based on latency and load balancing, which reduces response time for customer

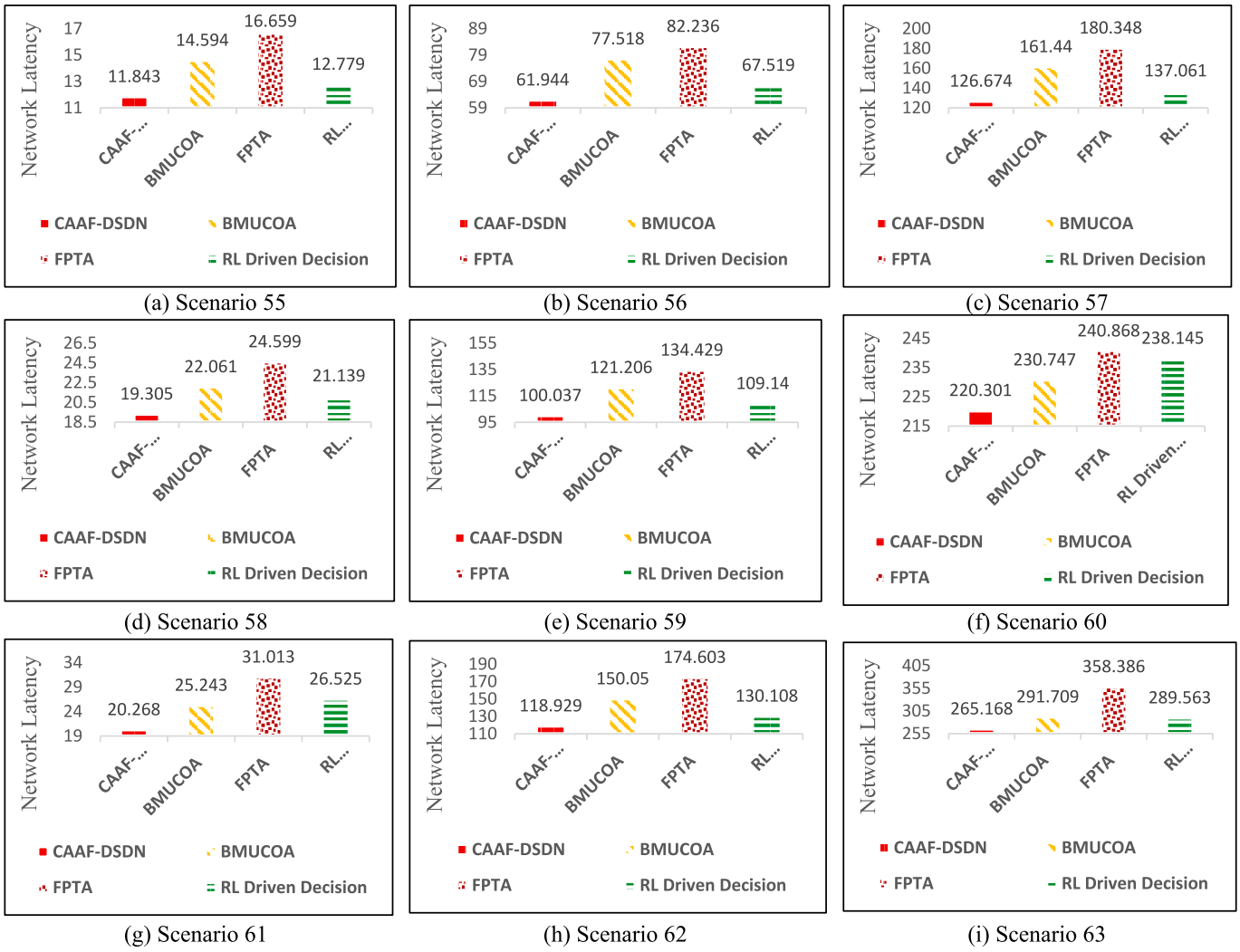

Fig. 36. Network Latency for 100 controllers and 300 VM in (a) Scenario 55, (b) Scenario 56, (c) Scenario 57, (d) Scenario 58, (e) Scenario 59, (f) Scenario 60, (g) Scenario 61, (h) Scenario 62 and (i) Scenario 63.

requests. This approach improves the load balancing index within the network and decreases overall network latency

Overall, these advantages collectively ensure that CAAF-DSDN outperforms other methods in terms of network load management and latency reduction. By leveraging more efficient clustering and resource allocation strategies, the method provides significant optimization in these areas.

Additionally, the results obtained from the simulations and the improvements observed in the three studied variables offer additional insights. By utilizing a more accurate similarity matrix, the clustering algorithm better considers the essential criteria for cluster formation, leading to more precise clustering. This reduces data ambiguity and allows for clearer identification of relevant patterns. Factorization of the similarity matrix helps to identify significant and influential features within the data, leading to improved clustering and a more effective reduction of data ambiguity. Detecting noisy data and large distances in the matrix can prevent improper clustering and improve data quality.

The overall clustering structure in CAAF-DSDN ensures that adjacent data points are appropriately selected, preventing the formation of unreasonable clusters. This improves the accuracy and quality of clustering, which is crucial for handling noisy data and outliers. The method's robustness in this regard significantly enhances the quality of the data and ensures that accurate modeling is achieved. In general, considering the results across the three variables, the following key conclusions can be drawn:

- Superior Performance: CAAF-DSDN outperforms the other methods in terms of the three measured variables, offering a clear advantage in network load management and latency optimization.
- Stability of Performance: The proposed method demonstrates greater stability compared to BMUCOA (Sharma & Seth, 2024), FPTA (Huandong Wang et al., 2017), and RL-Driven Decision (Alqam et al., 2023), ensuring consistent performance across different network conditions.
- Reduced Impact of Network Conditions: CAAF-DSDN shows a lesser dependency on varying network conditions, making it more adaptable to fluctuating network environments compared to the other methods.
- Network Latency Reduction: The method's ability to efficiently distribute traffic to low-traffic areas is a key factor in reducing network latency, contributing to a smoother traffic flow across the entire network.
- Reduction of Data Ambiguity: The CAAF-DSDN method effectively addresses data ambiguity, enhancing data clarity and enabling better decision-making and knowledge discovery.

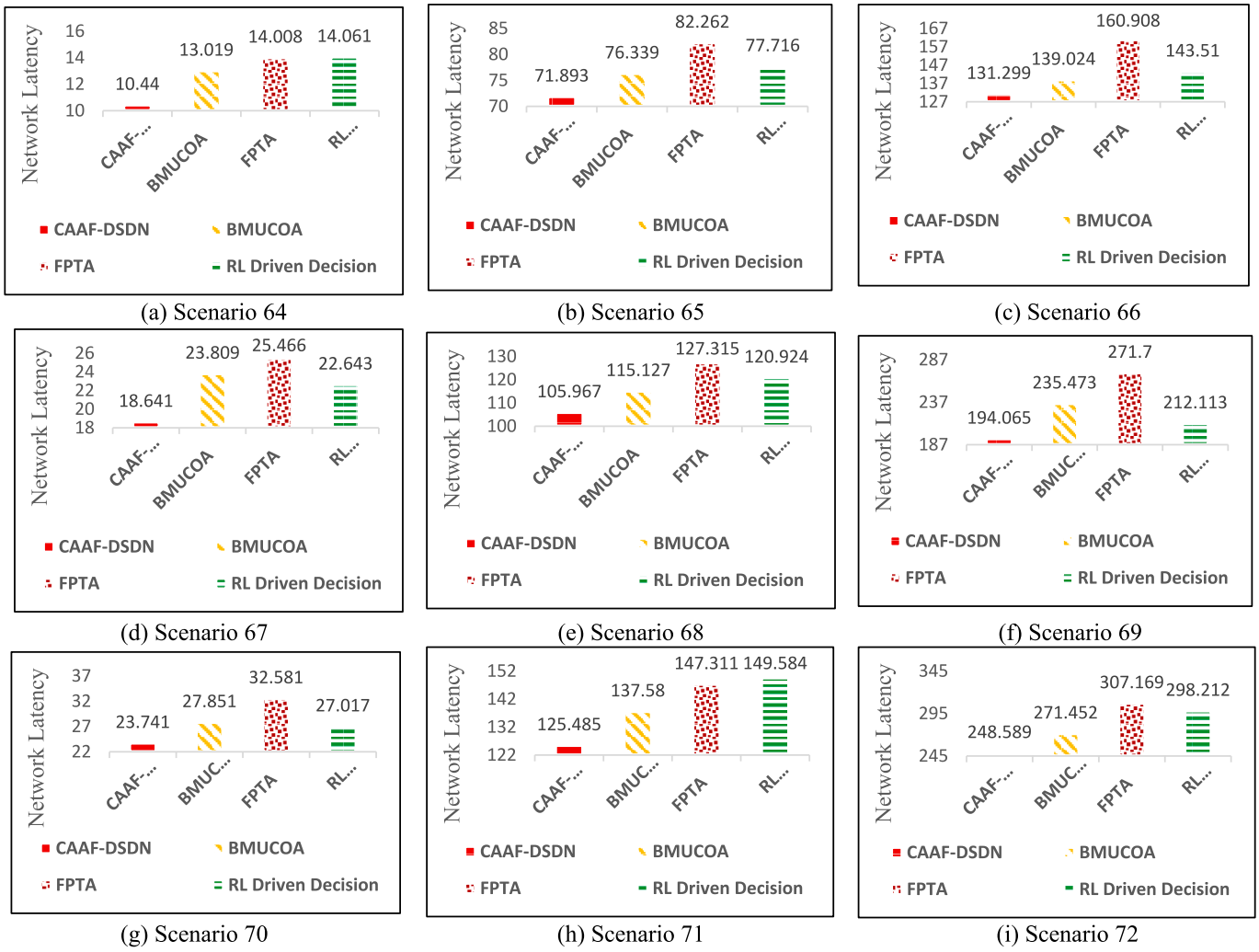

Fig. 37. Network Latency for 100 controllers and 500 VM in (a) Scenario 64, (b) Scenario 65, (c) Scenario 66, (d) Scenario 67, (e) Scenario 68, (f) Scenario 69, (g) Scenario 70, (h) Scenario 71 and (i) Scenario 72.

- Optimal Server Selection: The ability to select the most appropriate server based on workload and traffic conditions ensures that resources are optimally utilized, enhancing overall system efficiency.

In summary, CAAF-DSDN not only optimizes network performance but also enhances data quality, providing valuable improvements in both network management and data analysis. This makes it a highly efficient and effective solution for large-scale, dynamic network environments, where both performance and data integrity are crucial.

6. Conclusion and future work

Distributed software-defined networks (DSDN), as one of the new technologies, play a vital role in optimizing resources, increasing productivity, and reducing overall network costs. One of the major challenges in this field is the optimal balance between resources, network efficiency, and operational capacity. In this study, the CAAF-DSDN method is proposed for load balancing in Fog-DSDN networks. Using adaptive algebra and the normal cone estimation function, this method, as an innovative and efficient method, optimizes resource allocation and traffic management in complex processing networks. The results show that applying this method facilitates more accurate and intelligent control of resource status, network traffic, and service requirements. This in turn leads to significant improvements in network load balancing, latency, and response time. Specifically, the simulations show a 15.13 % reduction in network latency, an 18.73 % improvement in load balancing, and a 14.50 % reduction in response time. Overall, the results confirm the effectiveness of the CAAF-DSDN method in optimizing the performance and resilience of Fog-DSDN networks. The proposed method can be used as the basis for resource allocation and management strategies in distributed software-defined networks. Although this study presents significant contributions, several limitations and challenges remain, which could be addressed in future research. One of the main limitations of this study is the constrained scale of the simulations and experimental environments. Specifically, the experiments were conducted under specific conditions and with predefined models, and it is necessary to extend this approach to a wider range of scenarios and more complex conditions. Additionally, implementing the algorithm at a real-world scale, particularly under the operational conditions of large-scale networks, presents challenges such as high latency, resource constraints, and increased complexity. These factors will require further optimization and refinement in future work. In the pursuit of further enhancing the Fog-DSDN architecture and its clustering algorithms, several key areas of future research can be identified. These areas focus on optimizing performance, improving adaptability, and expanding the potential applications of the method. Key research directions include: (i) Federated Learning Integration for Decentralized Clustering**:** The integration of federated learning within the Fog-DSDN framework represents a promising direction for future research. Federated learning allows for decentralized training of

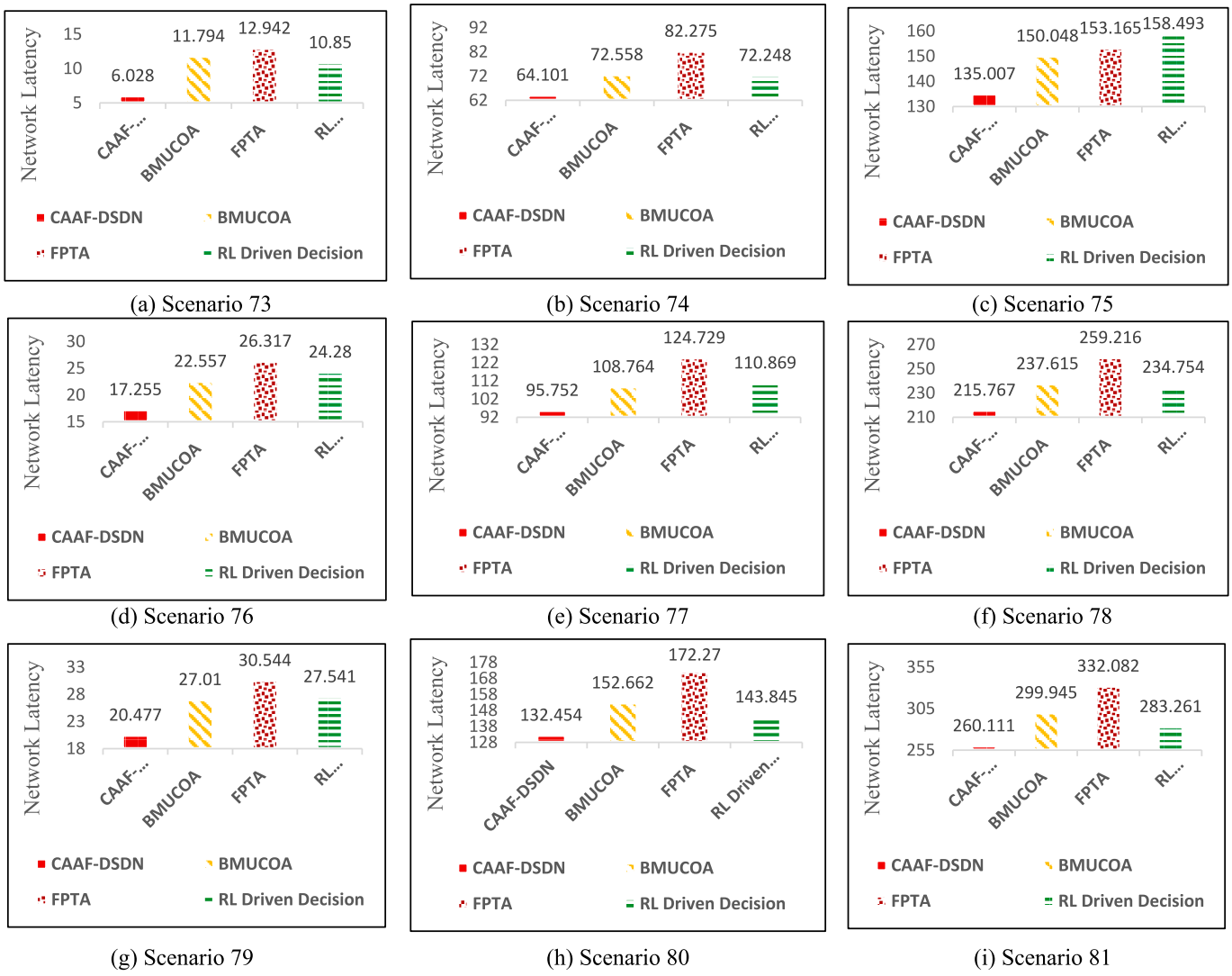

Fig. 38. Network Latency for 100 controllers and 1000 VM in (a) Scenario 73, (b) Scenario 74 (c) Scenario 75, (d) Scenario 76, (e) Scenario 77, (f) Scenario 78, (g) Scenario 79, (h) Scenario 80 and (i) Scenario 81.

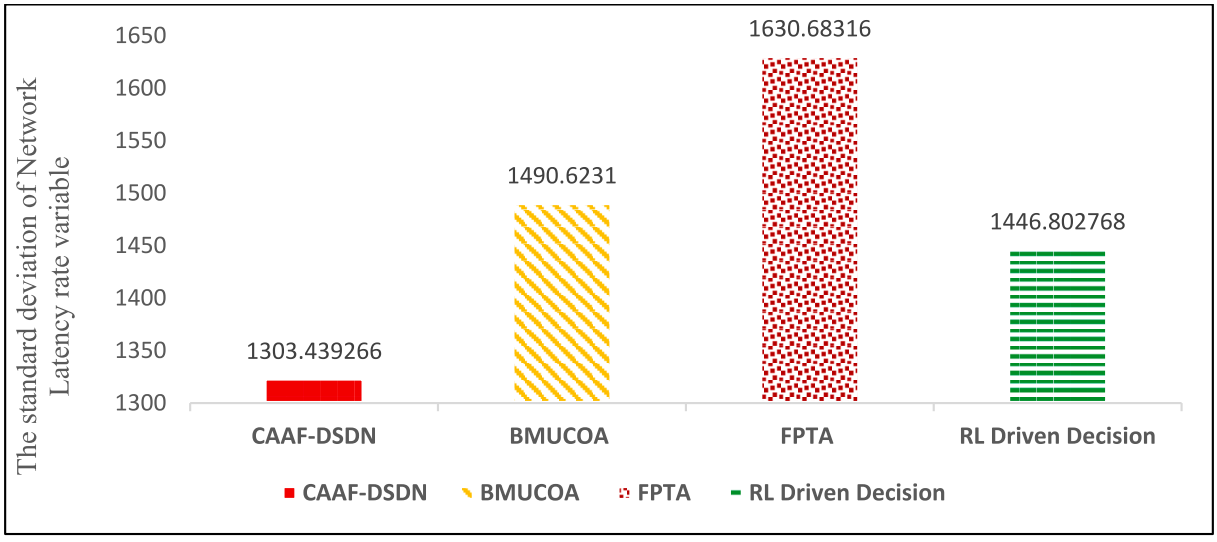

Fig. 39. Standard Deviation of the Load Balance Rate Variable.

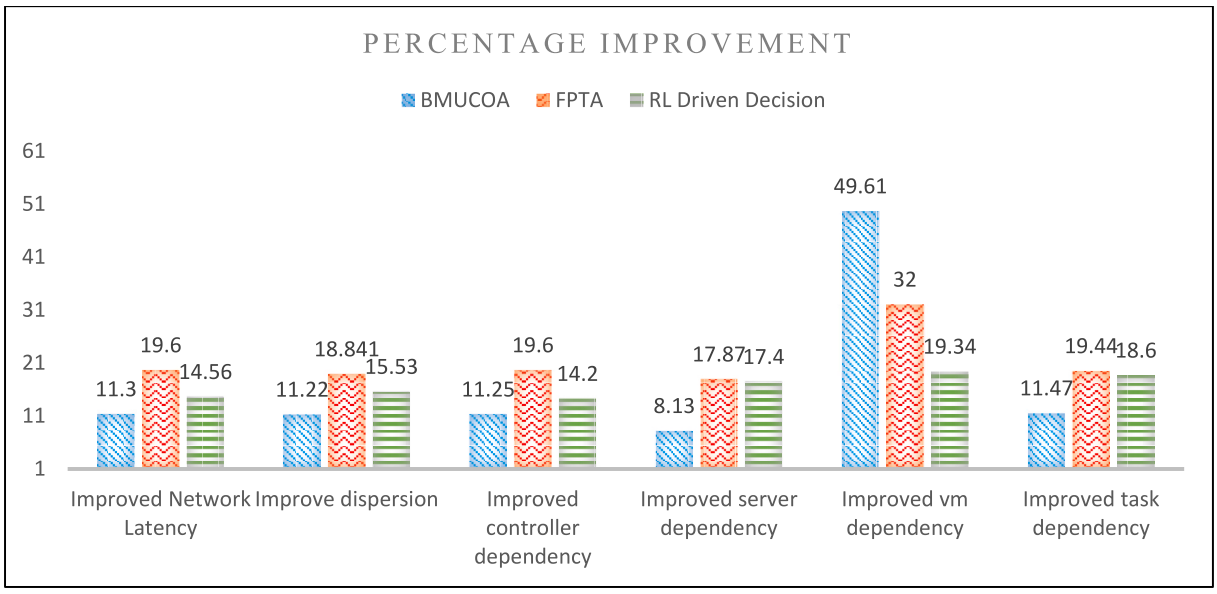

Fig. 40. Comparison of Improvements in the Proposed Method CAAF-DSDN over BMUCOA, FPTA, and RL Driven Decision in Terms of Network Latency.

clustering models across multiple devices or nodes without the need to transfer sensitive data to a central server. This approach not only ensures data privacy and security but also reduces the communication overhead, which is particularly beneficial in scenarios where data privacy is paramount, such as in healthcare, financial services, and smart cities. The application of federated learning could significantly enhance the scalability and computational efficiency of the Fog-DSDN system, (ii) Hybrid AI Techniques for Enhanced Clustering Performance: A critical avenue for improving the clustering performance of Fog-DSDN lies in the combination of deep learning approaches with metaheuristic optimization algorithms such as genetic algorithms, ant colony optimization, and particle swarm optimization. These hybrid models can leverage the strengths of deep learning in feature extraction and pattern recognition, while metaheuristics can be employed to optimize the clustering process. The synergistic use of both techniques can improve the accuracy, scalability, and adaptability of clustering in complex network environments. This approach is particularly effective when dealing with large-scale datasets and dynamic, heterogeneous environments, (iii) Expansion of Application Scenarios: While the current application of CAAF-DSDN has demonstrated significant promise in network load balancing and latency management, its application can be expanded to a wider range of real-world scenarios. Future research could explore the applicability of this approach in IoT networks, smart grid infrastructures, 5G/6G communication networks, and autonomous vehicular networks. Each of these domains presents unique challenges in terms of real-time data processing, load distribution, and resource optimization. Tailoring CAAF-DSDN to these environments will provide valuable insights into its versatility and adaptability, (iv) Algorithm Optimization for Heterogeneous and Dynamic Network Conditions: To further improve the performance of CAAF-DSDN, future research should focus on optimizing the algorithm for heterogeneous network environments that may include different types of servers, varied network conditions, and multiple forms of traffic. Developing specialized techniques to handle dynamic resource allocation, considering factors such as varying network bandwidth, latency, and processing power, is essential. Specifically, enhancing the ability of the algorithm to adapt in real-time to changing traffic patterns (e.g., during peak loads or network failures) will contribute to improved system resilience and efficiency, (v) Real-Time Adaptation and Scalability Enhancements: The ability of CAAF-DSDN to handle increasing amounts of traffic and scale efficiently across larger networks remains a critical area for further investigation. Real-time adaptation mechanisms, capable of detecting and responding to network anomalies, traffic spikes, or device failures, must be integrated into the system to ensure uninterrupted service. These mechanisms should be designed to operate autonomously, adjusting load balancing and resource allocation dynamically based on real-time network conditions. Improving the scalability of the system, particularly in large-scale, distributed network environments, will ensure that CAAF-DSDN can maintain high levels of performance regardless of network size or complexity. The future enhancement of the CAAF-DSDN method holds significant potential for optimizing network performance in diverse and dynamic environments. By integrating federated learning, utilizing hybrid AI techniques, expanding its application domains, optimizing for heterogeneous network conditions, and improving real-time adaptation, the method can be further refined to meet the demands of next-generation networks. The continued development of these research directions will ensure that CAAF-DSDN remains at the forefront of network load balancing and resource optimization, providing robust and scalable solutions for both current and emerging network architectures.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

Abderrahmane, A., Drid, H., & Behaz, A. (2024). A Survey of Controller Placement Problem in SDN-IoT Network. *International Journal of Networked and Distributed Computing*, 1–15.

Albowarab, M. H., Zakaria, N. A., & Zainal Abidin, Z. (2021). Directionally-Enhanced binary multi-objective particle swarm optimisation for load balancing in software defined networks. *Sensors, 21*(10), 3356.

Alhilali, A. H., & Montazerolghaem, A. (2023). Artificial intelligence based load balancing in SDN: A comprehensive survey. *Internet of Things, 22*, Article 100814.

Alqam, S., Alzeidi, N., Touzene, A., & Day, K. (2023). Reinforcement learning-driven decision-making for live virtual machine migration in fog computing. *The Journal of Engineering Research [TJER], 20*(2), 113–122.

Arefian, Z., Khayyambashi, M. R., & Movahhedinia, N. (2023). Delay reduction in MTC using SDN based offloading in Fog computing. *Plos one, 18*(5), Article e0286483.

Ateya, A. A., Muthanna, A., Vybornova, A., Algarni, A. D., Abuarqoub, A., Koucheryavy, Y., & Koucheryavy, A. (2019). Chaotic salp swarm algorithm for SDN multi-controller networks. *Engineering Science and Technology, an International Journal, 22*(4), 1001–1012.

Babbar, H., Parthiban, S., Radhakrishnan, G., & Rani, S. (2022). A genetic load balancing algorithm to improve the QoS metrics for software defined networking for multimedia applications. *Multimedia tools and applications, 81*(7), 9111–9129.

Bakhtiari, S., & Guran Ourimi, V. (2022). A novel approach based on gray wolf evolutionary algorithm for controller load balancing in software defined networks using dynamic switch migration. *Journal of Soft Computing and Information Technology, 11*(1), 32–48.

Batista, E., Figueiredo, G., & Prazeres, C. (2022). Load balancing between fog and cloud in fog of things based platforms through software-defined networking. *Journal of King Saud University-Computer and Information Sciences, 34*(9), 7111–7125.

Chakravarthy, V. D., & Amutha, B. (2022). A novel software-defined networking approach for load balancing in data center networks. *International journal of communication systems, 35*(2), e4213.

Chen, J., Wang, Y., Ou, J., Fan, C., Lu, X., Liao, C., & Zhang, H. (2022). ALBRL: Automatic Load-Balancing Architecture Based on Reinforcement Learning in Software-Defined Networking. *Wireless Communications and Mobile Computing, 2022*(1), Article 3866143.

Chentouf, A., Miyara, M., & Chiba, Z. (2024). *Constrained K-means++ for multi-controller placement load balance. Paper presented at the 2024 International Conference on Circuit, Systems and Communication (ICCSC)*.

Chou, L.-D., Yang, Y.-T., Hong, Y.-M., Hu, J.-K., & Jean, B. (2014). *A genetic-based load balancing algorithm in openflow network*. Paper presented at the Advanced Technologies, Embedded and Multimedia for Human-centric Computing: HumanCom and EMC 2013.

Ebneyousef, S., & Shirmarz, A. (2023). A taxonomy of load balancing algorithms and approaches in fog computing: A survey. *Cluster Computing, 26*(5), 3187–3208.

Ebrahim, M., & Hafid, A. (2023). Resilience and load balancing in fog networks: A multi-criteria decision analysis approach. *Microprocessors and Microsystems, 101*, Article 104893.

El Kamel, A. (2024). Using FlowVisor and evolutionary algorithms to improve the switch migration in SDN. *Journal of network and computer applications, 222*, Article 103807.

Fang, K.-C., Wang, K., & Wang, J.-H. (2016). *A fast and load-aware controller failover mechanism for software-defined networks. Paper presented at the 2016 10th International Symposium on Communication Systems, Networks and Digital Signal Processing (CSNDSP)*.

Farhan, M., Shah, N., Wang, L., Muntean, G.-M., & Song, H. H. (2024). RDG-TE: Link reliability-aware DRL-GNN-based traffic engineering in SDN. *Expert systems with applications*, 125963.

Guan, S., Li, J., Li, Y., & Wang, Z. (2022). A multi-controller placement method for software defined network based on improved firefly algorithm. *Transactions on Emerging Telecommunications Technologies, 33*(7), e4482.

Gunavathie, M., & Umamaheswari, S. (2024). Traffic-aware optimal routing in software defined networks by predicting traffic using neural network. *Expert systems with applications, 239*, Article 122415.

Hamdan, M., Khan, S., Abdelaziz, A., Sadiah, S., Shaikh-Husin, N., Al Otaibi, S., & Marsono, M. N. (2021). DPLBAnt: Improved load balancing technique based on detection and rerouting of elephant flows in software-defined networks. *Computer Communications, 180*, 315–327.

He, X., Ren, Z., Shi, C., & Fang, J. (2016). A novel load balancing strategy of software-defined cloud/fog networking in the Internet of Vehicles. *China Communications, 13*(Supplement2), 140–149.

Hsieh, H.-H., & Wang, K. (2019). *A Simulated Annealing-based Efficient Failover Mechanism for Hierarchical SDN Controllers. Paper presented at the TENCON 2019-2019 IEEE Region 10 Conference (TENCON)*.

Hu, W., Huang, T., Ding, J., Wang, J., Liu, J., & Liu, Y. (2016). Towards consistency of virtual machine migration in Software-Defined Network. *Intelligent Automation & Soft Computing, 22*(4), 551–560.

Ider, M., & Barekatain, B. (2021). An enhanced AHP–TOPSIS-based load balancing algorithm for switch migration in software-defined networks. *The Journal of Supercomputing, 77*, 563–596.

Isyaku, B., & bin Abu Bakar, K., Yusuf, N. M., Abaker, M., Abdelmaboud, A., & Nagmeldin, W.. (2024). Software defined wireless sensor load balancing routing for internet of things applications: Review of approaches. *Heliyon*.

Jasim, A. M., & Al-Raweshidy, H. (2024). An Adaptive SDN-Based Load Balancing Method for Edge/Fog-Based Real-Time Healthcare Systems. *IEEE Systems Journal*.

Javanmardi, S., Shojafar, M., Mohammadi, R., Persico, V., & Pescapè, A. (2023). S-FoS: A secure workflow scheduling approach for performance optimization in SDN-based IoT-Fog networks. *Journal of Information Security and Applications, 72*, Article 103404.

Kang, S.-B., & Kwon, G.-I. (2016). Load balancing of software-defined network controller using genetic algorithm. *Contemporary Engineering Sciences, 9*(18), 881–888.

Kim, S., Ebay, S. K., Lee, B., Kim, K., & Youn, H. Y. (2019). *Load balancing for distributed SDN with harmony search. Paper presented at the 2019 16th IEEE Annual Consumer Communications & Networking Conference (CCNC)*.

Kim, S., Kim, S., Birzhandi, P., Li, P. H., & Youn, H. Y. (2019). *A Solution for Dynamic Controller Provisioning Problem of SDN using Harmony Search. Paper presented at the 2019 8th International Congress on Advanced Applied Informatics (IIAI-AAI)*.

Kulakov, Y., Kohan, A., Kopychko, S., & Cherevatenko, R. (2021). *Load Balancing in Software Defined Networks Using Multipath Routing. Paper presented at the Advances in Computer Science for Engineering and Education, III*, 3.

Kurroliya, K., Mohanty, S., Kanodia, K., & Sahoo, B. (2020). *Grey wolf aware energy-saving and load-balancing in software defined networks considering real time traffic. Paper presented at the 2020 International Conference on Inventive Computation Technologies (ICICT)*.

Lai, W.-K., Wang, Y.-C., Chen, Y.-C., & Tsai, Z.-T. (2022). TSSM: Time-sharing switch migration to balance loads of distributed SDN controllers. *IEEE Transactions on Network and Service Management, 19*(2), 1585–1597.

Lambora, A., Gupta, K., & Chopra, K. (2019). *Genetic algorithm-A literature review. Paper presented at the 2019 international conference on machine learning, big data, cloud and parallel computing (COMITCon)*.

Li, G., Li, K., Liu, Y., & Pan, Y. (2019). An efficient dynamic load balancing scheme based on Nash bargaining in SDN. *Future Internet, 11*(12), 252.

Li, Y., Sun, W., & Guan, S. (2019). *A firefly inspired controller placement algorithm in software defined network. Paper presented at the 2019 IEEE 2nd international conference on computer and communication engineering technology (CCET)*.

Lin, F.-P.-C., & Tsai, Z. (2019). Hierarchical edge-cloud SDN controller system with optimal adaptive resource allocation for load-balancing. *IEEE Systems Journal, 14*(1), 265–276.

Lin, W., & Zhang, L. (2016). *The load balancing research of SDN based on ant colony algorithm with job classification. Paper presented at the 2016 2nd Workshop on Advanced Research and Technology in Industry Applications (WARTIA-16)*.

Mahlab, U., Omiyi, P. E., Hundert, H., Wolbrum, Y., Elimelech, O., Aharon, I., & Zarakovsky, S. (2017). *Entropy-based load-balancing for software-defined elastic optical networks. Paper presented at the 2017 19th International Conference on Transparent Optical Networks (ICTON)*.

Mahmoudi, M., Avokh, A., & Barekatain, B. (2022). SDN-DVFS: An enhanced QoS-aware load-balancing method in software defined networks. *Cluster Computing, 25*(2), 1237–1262.

Mahmoudi, M., Barekatain, B., Beheshti, Z., & Quintana, A. A. (2024). MBL-DSDN: A novel load balancing algorithm in distributed software-defined networks based on micro-clustering and B-LSTM methods. *The Journal of Supercomputing*, 1–67.

Malakar, M., Mahapatro, J., & Ghosh, T. (2024). A survey on routing and load-balancing mechanisms in software-defined vehicular networks. *Wireless Networks*, 1–17.

Min, Z., Sun, H., Bao, S., Gokhale, A. S., & Gokhale, S. S. (2021). *A self-adaptive load balancing approach for software-defined networks in iot. Paper presented at the 2021 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*.

Minardi, M., Vu, T. X., Maity, I., Politis, C., & Chatzinotas, S. (2024). Traffic-Aware Virtual Network Embedding With Joint Load Balancing and Datarate Assignment for SDN-Based Networks. *IEEE Transactions on Network and Service Management*.

Montazerolghaem, A., & Yaghmaee, M. H. (2020). Load-balanced and QoS-aware software-defined Internet of Things. *IEEE Internet of things Journal, 7*(4), 3323–3337.

Nagasri, B., & Anbarasu, V. (2022). *An Optimization Analysis on Ant Colony and Particle Swarm for Load Balancing Techniques and Policies with Software Defined Networks (SDN). Paper presented at the 2022 International Conference on Innovative Computing, Intelligent Communication and Smart Electrical Systems (ICSES)*.

Neghabi, A. A., Navimipour, N. J., Hosseinzadeh, M., & Rezaee, A. (2020). Energy-aware dynamic-link load balancing method for a software-defined network using a multi-objective artificial bee colony algorithm and genetic operators. *IET communications, 14*(18), 3284–3293.

Obaida, T. H., & Salman, H. A. (2022). A novel method to find the best path in SDN using firefly algorithm. *Journal of Intelligent Systems, 31*(1), 902–914.

Perepelkin, D., & Nguyen, T. (2022). *Research of multipath routing and load balancing processes in software defined networks based on artificial bee colony algorithm. Paper presented at the 2022 ELEKTRO (ELEKTRO)*.

Poularakis, K., Qin, Q., Nahum, E. M., Rio, M., & Tassiulas, L. (2019). Flexible SDN control in tactical ad hoc networks. *Ad Hoc Networks, 85*, 71–80.

Prabakaran, S., & Ramar, R. (2021). Software defined network: Load balancing algorithm design and analysis. *Int. Arab J. Inf. Technol.,, 18*(3), 312–318.

Rezaee, M. R., Hamid, N. A. W. A., Hussin, M., & Zukarnain, Z. A. (2024). *Fog Offloading and Task Management in IoT-Fog-Cloud Environment: Review of Algorithms*. IEEE Access: Networks and SDN Application.

Rostami, M., & Goli-Bidgoli, S. (2024). An overview of QoS-aware load balancing techniques in SDN-based IoT networks. *Journal of Cloud Computing, 13*(1), 89.

Salehnia, T., Montazerolghaem, A., Mirjalili, S., Khayyambashi, M. R., & Abualigah, L. (2024). *SDN-based optimal task scheduling method in Fog-IoT network using combination of AO and WOA. In Handbook of Whale Optimization Algorithm* (pp. 109–128). Elsevier.

Sathyanarayana, S., & Moh, M. (2016). *Joint route-server load balancing in software defined networks using ant colony optimization. Paper presented at the 2016 international conference on high performance computing & simulation (HPCS)*.

Shahrbabaki, P. P., Coutinho, R. W., & Shayan, Y. R. (2024). SDN-LB: A novel server workload balancing algorithm for IoT video analytics. *Ad Hoc Networks, 155*, Article 103398.

Sharma, S., & Seth, D. (2024). Blue monkey updated chimp optimization algorithm for enhanced load balancing model. *Expert systems with applications, 242*, Article 122578.

Sikder, R., Hossain, M. S., Alenazi, M. J., & Atiquzzaman, M. (2024). A relaxed and faster switch migration framework to balance the load of distributed control plane in software defined networks. *Cluster Computing*, 1–18.

Singh, G. D., Tripathi, V., Dumka, A., Rathore, R. S., Bajaj, M., Escorcia-Gutierrez, J., & Prokop, L. (2024). A novel framework for capacitated SDN controller placement: Balancing latency and reliability with PSO algorithm. *Alexandria Engineering Journal, 87*, 77–92.

Sridevi, K., & Saifulla, M. A. (2023). LBABC: Distributed controller load balancing using artificial bee colony optimization in an SDN. *Peer-to-Peer Networking and Applications, 16*(2), 947–957.

Sun, P., Guo, Z., Wang, G., Lan, J., & Hu, Y. (2020). MARVEL: Enabling controller load balancing in software-defined networks with multi-agent reinforcement learning. *Computer Networks, 177*, Article 107230.

Tache, M. D., Păscuțoiu, O., & Borcoci, E. (2024). Optimization Algorithms in SDN: Routing, Load Balancing, and Delay Optimization. *Applied Sciences, 14*(14), 5967.

Tamri, R., Antari, J., & Iqdour, R. (2024). A New WRR Algorithm for an Efficient Load Balancing System in IoT Networks under SDN. *International Journal of Interactive Mobile Technologies, 18*(3).

Wang, C., Zhang, G., Xu, H., & Chen, H. (2016). *An ACO-based link load-balancing algorithm in SDN. Paper presented at the 2016 7th International Conference on Cloud Computing and Big Data (CCBD)*.

Wang, H., Li, Y., Zhang, Y., & Jin, D. (2017). Virtual machine migration planning in software-defined networks. *IEEE Transactions on Cloud Computing, 7*(4), 1168–1182.

Wang, H., Xu, H., Qian, C., Ge, J., Liu, J., & Huang, H. (2020). PrePass: Load balancing with data plane resource constraints using commodity SDN switches. *Computer Networks, 178*, Article 107339.

Xiao, J., Pan, X., Liu, J., Wang, J., Zhang, P., & Abualigah, L. (2024). Load balancing strategy for SDN multi-controller clusters based on load prediction. *The Journal of Supercomputing, 80*(4), 5136–5162.

Xue, H., Kim, K. T., & Youn, H. Y. (2019). Dynamic load balancing of software-defined networking based on genetic-ant colony optimization. *Sensors, 19*(2), 311.

Zhong, H., Fang, Y., & Cui, J. (2018). Reprint of "LBBSRT: An efficient SDN load balancing scheme based on server response time". *Future Generation Computer Systems, 80*, 409–416.

Zhou, Y., Ren, B., Xie, J., Luo, L., Guo, D., & Zhou, X. (2023). Enable the proactively load-balanced control plane for SDN via intelligent switch-to-controller selection strategy. *Computer Networks, 233*, Article 109867.

Zhou, Y., & Wu, W. (2024). *Load-balancing system for SDN data center based on improved ant colony optimization.* Paper presented at the International Conference on Computer Network Security and Software Engineering (CNSSE 2024).